

**LAPORAN AKHIR PRAKTIKUM
ALGORITMA DAN STRUKTUR DATA**



Disusun oleh :

Vivaldi Fachmy Fahlevi

NIM.2510817210017

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2026**

LEMBAR PENGESAHAN
LAPORAN AKHIR PRAKTIKUM ALGORITMA DAN
STRUKTUR DATA

Laporan Akhir Praktikum Algoritma Dan Struktur Data ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma Dan Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikum : Vivaldi Fachmy Fahlevi
NIM : 2510817210017

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir.Muhmmad Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	i
DAFTAR ISI	ii
DAFTAR GAMBAR	vi
DAFTAR TABEL.....	viii
MODUL 1 : Struct dan Pointer	1
SOAL 1	1
A. Source Code	1
B. Output Program.....	4
C. Pembahasan.....	5
SOAL 2	7
A. Source Code	7
B. Output Program.....	11
C. Pembahasan.....	12
MODUL 2 : Stack dan Queue	14
SOAL 1	14
A. Source Code	14
B. Pembahasan.....	17
SOAL 2	20
A. Source Code	23
B. Output Program.....	25
C. Pembahasan.....	26
SOAL 3	28
A. Source Code	28
B. Pembahasan.....	31
SOAL 4	34
A. Source Code	36
B. Output Program.....	38
C. Pembahasan.....	39
MODUL 3 : Linked List dan Double Linked List	41
SOAL 1 : Implementasi Senarai Berantai Tunggal Melingkar.....	41

A. Source Code	41
B. Output Program.....	52
C. Pembahasan.....	53
SOAL 2 : Si Palui dan Relikui Siklus Resonansi.....	60
A. Source Code	62
B. Output Program.....	65
C. Pembahasan.....	66
SOAL 3	69
A. Source Code	69
B. Output Program.....	80
C. Pembahasan.....	81
SOAL 4	83
A. Source Code	85
B. Output Program.....	90
C. Pembahasan.....	91
MODUL 4 : Sort	93
SORTING ALGORITHM	93
A. Source Code	93
B. Output Program.....	111
C. Pembahasan.....	113
SOAL 1	131
A. Bubble Sort	132
B. Selection Sort	134
C. Insertion Sort.....	136
D. Merge Sort	138
E. Quick Sort.....	140
F. Radix Sort.....	142
SOAL 2	144
A. Bubble Sort	145
B. Selection Sort	147
C. Insertion Sort.....	149
D. Merge Sort	151
E. Quick Sort.....	153

F. Radix Sort.....	156
SOAL 3	160
A. Bubble Sort	161
B. Selection Sort	163
C. Insertion Sort.....	165
D. Merge Sort	167
E. Quick Sort.....	169
F. Radix Sort.....	171
SOAL 4	173
MODUL 5 : Search	176
Searching Algorithm	176
A. Source Code	176
B. Output Program.....	184
C. Pembahasan.....	185
Linear Search	199
Binary Search	203
MODUL 6 : Graph dan Tree	209
Soal 1 : Konversi Adjacency List ke Adjacency Matrix.....	209
A. Source Code	212
B. Output Program.....	214
C. Pembahasan.....	215
Soal 2: Konversi Adjacency Matrix ke Adjacency List.....	217
A. Source Code	219
B. Output Program.....	221
C. Pembahasan.....	222
Soal 3: BFS: Jarak Terpendek Keluar Labirin	225
A. Source Code	228
B. Output Program.....	230
C. Pembahasan.....	231
Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin	234
A. Source Code	237
B. Output Program.....	239
C. Pembahasan.....	240

Soal 5: Tree Traversal	243
A. Source Code	245
B. Output Program	259
C. Pembahasan	260
TAUTAN GITHUB	273

DAFTAR GAMBAR

MODUL 1 : Struct dan Pointer

Gambar 1 Screenshot Output Program Soal 1	4
Gambar 2 Screenshot Output Program Soal 2	11

MODUL 2 : Stack dan Queue

Gambar 3 Screenshot Output Program Soal 2 File prob2.cpp	25
Gambar 4 Screenshot Output Program Soal 4 File prob2.cpp	38

MODUL 3 : Linked List dan Double Linked List

Gambar 5 Screenshot Output Program Soal 1	52
Gambar 6 Screenshot Output Program Soal 2	65
Gambar 7 Screenshot Output Program Soal 3	80
Gambar 8 Output Program Soal 4	90

MODUL 4 : Sort

Gambar 9 Hasil Pengujian Bubble Sort	111
Gambar 10 Hasil Pengujian Selection Sort	111
Gambar 11 Hasil Pengujian Insertion Sort	111
Gambar 12 Hasil Pengujian Merge Sort	112
Gambar 13 Hasil Pengujian Quick Sort	112
Gambar 14 Hasil Pengujian Radix Sort	112
Gambar 15 Hasil Pengujian Bubble Sort	132
Gambar 16 Hasil Pengujian Selection Sort	134
Gambar 17 Hasil Pengujian Insertion Sort	136
Gambar 18 Hasil Pengujian Merge Sort	138
Gambar 19 Hasil Pengujian Quick Sort	140
Gambar 20 Hasil Pengujian Radix Sort	142

MODUL 5 : Search

Gambar 21 Hasil Pengujian Linear Search	184
Gambar 22 Hasil Pengujian Binary Search	184

MODUL 6 : Graph dan Tree

Gambar 23 Screenshot Output Program Soal 1	214
Gambar 24 Screenshot Output Program Soal 2	221
Gambar 25 Screenshot Output Program Soal 3	230
Gambar 26 Screenshot Output Program Soal 4	239
Gambar 27 Screenshot Output Program Soal 5	259

DAFTAR TABEL

MODUL 1 : Struct dan Pointer

Tabel 1 Source Code File point.h	1
Tabel 2 Source Code File line.h	2
Tabel 3 Source Code File line.cpp	2
Tabel 4 Source Code File prob1.cpp	3
Tabel 5 Source Code File point.h	7
Tabel 6 Source Code File circle.h	8
Tabel 7 Source Code File prob2.cpp	9
Tabel 8 Source Code File circle.cpp	10

MODUL 2 : Stack dan Queue

Tabel 9 Source Code File stack.h	14
Tabel 10 Source Code File stack.cpp	15
Tabel 11 Source Code File prob1.cpp	23
Tabel 12 Source Code File queue.h	28
Tabel 13 Source Code File queue.cpp	29
Tabel 14 Source Code File prob2.cpp	36

MODUL 3 : Linked List dan Double Linked List

Tabel 15 Source Code File single_linked_list.h	41
Tabel 16 Source Code File single_linked_list.cpp	42
Tabel 17 Source Code File test_single.cpp	50
Tabel 18 Source Code File prob_a.cpp	62
Tabel 19 Source Code File double_linked_list.h	69
Tabel 20 Source Code File double_linked_list.h	70
Tabel 21 Source Code File test_double.cpp	78
Tabel 22 Source Code File prob_b.cpp	85

MODUL 4 : Sort

Tabel 23 Source Code file algorithm.cpp	93
Tabel 24 Source Code File sorting_algorithms.h	102

Tabel 25 Source Code File metric.h	102
Tabel 26 Source Code File report.h.....	103
Tabel 27 Source Code File reporter.cpp	104
Tabel 28 Source Code File main.cpp	105
Tabel 29 Source Code File data_generator.h.....	108
Tabel 30 Source Code File generator.cpp	108

MODUL 5 : Search

Tabel 31 Source Code File searching_algorithms.h.....	176
Tabel 32 Source Code File searching_algorithms.cpp.....	176
Tabel 33 Source Code File metrics.h	178
Tabel 34 Source Code File reporter.h.....	179
Tabel 35 Source Code File reporter.cpp	179
Tabel 36 Source Code File data_generator.h.....	180
Tabel 37 Source Code File data_generator.cpp.....	181

MODUL 6 : Graph dan Tree

Tabel 38. Source Code File prob1.cpp	212
Tabel 39 Source Code File prob2.cpp	219
Tabel 40 Source Code File prob3.cpp	228
Tabel 41 Source Code File prob4.cpp	237
Tabel 42 Source Code File RedBlackTree.h	245
Tabel 43 Source Code File RedBlackTree.cpp	246
Tabel 44 Source Code File prob5.cpp	254

MODUL 1 : Struct dan Pointer

SOAL 1

Diberikan sebuah garis l yang direpresentasikan dalam bentuk persamaan umum:

$$ax + by + c = 0$$

dan sebuah titik $P = (x_p, y_p)$. Tentukan posisi titik P terhadap garis l dengan cara mengimplementasikan fungsi `gradient` dan `CheckPointPosition` pada file header yang telah diberikan.

Input	Output
1 0 -3 5 0	Left
1 0 -3 1 7	Right
2 -1 -1 1 1	On Line

A. Source Code

Tabel 1 Source Code File point.h

1	<code>#ifndef POINT_H</code>
2	<code>#define POINT_H</code>
3	
4	<code>struct Point {</code>
5	<code> double x;</code>
6	<code> double y;</code>
7	<code>};</code>
8	
9	<code>#endif</code>

Tabel 2 Source Code File line.h

1	#ifndef LINE_H
2	#define LINE_H
3	
4	#include "point.h"
5	#include <string>
6	
7	struct Line {
8	double a;
9	double b;
10	double c;
11	};
12	
13	inline constexpr double EPSILON = 1e-9;
14	
15	double gradient(const Line *l, const Point *p);
16	std::string CheckPointPosition(double value);
17	
18	#endif

Tabel 3 Source Code File line.cpp

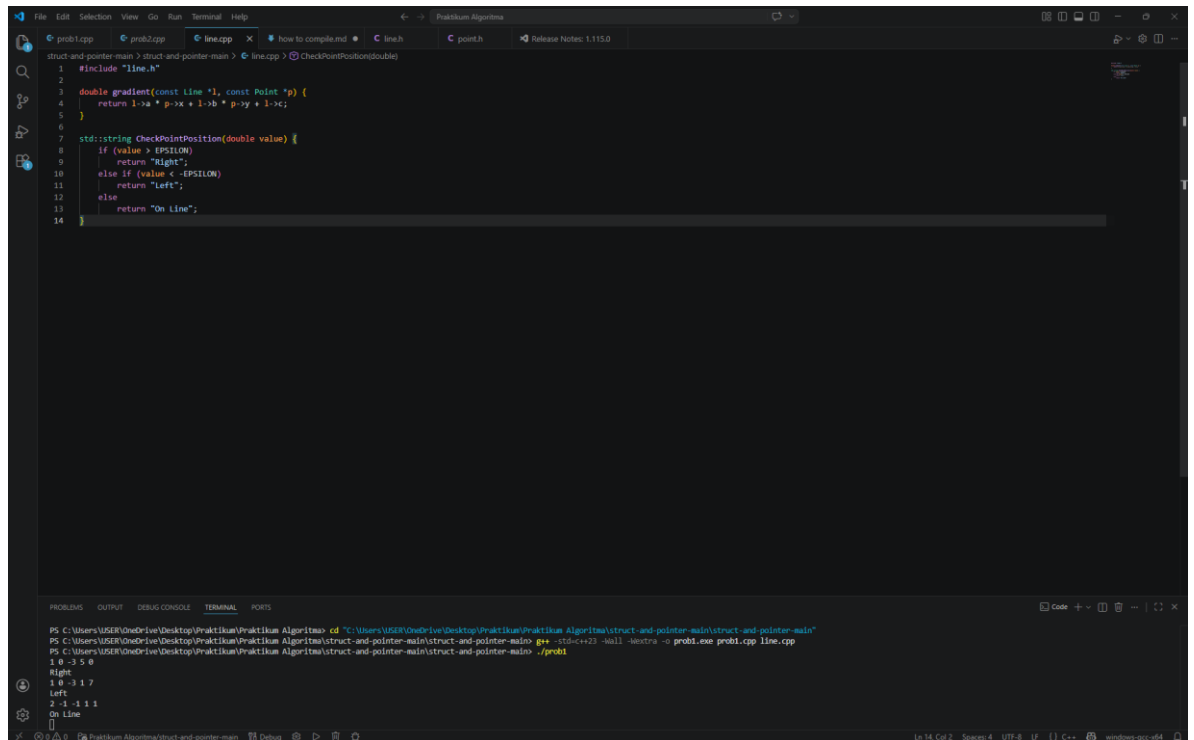
1	#include "line.h"
2	
3	double gradient(const Line *l, const Point *p) {
4	return l->a * p->x + l->b * p->y + l->c;
5	}
6	
7	std::string CheckPointPosition(double value) {
8	if (value > EPSILON)
9	return "Right";
10	else if (value < -EPSILON)
11	return "Left";

12	else
13	return "On Line";
14	}

Tabel 4 Source Code File prob1.cpp

1	#include <iostream>
2	#include "line.h"
3	#include "point.h"
4	
5	using namespace std;
6	
7	int main() {
8	double a, b, c, px, py;
9	while (cin >> a >> b >> c >> px >> py) {
10	Line l = {a, b, c};
11	Point p = {px, py};
12	
13	double hasil = gradient(&l, &p);
14	cout << CheckPointPosition(hasil) << endl;
15	}
16	
17	return 0;
18	}

B. Output Program



The screenshot shows a C++ IDE with a dark theme. The editor displays the following code:

```
1 #include "line.h"
2
3 double gradient(const Line *l, const Point *p) {
4     return l->a * p->x + l->b * p->y + l->c;
5 }
6
7 std::string CheckPointPosition(double value) {
8     if (value > EPSILON)
9         return "Right";
10    else if (value < -EPSILON)
11        return "Left";
12    else
13        return "On Line";
14 }
```

The terminal at the bottom shows the compilation and execution process:

```
PS C:\Users\User\OneDrive\Desktop\Praktikum\Praktikum Algoritma> cd "C:\Users\User\OneDrive\Desktop\Praktikum\Praktikum Algoritma\struct-and-pointer-main\struct-and-pointer-main"
PS C:\Users\User\OneDrive\Desktop\Praktikum\Praktikum Algoritma\struct-and-pointer-main\struct-and-pointer-main> g++ -std=c++20 -Wall -std=c++20 -o prob1.exe prob1.cpp line.cpp
1 0 -3 5 0
Right
1 0 -3 1 7
Left
2 -1 -1 1 1
On Line
[]
```

Gambar 1 Screenshot Output Program Soal 1

C. Pembahasan

File point.h digunakan untuk mendefinisikan struktur data yang merepresentasikan sebuah titik dalam koordinat dua dimensi. Pada baris [1–2] terdapat *header guard* (`#ifndef`, `#define`) yang berfungsi untuk mencegah file ini diproses lebih dari satu kali saat proses kompilasi. Hal ini penting agar tidak terjadi duplikasi definisi. Pada baris [4–7] didefinisikan sebuah struct `Point` yang memiliki dua atribut yaitu `x` dan `y` bertipe `double`, yang merepresentasikan posisi titik pada sumbu koordinat. Terakhir, pada baris [9] terdapat `#endif` yang menutup *header guard*. File ini tidak memiliki fungsi, hanya sebagai penyedia tipe data yang akan digunakan oleh file lain.

File line.h berfungsi sebagai header yang mendeklarasikan struktur garis dan fungsi-fungsi yang akan digunakan untuk menentukan posisi titik terhadap garis. Pada baris [1–2] digunakan *header guard* untuk menghindari duplikasi. Baris [4] menyertakan file `point.h` karena struktur `Line` akan digunakan bersama dengan `Point`, dan baris [5] menyertakan pustaka `<string>` untuk penggunaan tipe data `string`. Pada baris [7–11] didefinisikan struct `Line` dengan tiga parameter yaitu `a`, `b`, dan `c` yang merepresentasikan persamaan garis umum $ax + by + c = 0$. Pada baris [13] dideklarasikan konstanta `EPSILON` yang digunakan sebagai toleransi untuk perbandingan bilangan desimal agar lebih akurat. Selanjutnya pada baris [15–16] terdapat deklarasi dua fungsi, yaitu `gradient()` untuk menghitung nilai substitusi titik ke persamaan garis, dan `CheckPointPosition()` untuk menentukan posisi titik berdasarkan hasil perhitungan tersebut.

File line.cpp berisi implementasi dari fungsi-fungsi yang telah dideklarasikan di `line.h`. Pada baris [1] dilakukan `#include "line.h"` agar file ini mengetahui deklarasi yang akan diimplementasikan. Fungsi `gradient` pada baris [3–5] menerima parameter pointer ke `Line` dan `Point`, lalu menghitung nilai dari persamaan $ax + by + c$ dengan cara mengakses nilai melalui pointer (`l->a`, `p->x`, dll). Nilai ini digunakan untuk menentukan posisi titik terhadap garis. Selanjutnya, fungsi `CheckPointPosition` pada baris [7–13] digunakan untuk mengklasifikasikan posisi titik. Jika nilai lebih besar dari `EPSILON`, maka titik berada di sebelah kanan garis (*Right*). Jika nilai lebih kecil dari negatif

EPSILON, maka titik berada di sebelah kiri (*Left*). Jika nilainya mendekati nol (dalam rentang EPSILON), maka titik berada tepat pada garis (*On Line*). Penggunaan EPSILON penting untuk menghindari kesalahan akibat pembulatan angka desimal.

File prob1.cpp merupakan program utama yang menjalankan keseluruhan logika. Pada baris [1–3] dilakukan *include* terhadap pustaka standar *iostream* serta file *line.h* dan *point.h* agar dapat menggunakan fungsi dan struktur yang telah dibuat sebelumnya. Baris [5] menggunakan *using namespace std* untuk mempermudah penulisan tanpa harus menambahkan *std::*. Pada baris [7] dimulai fungsi *main()* sebagai titik awal eksekusi program. Baris 8 mendeklarasikan variabel input yaitu *a*, *b*, *c* untuk garis, serta *px*, *py* untuk titik. Pada baris [9] digunakan perulangan *while(cin >> ...)* untuk membaca input secara berulang. Baris [10–11] membuat objek *Line* dan *Point* berdasarkan input yang diberikan. Pada baris [13] dilakukan pemanggilan fungsi *gradient()* untuk menghitung hasil substitusi titik ke persamaan garis. Kemudian pada baris [14] hasil tersebut dikirim ke fungsi *CheckPointPosition()* untuk menentukan posisi titik, dan hasilnya ditampilkan ke layar. Program akan terus berjalan selama masih ada input, dan diakhiri dengan *return 0* pada baris [17].

SOAL 2

Diberikan sebuah lingkaran C dengan pusat (c_x, c_y) dan jari-jari $r > 0$, serta sebuah titik $P = (p_x, p_y)$. Tentukan posisi titik P terhadap lingkaran C dan mengimplementasikan fungsi `distance` dan `CheckPointInCircle` pada file header yang diberikan.

Input	Output
0 0 5 3 4	On Circle
0 0 5 1 1	Inside
0 0 5 4 4	Outside
-2 -2 4 2 -2	On Circle

A. Source Code

Tabel 5 Source Code File point.h

1	<code>#ifndef LINE_H</code>
2	<code>#define LINE_H</code>
3	
4	<code>#include "point.h"</code>
5	<code>#include <string></code>
6	
7	<code>struct Line {</code>
8	<code> double a;</code>
9	<code> double b;</code>

Tabel 6 Source Code File circle.h

1	#ifndef CIRCLE_H
2	#define CIRCLE_H
3	
4	#include "point.h"
5	#include <string>
6	
7	struct Circle {
8	Point centre;
9	double radius;
10	};
11	
12	inline constexpr double EPSILON = 1e-9;
13	
14	double distance(const Circle *c, const Point *p);
15	std::string CheckPointInCircle(double distance,
	double radius);
16	
17	#endif

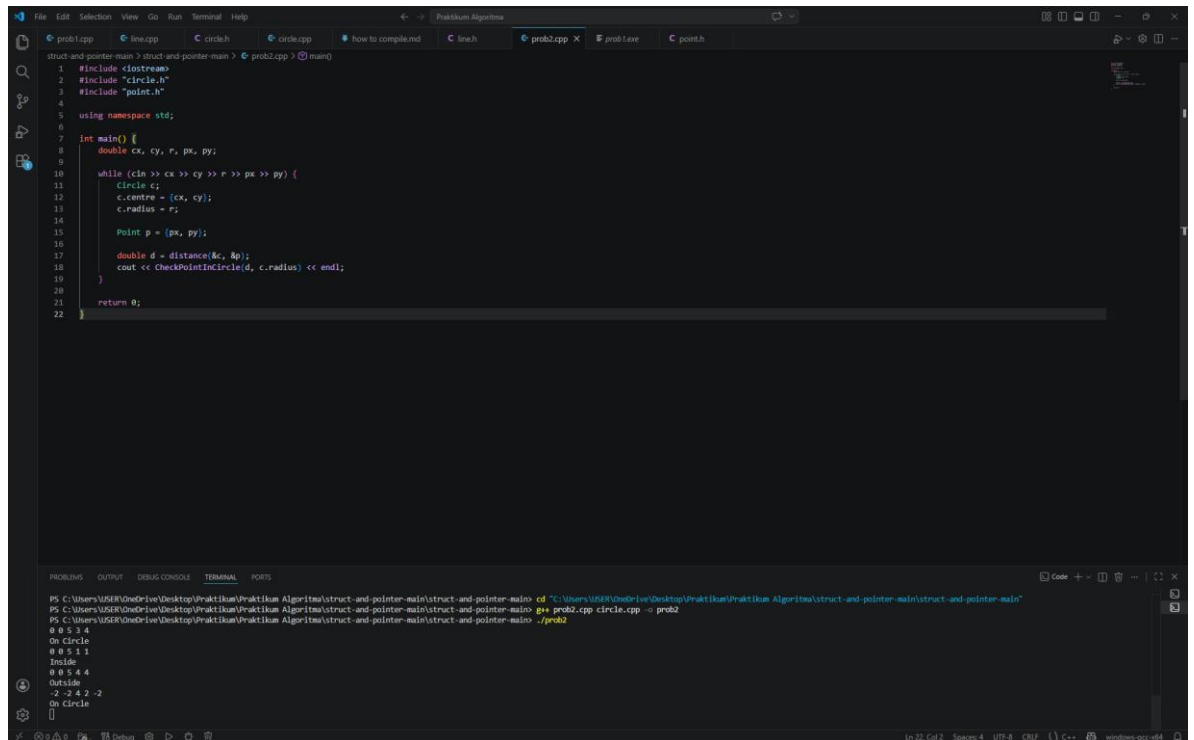
Tabel 7 Source Code File prob2.cpp

1	#include <iostream>
2	#include "circle.h"
3	#include "point.h"
4	
5	using namespace std;
6	
7	int main() {
8	double cx, cy, r, px, py;
9	
10	while (cin >> cx >> cy >> r >> px >> py) {
11	Circle c;
12	c.centre = {cx, cy};
13	c.radius = r;
14	
15	Point p = {px, py};
16	
17	double d = distance(&c, &p);
18	cout << CheckPointInCircle(d, c.radius) <<
	endl;
19	}
20	
21	return 0;
22	}

Tabel 8 Source Code File circle.cpp

1	#include "circle.h"
2	#include <cmath>
3	
4	double distance(const Circle *c, const Point *p) {
5	return sqrt((p->x - c->centre.x) * (p->x - c->centre.x) +
6	(p->y - c->centre.y) * (p->y - c->centre.y));
7	}
8	
9	std::string CheckPointInCircle(double dist, double r) {
10	if (fabs(dist - r) <= EPSILON)
11	return "On Circle";
12	else if (dist < r)
13	return "Inside";
14	else
15	return "Outside";
16	}

B. Output Program



```
File Edit Selection View Go Run Terminal Help
struct-and-pointer-main > struct-and-pointer-main > prob2.cpp > main()
1 #include <iostream>
2 #include "circle.h"
3 #include "point.h"
4
5 using namespace std;
6
7 int main() {
8     double cx, cy, r, px, py;
9
10    while (cin >> cx >> cy >> r >> px >> py) {
11        Circle c;
12        c.centre = {cx, cy};
13        c.radius = r;
14
15        Point p = {px, py};
16
17        double d = distance(&c, &p);
18        cout << CheckPointInCircle(d, c.radius) << endl;
19    }
20
21    return 0;
22 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\USER\OneDrive\Desktop\Praktikum Algoritma> struct-and-pointer-main cd "C:\Users\USER\OneDrive\Desktop\Praktikum Algoritma\struct-and-pointer-main\struct-and-pointer-main"
PS C:\Users\USER\OneDrive\Desktop\Praktikum Algoritma> struct-and-pointer-main g++ prob2.cpp circle.cpp -o prob2
PS C:\Users\USER\OneDrive\Desktop\Praktikum Algoritma> struct-and-pointer-main .\prob2
0 0 5 3 4
On Circle
0 0 5 1 1
Inside
0 0 5 4 4
Outside
-2 -2 4 2 2
On Circle
0
```

Gambar 2 Screenshot Output Program Soal 2

C. Pembahasan

File point.h digunakan untuk mendefinisikan struktur data yang merepresentasikan sebuah titik dalam koordinat dua dimensi. Pada baris [1–2] terdapat *header guard* (`#ifndef`, `#define`) yang berfungsi untuk mencegah file ini diproses lebih dari satu kali saat proses kompilasi. Hal ini penting agar tidak terjadi duplikasi definisi. Pada baris [4–7] didefinisikan sebuah struct `Point` yang memiliki dua atribut yaitu `x` dan `y` bertipe `double`, yang merepresentasikan posisi titik pada sumbu koordinat. Terakhir, pada baris [9] terdapat `#endif` yang menutup *header guard*. File ini tidak memiliki fungsi, hanya sebagai penyedia tipe data yang akan digunakan oleh file lain.

File circle.h merupakan header file yang digunakan untuk mendefinisikan struktur data lingkaran serta mendeklarasikan fungsi yang akan digunakan dalam program. Pada baris [1–2] terdapat *header guard* (`#ifndef` dan `#define`) yang berfungsi untuk mencegah file ini diproses lebih dari satu kali saat kompilasi. Pada baris [4] file `point.h` disertakan karena struktur `Circle` menggunakan tipe data `Point` sebagai pusat lingkaran, sedangkan pada baris [5] disertakan pustaka `<string>` untuk penggunaan tipe data string. Baris [7–10] mendefinisikan struct `Circle` yang terdiri dari `centre` (titik pusat lingkaran) dan `radius` (jari-jari lingkaran). Pada baris [12] terdapat konstanta `EPSILON` yang digunakan sebagai toleransi dalam perbandingan bilangan desimal agar hasil lebih akurat. Selanjutnya pada baris [14–15] terdapat deklarasi dua fungsi, yaitu `distance()` untuk menghitung jarak antara titik dengan pusat lingkaran, dan `CheckPointInCircle()` untuk menentukan posisi titik apakah berada di dalam, di luar, atau tepat pada lingkaran.

File circle.cpp berisi implementasi dari fungsi-fungsi yang telah dideklarasikan pada `circle.h`. Pada baris [1] dilakukan `#include "circle.h"` agar file ini dapat menggunakan struktur dan deklarasi fungsi yang telah dibuat sebelumnya, dan pada baris [2] ditambahkan `<cmath>` untuk menggunakan fungsi matematika seperti `sqrt` dan `fabs`. Fungsi `distance` pada baris [4–7] digunakan untuk menghitung jarak antara titik dan pusat lingkaran menggunakan rumus Pythagoras, yaitu :

$$d = \sqrt{(x_p - c_x)^2 + (y_p - c_y)^2}$$

Rumus ini berasal dari konsep jarak dua titik pada bidang kartesius, yaitu selisih koordinat x dan y dikuadratkan, kemudian dijumlahkan dan diakarkan. Implementasi pada kode menghitung bagian dalam akar terlebih dahulu, lalu menggunakan fungsi `sqrt()` untuk mendapatkan nilai akhirnya. Nilai jarak tersebut kemudian digunakan oleh fungsi `CheckPointInCircle` pada baris [9–15] untuk menentukan posisi titik terhadap lingkaran. Jika selisih antara jarak dan jari-jari sangat kecil (dalam batas EPSILON), maka titik berada tepat pada lingkaran (*On Circle*). Jika jarak lebih kecil dari jari-jari, maka titik berada di dalam lingkaran (*Inside*), dan jika lebih besar maka berada di luar lingkaran (*Outside*). Penggunaan EPSILON penting untuk menghindari kesalahan akibat pembulatan angka desimal.

File prob2.cpp merupakan program utama yang menjalankan seluruh logika pengecekan posisi titik terhadap lingkaran. Pada baris [1–3] dilakukan *include* terhadap pustaka `iostream` serta file `circle.h` dan `point.h` agar dapat menggunakan struktur dan fungsi yang telah dibuat. Pada baris [5] digunakan `using namespace std` untuk mempermudah penulisan kode. Baris [7] merupakan awal dari fungsi `main()` sebagai titik masuk program. Pada baris 8 dideklarasikan variabel input yaitu `cx`, `cy` sebagai pusat lingkaran, `r` sebagai jari-jari, serta `px`, `py` sebagai koordinat titik. Baris [10] menggunakan perulangan `while(cin >> ...)` untuk membaca input secara berulang. Pada baris [11–13] dibuat objek `Circle` dan diisi dengan nilai pusat dan radius, sedangkan pada baris [15] dibuat objek `Point` untuk titik yang akan dicek. Selanjutnya pada baris [17] dipanggil fungsi `distance()` untuk menghitung jarak titik ke pusat lingkaran, dan pada baris [18] hasil tersebut dikirim ke fungsi `CheckPointInCircle()` untuk menentukan posisi titik. Hasilnya kemudian ditampilkan ke layar. Program akan terus berjalan selama masih ada input, dan diakhiri dengan `return 0` pada baris [21].

MODUL 2 : Stack dan Queue

SOAL 1

Pada file `stack.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `stack.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`. Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 9 Source Code File stack.h

1	<code>#ifndef STACK_H</code>
2	<code>#define STACK_H</code>
3	
4	<code>#define MAX 100</code>
5	
6	<code>struct Stack {</code>
7	<code> int data[MAX];</code>
8	<code> int* top;</code>
9	<code>};</code>
10	
11	<code>void init(Stack* s);</code>
12	<code>bool isEmpty(const Stack* s);</code>
13	<code>bool isFull(const Stack* s);</code>
14	<code>void push(Stack* s, int value);</code>
15	<code>void pop(Stack* s);</code>
16	<code>int peek(const Stack* s);</code>
17	
18	<code>#endif</code>

Tabel 10 Source Code File stack.cpp

1	#include "stack.h"
2	#include <stdexcept>
3	
4	void init(Stack* s) {
5	s->top = s->data - 1;
6	}
7	
8	bool isEmpty(const Stack* s) {
9	return s->top < s->data;
10	}
11	
12	bool isFull(const Stack* s) {
13	return s->top == s->data + MAX - 1;
14	}
15	
16	void push(Stack* s, int value) {
17	if (isFull(s)) {
18	throw std::overflow_error("Stack penuh");
19	}
20	s->top++;
21	*(s->top) = value;
22	}
23	
24	void pop(Stack* s) {
25	if (isEmpty(s)) {
26	throw std::underflow_error("Stack kosong");
27	}
28	s->top--;

```
29 }  
30  
31 int peek(const Stack* s) {  
32     if (isEmpty(s)) {  
33         throw std::underflow_error("Stack kosong");  
34     }  
35     return *(s->top);  
36 }
```

B. Pembahasan

File `stack.h`

Pada file `stack.h`, didefinisikan struktur dasar dari stack beserta deklarasi fungsi-fungsi yang akan digunakan. Pada baris [4], terdapat konstanta `MAX` yang menentukan kapasitas maksimum stack yaitu 100 elemen. Hal ini berarti stack memiliki ukuran tetap (statis) dan tidak dapat menampung lebih dari jumlah tersebut.

Selanjutnya pada baris [6–9], dideklarasikan sebuah `struct Stack` yang berfungsi sebagai wadah penyimpanan data. Pada baris [7], `data[MAX]` merupakan array yang digunakan untuk menyimpan elemen stack. Kemudian pada baris [8], terdapat pointer `top` yang berfungsi untuk menunjuk posisi elemen teratas dalam stack. Penggunaan pointer di sini menunjukkan bahwa pendekatan yang digunakan adalah pointer-based, dimana posisi elemen ditentukan berdasarkan alamat memori, bukan indeks secara langsung.

Pada baris [11–16], dideklarasikan fungsi-fungsi utama yang akan diimplementasikan pada file `stack.cpp`. Fungsi `init` baris [11] digunakan untuk menginisialisasi stack agar berada pada kondisi awal (kosong). Fungsi `isEmpty` baris [12] digunakan untuk memeriksa apakah stack kosong, sedangkan `isFull` baris [13] untuk memeriksa apakah stack sudah penuh.

Fungsi `push` baris [14] berfungsi untuk menambahkan elemen ke dalam stack, sedangkan `pop` baris [15] digunakan untuk menghapus elemen teratas. Terakhir, fungsi `peek` baris [16] digunakan untuk melihat nilai elemen teratas tanpa menghapusnya. Dengan adanya deklarasi ini, file `stack.h` berperan sebagai interface atau penghubung antara program utama dengan implementasi stack yang terdapat pada `stack.cpp`.

File `stack.cpp`

Pada file `stack.cpp`, seluruh fungsi yang telah dideklarasikan diimplementasikan secara lengkap. Pada baris [4–6], fungsi `init` digunakan untuk menginisialisasi stack dengan cara mengatur pointer `top` ke posisi sebelum elemen pertama, yaitu `s->data - 1`. Hal ini menandakan bahwa stack dalam kondisi kosong karena belum ada elemen yang disimpan.

Pada baris [8–10], fungsi isEmpty memeriksa apakah stack kosong dengan membandingkan posisi top terhadap alamat awal array data. Jika top berada sebelum data, maka stack dianggap kosong. Logika ini benar karena pada kondisi awal top memang diset ke data - 1.

Pada baris [12–14], fungsi isFull digunakan untuk mengecek apakah stack sudah penuh. Kondisi penuh terjadi ketika pointer top berada pada elemen terakhir array, yaitu data + MAX - 1. Hal ini memastikan bahwa tidak ada elemen tambahan yang dapat dimasukkan lagi ke dalam stack.

Pada baris [16–22], fungsi push digunakan untuk menambahkan elemen ke dalam stack. Pertama, pada baris [17–19], dilakukan pengecekan apakah stack sudah penuh menggunakan isFull. Jika penuh, maka program akan melempar exception overflow_error. Jika tidak penuh maka pada baris [20], pointer top digeser ke depan (menaik), dan pada baris [21], nilai baru disimpan pada posisi yang ditunjuk oleh top. Algoritma ini benar karena elemen selalu ditambahkan di posisi teratas.

Pada baris [24–29], fungsi pop digunakan untuk menghapus elemen dari stack. Pada baris [25–27], dilakukan pengecekan apakah stack kosong menggunakan isEmpty. Jika kosong, maka akan dilempar exception underflow_error. Jika tidak kosong, maka pada baris [28], pointer top digeser ke belakang (menurun), yang berarti elemen teratas dihapus. Pendekatan ini benar karena tidak perlu menghapus data secara fisik, cukup dengan mengubah posisi top.

Pada baris [31–36], fungsi peek digunakan untuk melihat elemen teratas tanpa menghapusnya. Sama seperti pop, pada baris [32–34] dilakukan pengecekan kondisi kosong. Jika stack tidak kosong, maka pada baris [35], nilai yang ditunjuk oleh pointer top dikembalikan. Hal ini sesuai dengan konsep stack, dimana elemen teratas dapat diakses langsung.

Dari segi kebenaran algoritma, implementasi ini telah memenuhi prinsip **LIFO (Last In First Out)**, dimana elemen terakhir yang dimasukkan melalui fungsi push akan menjadi elemen pertama yang diambil melalui fungsi pop. Hal ini dapat dilihat dari penggunaan pointer top yang selalu bergerak maju saat penambahan data dan mundur saat penghapusan data.

Dari segi kompleksitas, seluruh operasi pada stack ini memiliki kompleksitas waktu **$O(1)$** karena setiap fungsi hanya melakukan operasi sederhana tanpa perulangan. Kompleksitas ruang yang digunakan adalah **$O(n)$** , dimana n adalah kapasitas maksimum stack (MAX), karena menggunakan array statis sebagai tempat penyimpanan data

SOAL 2

SI PALUI DAN MESIN HITUNG ANEH

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- N Adalah jumlah simbol dalam ekspresi
- A_i Adalah nilai bilangan pada input
- Simbol yang valid hanya :
Bilangan bulat
Operator : + , - , * , /

Format Input

n

A

1

A

2

.

.

.

A

n

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14

5 10 2 / 3 *	15
-----------------	----

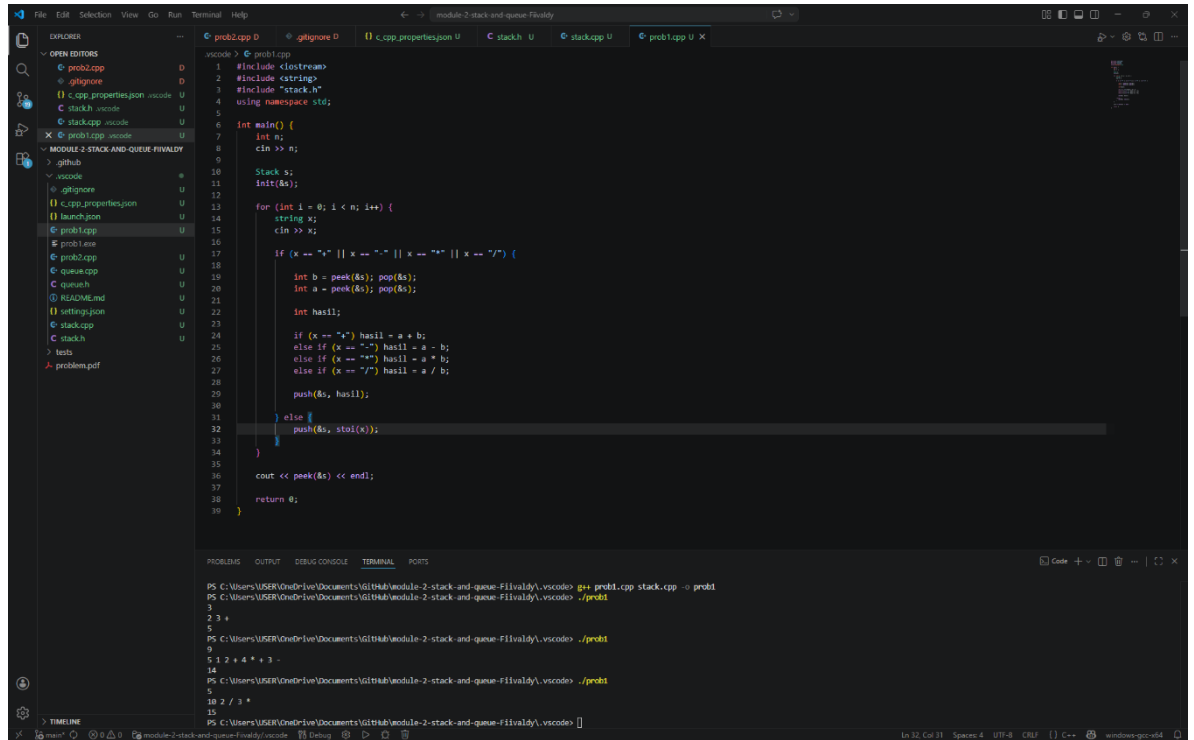
A. Source Code

Tabel 11 Source Code File prob1.cpp

1	#include <iostream>
2	#include <string>
3	#include "stack.h"
4	using namespace std;
5	
6	int main() {
7	int n;
8	cin >> n;
9	
10	Stack s;
11	init(&s);
12	
13	for (int i = 0; i < n; i++) {
14	string x;
15	cin >> x;
16	
17	if (x == "+" x == "-" x == "*" x
18	== "/") {
19	int b = peek(&s); pop(&s);
20	int a = peek(&s); pop(&s);
21	
22	int hasil;
23	
24	if (x == "+") hasil = a + b;
25	else if (x == "-") hasil = a - b;
26	else if (x == "*") hasil = a * b;
27	else if (x == "/") hasil = a / b;
28	

```
29         push(&s, hasil);
30
31     } else {
32         push(&s, stoi(x));
33     }
34 }
35
36 cout << peek(&s) << endl;
37
38 return 0;
39 }
```

B. Output Program



The screenshot shows a Visual Studio Code editor with a C++ file named `prob2.cpp` open. The code implements a stack and queue using a single array. The `main` function reads an integer `n` and a string `x` containing arithmetic operators. It processes the string character by character, performing operations on the stack and queue. The output of the program is shown in the terminal window at the bottom.

```
1 #include <iostream>
2 #include <string>
3 #include "stack.h"
4 using namespace std;
5
6 int main() {
7     int n;
8     cin >> n;
9
10    Stack s;
11    init(&s);
12
13    for (int i = 0; i < n; i++) {
14        string x;
15        cin >> x;
16
17        if (x == "+" || x == "-" || x == "*" || x == "/" ) {
18            int b = peek(&s); pop(&s);
19            int a = peek(&s); pop(&s);
20
21            int hasil;
22            if (x == "+") hasil = a + b;
23            else if (x == "-") hasil = a - b;
24            else if (x == "*") hasil = a * b;
25            else if (x == "/") hasil = a / b;
26
27            push(&s, hasil);
28        }
29        else {
30            push(&s, stoi(x));
31        }
32    }
33
34    cout << peek(&s) << endl;
35
36    return 0;
37 }
```

The terminal output shows the execution of the program with the following commands and results:

```
PS C:\Users\USER\OneDrive\Documents\Github\module-2-stack-and-queue-Filvaldy\vscode> g++ prob1.cpp stack.cpp -o prob1
PS C:\Users\USER\OneDrive\Documents\Github\module-2-stack-and-queue-Filvaldy\vscode> ./prob1
2 3 +
5
PS C:\Users\USER\OneDrive\Documents\Github\module-2-stack-and-queue-Filvaldy\vscode> ./prob1
5 1 2 + 4 * + 3 -
14
PS C:\Users\USER\OneDrive\Documents\Github\module-2-stack-and-queue-Filvaldy\vscode> ./prob1
10 2 / 3 *
15
PS C:\Users\USER\OneDrive\Documents\Github\module-2-stack-and-queue-Filvaldy\vscode>
```

Gambar 3 Screenshot Output Program Soal 2 File prob2.cpp

C. Pembahasan

Program pada Soal 2 sangat bergantung pada fungsi-fungsi yang telah diimplementasikan pada `stack.h` dan `stack.cpp`. Pada saat program memanggil `init(&s)` di baris [11], fungsi ini mengatur pointer `top` ke posisi awal (sebelum elemen pertama), sehingga stack siap digunakan dalam kondisi kosong.

Pada saat membaca bilangan, fungsi `push(&s, value)` yang dipanggil pada baris [28] dan [31] akan menambahkan elemen ke dalam stack dengan cara menggeser pointer `top` ke depan dan menyimpan nilai baru pada posisi tersebut. Sebaliknya, pada saat operator ditemukan, fungsi `peek(&s)` dan `pop(&s)` pada baris [19–20] digunakan untuk mengambil dan menghapus elemen teratas. Fungsi `peek` mengembalikan nilai tanpa menghapusnya, sedangkan `pop` hanya menggeser pointer `top` ke belakang untuk menghapus elemen secara logika.

Fungsi `isEmpty` dan `isFull` yang terdapat dalam `stack.cpp` secara tidak langsung juga berperan penting karena digunakan di dalam `push` dan `pop` untuk mencegah kondisi tidak valid. Jika terjadi kondisi seperti stack kosong saat `pop` atau penuh saat `push`, maka program akan melempar exception menggunakan `throw`, sehingga program menjadi lebih aman dan tidak mengalami error yang tidak terkontrol.

Dengan demikian, implementasi stack pada Soal 1 terbukti berhasil digunakan sebagai dasar penyelesaian Soal 2. Seluruh operasi berjalan dengan kompleksitas waktu $O(1)$ untuk setiap operasi stack, sehingga keseluruhan program tetap efisien. Hal ini menunjukkan bahwa struktur data stack sangat cocok digunakan untuk menyelesaikan permasalahan evaluasi ekspresi postfix.

Program pada Soal 2 berfungsi untuk menghitung operasi matematika dengan cara membaca urutan angka dan operator dari kiri ke kanan, kemudian memprosesnya menggunakan struktur data stack. Pada baris [7–8], program membaca jumlah simbol `n` yang akan diproses. Kemudian pada baris [10–11], dibuat sebuah variabel `Stack s` dan dilakukan inisialisasi menggunakan fungsi `init(&s)` agar stack berada dalam kondisi kosong sebelum digunakan.

Selanjutnya pada baris [13–30], dilakukan perulangan sebanyak `n` kali untuk membaca setiap simbol yang dimasukkan oleh pengguna. Pada baris [14–15], setiap simbol dibaca dalam bentuk string `x`, karena input dapat berupa angka

maupun operator. Pada baris [17], program memeriksa apakah simbol tersebut merupakan operator (+, -, *, atau /).

Jika simbol adalah operator, maka pada baris [19–20], program mengambil dua nilai teratas dari stack menggunakan fungsi peek dan pop. Nilai pertama yang diambil disimpan ke dalam variabel b, dan nilai berikutnya ke dalam a. Urutan ini penting karena operasi matematika harus dilakukan dalam bentuk a operator b.

Pada baris [22–26], dilakukan proses perhitungan sesuai operator yang dibaca. Hasil perhitungan tersebut kemudian disimpan kembali ke dalam stack menggunakan fungsi push pada baris [28]. Hal ini sesuai dengan cara kerja mesin hitung yang dijelaskan pada soal, dimana hasil sementara disimpan kembali untuk digunakan pada operasi berikutnya.

Jika simbol bukan operator, maka pada baris [30–32], simbol tersebut dianggap sebagai bilangan dan dikonversi dari string ke integer menggunakan fungsi stoi, kemudian dimasukkan ke dalam stack menggunakan push.

Setelah seluruh simbol selesai diproses, pada baris [35], program menampilkan hasil akhir dengan mengambil elemen teratas dari stack menggunakan fungsi peek. Nilai ini merupakan hasil akhir dari seluruh operasi karena dalam metode postfix hanya akan tersisa satu nilai di akhir proses.

Secara keseluruhan, dalam algoritma ini setiap operator selalu bekerja pada dua operand terakhir yang dimasukkan, sehingga sesuai dengan prinsip evaluasi postfix. Kompleksitas waktu dari algoritma ini adalah $O(n)$ karena setiap simbol diproses satu kali, sedangkan kompleksitas ruang adalah $O(n)$ karena stack dapat menyimpan hingga n bagian.

SOAL 3

Pada file `queue.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `queue.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 12 Source Code File queue.h

1	<code>#ifndef QUEUE_H</code>
2	<code>#define QUEUE_H</code>
3	
4	<code>#define MAX 100</code>
5	
6	<code>struct Queue {</code>
7	<code> int data[MAX];</code>
8	<code> int* front;</code>
9	<code> int* rear;</code>
10	<code>};</code>
11	
12	<code>void init(Queue* q);</code>
13	<code>bool isEmpty(const Queue* q);</code>
14	<code>bool isFull(const Queue* q);</code>
15	<code>void enqueue(Queue* q, int value);</code>
16	<code>void dequeue(Queue* q);</code>
17	<code>int front(const Queue* q);</code>
18	<code>int back(const Queue* q);</code>
19	
20	<code>#endif</code>

Tabel 13 Source Code File queue.cpp

```
1  #include "queue.h"
2  #include <stdexcept>
3
4  void init(Queue* q) {
5      q->front = q->data;
6      q->rear = q->data - 1;
7  }
8
9  bool isEmpty(const Queue* q) {
10     return q->rear < q->front;
11 }
12
13 bool isFull(const Queue* q) {
14     return q->rear == q->data + MAX - 1;
15 }
16
17 void enqueue(Queue* q, int value) {
18     if (isFull(q)) {
19         throw std::overflow_error("Queue penuh");
20     }
21     q->rear++;
22     *(q->rear) = value;
23 }
24
25 void dequeue(Queue* q) {
26     if (isEmpty(q)) {
27         throw std::underflow_error("Queue kosong");
28     }
29     q->front++;
30 }
```

```
31
32 int front(const Queue* q) {
33     if (isEmpty(q)) {
34         throw std::underflow_error("Queue kosong");
35     }
36     return *(q->front);
37 }
38
39 int back(const Queue* q) {
40     if (isEmpty(q)) {
41         throw std::underflow_error("Queue kosong");
42     }
43     return *(q->rear);
44 }
```


B. Pembahasan

File queue.h

Pada file queue.h, didefinisikan struktur dasar dari queue beserta deklarasi fungsi-fungsi yang akan digunakan. Pada baris [4], terdapat konstanta MAX yang menentukan kapasitas maksimum queue yaitu sebanyak 100 elemen. Hal ini menunjukkan bahwa queue menggunakan array statis dengan ukuran tetap.

Pada baris [6–10], dideklarasikan sebuah struct Queue sebagai tempat penyimpanan data. Pada baris [7], data[MAX] merupakan array yang digunakan untuk menyimpan elemen-elemen queue. Selanjutnya pada baris [8] dan [9], terdapat pointer front dan rear yang masing-masing berfungsi untuk menunjuk elemen paling depan dan paling belakang dalam queue. Penggunaan pointer ini menunjukkan bahwa pendekatan yang digunakan adalah pointer-based, dimana posisi elemen ditentukan berdasarkan alamat memori, bukan indeks secara langsung.

Pada baris [12–18], dideklarasikan fungsi-fungsi utama yang akan diimplementasikan pada file queue.cpp. Fungsi init (baris 12) digunakan untuk menginisialisasi queue agar berada dalam kondisi kosong. Fungsi isEmpty baris [13] berfungsi untuk memeriksa apakah queue kosong, sedangkan isFull baris [14] digunakan untuk memeriksa apakah queue sudah penuh.

Fungsi enqueue baris [15] digunakan untuk menambahkan elemen ke bagian belakang queue, sedangkan dequeue baris [16] digunakan untuk menghapus elemen dari bagian depan. Kemudian fungsi front baris [17] digunakan untuk melihat elemen paling depan tanpa menghapusnya, dan back baris [18] digunakan untuk melihat elemen paling belakang. Dengan adanya deklarasi ini, file queue.h berfungsi sebagai antarmuka yang menghubungkan program utama dengan implementasi queue pada queue.cpp.

File queue.cpp

Pada file `queue.cpp`, seluruh fungsi yang telah dideklarasikan pada `queue.h` diimplementasikan secara lengkap. Pada baris [4–7], fungsi `init` digunakan untuk menginisialisasi `queue`. Pointer `front` diatur ke awal array (`q->data`), sedangkan pointer `rear` diatur ke posisi sebelum elemen pertama (`q->data - 1`). Hal ini menandakan bahwa `queue` masih kosong karena belum ada elemen yang masuk.

Pada baris [9–11], fungsi `isEmpty` digunakan untuk memeriksa apakah `queue` kosong dengan membandingkan posisi pointer `rear` dan `front`. Jika `rear` berada sebelum `front`, maka `queue` dianggap kosong. Logika ini benar karena saat awal inisialisasi memang `rear` berada sebelum `front`.

Pada baris [13–15], fungsi `isFull` digunakan untuk mengecek apakah `queue` sudah penuh. Kondisi penuh terjadi ketika pointer `rear` sudah mencapai elemen terakhir dalam array, yaitu `data + MAX - 1`. Hal ini memastikan bahwa tidak ada ruang lagi untuk menambahkan elemen baru.

Pada baris [17–23], fungsi `enqueue` digunakan untuk menambahkan elemen ke dalam `queue`. Pada baris [18–20], dilakukan pengecekan apakah `queue` sudah penuh menggunakan `isFull`. Jika penuh, maka program akan melempar exception `overflow_error`. Jika tidak penuh, maka pada baris [21], pointer `rear` digeser ke depan, dan pada baris [22], nilai baru disimpan pada posisi tersebut. Proses ini sesuai dengan konsep `queue`, dimana elemen baru selalu ditambahkan di bagian belakang.

Pada baris [25–30], fungsi `dequeue` digunakan untuk menghapus elemen dari bagian depan `queue`. Pada baris [26–28], dilakukan pengecekan apakah `queue` kosong menggunakan `isEmpty`. Jika kosong, maka program akan melempar exception `underflow_error`. Jika tidak kosong, maka pada baris [29], pointer `front` digeser ke depan, yang berarti elemen paling depan telah dihapus secara logika. Pendekatan ini efisien karena tidak perlu memindahkan elemen lain dalam array.

Pada baris [32–37], fungsi `front` digunakan untuk melihat elemen paling depan tanpa menghapusnya. Jika `queue` kosong, akan dilempar exception. Jika tidak, maka nilai pada alamat yang ditunjuk oleh pointer `front` dikembalikan. Sedangkan pada baris [39–44], fungsi `back` digunakan untuk melihat elemen paling belakang dengan cara mengembalikan nilai yang ditunjuk oleh pointer `rear`.

Secara keseluruhan, implementasi ini mengikuti prinsip FIFO (First In First Out), dimana elemen pertama yang masuk akan menjadi elemen pertama yang keluar. Hal ini dibuktikan dengan operasi enqueue yang menambahkan elemen di belakang dan dequeue yang menghapus elemen dari depan. Semua operasi seperti enqueue, dequeue, front, dan back memiliki kompleksitas waktu $O(1)$ karena hanya melibatkan perubahan pointer tanpa perulangan. Kompleksitas ruang juga efisien yaitu $O(1)$ untuk setiap operasi, karena tidak ada alokasi memori tambahan selain array yang telah ditentukan di awal.

SOAL 4

Time Limit: 1.0 s

Memory Limit: 64 MB

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan.

Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n : jumlah elemen
- k : ukuran jendela
- A_i : elemen array

Format Input

n

k

A

1

A

2

.

•
•
A
n

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok yang dibentuk
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source Code

Tabel 14 Source Code File prob2.cpp

1	#include "queue.h"
2	#include <iostream>
3	using namespace std;
4	
5	int main() {
6	int n, k;
7	cin >> n >> k;
8	
9	int arr[100];
10	
11	for (int i = 0; i < n; i++) {
12	cin >> arr[i];
13	}
14	
15	if (k > n k <= 0) return 0;
16	
17	int sum = 0;
18	
19	for (int i = 0; i < k; i++) {
20	sum += arr[i];
21	}
22	
23	cout << sum;
24	
25	for (int i = k; i < n; i++) {
26	sum = sum - arr[i - k] + arr[i];
27	cout << " " << sum;
28	}
29	cout << endl;

30	return 0;
31	}

B. Output Program

The screenshot shows the Visual Studio Code interface with the file `prob2.cpp` open. The code implements a program that reads an integer `n` and a character `k`. It then reads `n` integers into an array `arr`. If `k` is 'D', it prints the sum of all elements in the array. If `k` is 'Q', it prints the sum of elements from index 1 to `k`. The output in the terminal shows the program being compiled and executed with the input `5 D`, resulting in the output `1 2 3 4 5` and `15`.

```
1 #include "queue.h"
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     int n, k;
7     cin >> n >> k;
8     int arr[100];
9
10    for (int i = 0; i < n; i++) {
11        cin >> arr[i];
12    }
13
14    if (k > n || k <= 0) return 0;
15
16    int sum = 0;
17
18    for (int i = 0; i < k; i++) {
19        sum += arr[i];
20    }
21
22    cout << sum;
23
24    for (int i = k; i < n; i++) {
25        sum = sum - arr[i - k] + arr[i];
26        cout << " " << sum;
27    }
28    cout << endl;
29    return 0;
30 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\USER\OneDrive\Documents\Github\module-2-stack-and-queue-Final\vscode> g++ prob2.cpp queue.cpp -o prob2
PS C:\Users\USER\OneDrive\Documents\Github\module-2-stack-and-queue-Final\vscode> ./prob2
5 D
1 2 3 4 5
15
PS C:\Users\USER\OneDrive\Documents\Github\module-2-stack-and-queue-Final\vscode> ./prob2
5 5
1 2 3 4 5
15
PS C:\Users\USER\OneDrive\Documents\Github\module-2-stack-and-queue-Final\vscode> ./prob2
4 -1 2 -3 5 1
3 1 -1 2 6
```

Gambar 4 Screenshot Output Program Soal 4 File prob2.cpp

C. Pembahasan

Program pada file prob2.cpp digunakan untuk menghitung jumlah dari sejumlah elemen yang berurutan dalam sebuah array dengan menggunakan teknik *sliding window*. Pada baris [6–7], program membaca dua buah input yaitu n sebagai jumlah elemen dan k sebagai ukuran jendela (jumlah elemen yang dijumlahkan dalam satu kelompok).

Selanjutnya pada baris [9], dideklarasikan array `arr[100]` yang digunakan untuk menyimpan data angka yang dimasukkan oleh pengguna. Pada baris [11–13], dilakukan perulangan untuk membaca sebanyak n elemen dan menyimpannya ke dalam array tersebut.

Pada baris [15], terdapat pengecekan kondisi untuk memastikan nilai k valid. Jika k lebih besar dari n atau kurang dari atau sama dengan nol, maka program langsung berhenti dengan `return 0`. Hal ini penting untuk mencegah kesalahan dalam proses perhitungan.

Pada baris [17], variabel `sum` diinisialisasi dengan nilai 0. Variabel ini akan digunakan untuk menyimpan jumlah dari elemen-elemen dalam satu jendela. Kemudian pada baris [19–21], dilakukan perhitungan jumlah awal untuk k elemen pertama dalam array. Proses ini disebut sebagai *window pertama*.

Hasil dari perhitungan awal tersebut kemudian ditampilkan pada baris [23]. Setelah itu, pada baris [25–28], program mulai menerapkan teknik *sliding window*. Pada setiap iterasi, nilai `sum` diperbarui dengan cara mengurangi elemen yang keluar dari jendela (`arr[i - k]`) dan menambahkan elemen baru yang masuk ke dalam jendela (`arr[i]`). Dengan cara ini, program tidak perlu menghitung ulang seluruh elemen dalam setiap jendela, sehingga lebih efisien.

Setiap hasil perhitungan jendela ditampilkan secara berurutan dalam satu baris pada baris [27], dipisahkan dengan spasi. Terakhir, pada baris 29, program mencetak baris baru agar output lebih rapi, dan pada baris [30] program selesai dijalankan.

Secara keseluruhan, Dalam algoritma ini setiap langkah hanya menggeser jendela satu posisi ke kanan dan memperbarui jumlah berdasarkan perubahan elemen yang masuk dan keluar. Kompleksitas waktu dari algoritma ini adalah $O(n)$ karena setiap elemen hanya diproses satu kali, sedangkan kompleksitas ruang

adalah $O(1)$ karena hanya menggunakan variabel tambahan tanpa alokasi memori baru yang signifikan. Dengan demikian, metode ini lebih efisien dibandingkan cara biasa yang menghitung ulang setiap jendela dari awal.

MODUL 3 : Linked List dan Double Linked List

SOAL 1 : Implementasi Senarai Berantai Tunggal Melingkar

Pada file `single_linked_list.h` yang diberikan, implementasikan struktur `SingleLinkedList` beserta fungsi-fungsinya ke dalam file `single_linked_list.cpp`. Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap. Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

Tabel 15 Source Code File `single_linked_list.h`

1	<code>#ifndef SINGLE_LINKED_LIST</code>
2	<code>#define SINGLE_LINKED_LIST</code>
3	
4	<code>struct Node {</code>
5	<code> int data;</code>
6	<code> Node* next;</code>
7	<code>};</code>
8	
9	<code>struct SingleLinkedList {</code>
10	
11	<code> Node* head = nullptr;</code>
12	<code> Node* tail = nullptr;</code>
13	
14	<code> int size = 0;</code>
15	
16	<code> void init();</code>
17	<code> bool is_empty();</code>
18	
19	<code> void add_front(int inputValue);</code>

20	void add_back(int inputValue);
21	void add_idx(int inputValue, int indexPosition);
22	
23	void delete_front();
24	void delete_back();
25	void delete_idx(int indexPosition);
26	
27	void display();
28	
29	int get(int indexPosition);
30	
31	void set(int inputValue, int indexPosition);
32	
33	void clear();
34	};
35	
36	#endif

Tabel 16 Source Code File single_linked_list.cpp

1	#include "single_linked_list.h"
2	#include <iostream>
3	
4	using namespace std;
5	
6	void SingleLinkedList::init() {
7	
8	head = nullptr;
9	tail = nullptr;
10	
11	size = 0;
12	}

```

13
14 bool SingleLinkedList::is_empty() {
15
16     return head == nullptr;
17 }
18
19 void SingleLinkedList::add_front(int inputValue)
20 {
21     Node* newNode = new Node;
22
23     newNode->data = inputValue;
24     newNode->next = nullptr;
25
26     if (is_empty()) {
27
28         head = newNode;
29         tail = newNode;
30         tail->next = head;
31     }
32     else {
33
34         newNode->next = head;
35         head = newNode;
36         tail->next = head;
37     }
38
39     size++;
40 }
41
42 void SingleLinkedList::add_back(int inputValue) {
43

```

```

44     Node* newNode = new Node;
45
46     newNode->data = inputValue;
47     newNode->next = nullptr;
48
49     if (is_empty()) {
50
51         head = newNode;
52         tail = newNode;
53
54         tail->next = head;
55     }
56     else {
57         tail->next = newNode;
58         tail = newNode;
59         tail->next = head;
60     }
61
62     size++;
63 }
64
65 void SingleLinkedList::add_idx(int inputValue,
int indexPosition) {
    if (indexPosition < 0 || indexPosition > size)
66 {
67
68     cout << "Index tidak valid\n";
69     return;
70 }
71 if (indexPosition == 0) {
72
73     add_front(inputValue);

```

```

74         return;
75     }
76     if (indexPosition == size) {
77
78         add_back(inputValue);
79         return;
80     }
81     Node* newNode = new Node;
82
83     newNode->data = inputValue;
84     newNode->next = nullptr;
85
86     Node* currentNode = head;
87
88     for (int currentIndex = 0;
89         currentIndex < indexPosition - 1;
90         currentIndex++) {
91
92         currentNode = currentNode->next;
93     }
94
95     newNode->next = currentNode->next;
96
97     currentNode->next = newNode;
98
99     size++;
100 }
101
102 void SingleLinkedList::delete_front() {
103
104     if (is_empty()) {
105

```

```

106         cout << "Linked list kosong\n";
107         return;
108     }
109     if (head == tail) {
110
111         delete head;
112
113         head = nullptr;
114         tail = nullptr;
115     }
116     else {
117
118         Node* deletedNode = head;
119         head = head->next;
120         tail->next = head;
121
122         delete deletedNode;
123     }
124
125     size--;
126 }
127
128 void SingleLinkedList::delete_back() {
129     if (is_empty()) {
130
131         cout << "Linked list kosong\n";
132         return;
133     }
134     if (head == tail) {
135
136         delete head;
137

```



```

138         head = nullptr;
139         tail = nullptr;
140     }
141     else {
142
143         Node* currentNode = head;
144         while (currentNode->next != tail) {
145
146             currentNode = currentNode->next;
147         }
148         delete tail;
149         tail = currentNode;
150         tail->next = head;
151     }
152
153     size--;
154 }
155 void SingleLinkedList::delete_idx(int
indexPosition) {
156     if (indexPosition < 0 || indexPosition >= size)
{
157
158         cout << "Index tidak valid\n";
159         return;
160     }
161     if (indexPosition == 0) {
162
163         delete_front();
164         return;
165     }
166     if (indexPosition == size - 1) {
167

```

```

168         delete_back();
169         return;
170     }
171
172     Node* currentNode = head;
173     for (int currentIndex = 0;
174         currentIndex < indexPosition - 1;
175         currentIndex++) {
176
177         currentNode = currentNode->next;
178     }
179
180     Node* deletedNode = currentNode->next;
181     currentNode->next = deletedNode->next;
182
183     delete deletedNode;
184
185     size--;
186 }
187
188 void SingleLinkedList::display() {
189
190     if (is_empty()) {
191
192         cout << "Linked list kosong\n";
193         return;
194     }
195
196     Node* currentNode = head;
197
198     cout << "Isi Linked List : ";
199     do {

```

```

200
201         cout << currentNode->data << " ";
202
203         currentNode = currentNode->next;
204
205     } while (currentNode != head);
206
207     cout << endl;
208 }
209
210 int SingleLinkedList::get(int indexPosition) {
211     if (indexPosition < 0 || indexPosition >= size)
212     {
213         cout << "Index tidak valid\n";
214         return -1;
215     }
216
217     Node* currentNode = head;
218     for (int currentIndex = 0;
219         currentIndex < indexPosition;
220         currentIndex++) {
221
222         currentNode = currentNode->next;
223     }
224
225     return currentNode->data;
226 }
227
228 void SingleLinkedList::set(int inputValue,
229                             int indexPosition) {
230

```

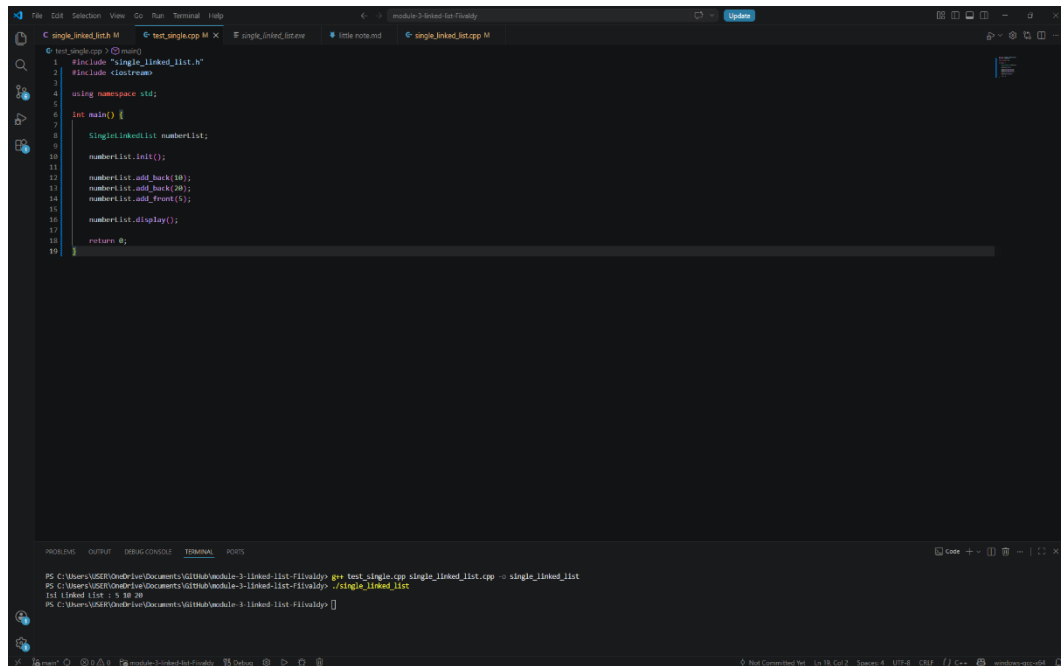
	if (indexPosition < 0 indexPosition >= size)
231	{
232	
233	cout << "Index tidak valid\n";
234	return;
235	}
236	
237	Node* currentNode = head;
238	for (int currentIndex = 0;
239	currentIndex < indexPosition;
240	currentIndex++) {
241	
242	currentNode = currentNode->next;
243	}
244	currentNode->data = inputValue;
245	}
246	void SingleLinkedList::clear() {
247	while (!is_empty()) {
248	
249	delete_front();
250	}
251	
252	head = nullptr;
253	tail = nullptr;
254	
255	size = 0;
256	}

Tabel 17 Source Code File test_single.cpp

1	#include "single_linked_list.h"
2	#include <iostream>
3	

4	using namespace std;
5	
6	int main() {
7	
8	SingleLinkedList numberList;
9	
10	numberList.init();
11	
12	numberList.add_back(10);
13	numberList.add_back(20);
14	numberList.add_front(5);
15	
16	numberList.display();
17	
18	return 0;
19	}

B. Output Program



```
1 #include "single_linked_list.h"
2 #include <iostream>
3
4 using namespace std;
5
6 int main() {
7     SinglyLinkedList numberList;
8
9     numberList.init();
10
11     numberList.add_back(10);
12     numberList.add_back(20);
13     numberList.add_front(5);
14
15     numberList.display();
16
17     return 0;
18 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\USER\OneDrive\Documents\GitHub\module-3-linked-list-filevaldy> g++ test_single.cpp single_linked_list.cpp -o single_linked_list
PS C:\Users\USER\OneDrive\Documents\GitHub\module-3-linked-list-filevaldy> ./single_linked_list
1st linked list : > 10 20
PS C:\Users\USER\OneDrive\Documents\GitHub\module-3-linked-list-filevaldy>
```

Gambar 5 Screenshot Output Program Soal 1

C. Pembahasan

Fungsi `clear()` pada implementasi Circular Single Linked List dirancang untuk memastikan seluruh node yang telah dialokasikan di dalam heap dapat dihapus dengan benar sehingga tidak terjadi *memory leak*. Pada implementasi program, proses penghapusan dilakukan menggunakan perulangan `while (!is_empty())` yang akan terus memanggil fungsi `delete_front()` hingga seluruh node habis terhapus. Dengan cara ini setiap node yang sebelumnya dibuat menggunakan sintaks `new Node` akan dibebaskan kembali menggunakan `delete`, sehingga memori yang dipakai program dapat dikembalikan ke sistem operasi. Setelah seluruh node terhapus, pointer `head` dan `tail` dikembalikan menjadi `nullptr`, sedangkan `size` diubah menjadi 0. Alur ini penting karena tanpa proses penghapusan manual, node yang sudah tidak digunakan masih tetap berada di heap dan menyebabkan kebocoran memori.

Seluruh manajemen memori pada program dilakukan secara manual menggunakan alokasi heap. Hal ini terlihat pada sintaks `Node* newNode = new Node;` yang digunakan ketika membuat node baru. Penggunaan heap dipilih karena jumlah node pada linked list bersifat dinamis dan dapat bertambah maupun berkurang selama program berjalan. Dengan alokasi heap, ukuran linked list tidak perlu ditentukan sejak awal seperti array statis. Namun, karena memori dialokasikan secara manual maka setiap node yang sudah tidak digunakan wajib dihapus menggunakan `delete`. Oleh sebab itu fungsi-fungsi seperti `delete_front()`, `delete_back()`, `delete_idx()`, dan `clear()` menjadi sangat penting dalam menjaga efisiensi penggunaan memori.

Pada fungsi `init()`, kompleksitas waktu yang digunakan adalah $O(1)$ karena program hanya mengubah nilai beberapa pointer dan variabel tanpa melakukan perulangan. Kompleksitas ruangnya juga $O(1)$ karena tidak ada penggunaan memori tambahan selain variabel yang sudah ada. Fungsi `is_empty()` juga memiliki kompleksitas waktu $O(1)$ karena hanya melakukan satu kali pengecekan kondisi pada pointer `head`. Kompleksitas ruangnya tetap $O(1)$. Pada fungsi `add_front()` dan `add_back()`, kompleksitas waktunya adalah $O(1)$ karena penambahan node dilakukan langsung pada bagian depan atau belakang tanpa perlu menelusuri seluruh linked list. Kompleksitas ruangnya adalah $O(1)$ karena hanya

membutuhkan satu node baru sebagai memori tambahan. Fungsi `add_idx()` memiliki kompleksitas waktu $O(n)$ karena program harus melakukan traversal atau penelusuran node hingga mencapai posisi index tertentu. Pada kondisi terburuk, traversal dilakukan sampai mendekati node terakhir. Kompleksitas ruangnya tetap $O(1)$ karena hanya menggunakan satu node tambahan. Fungsi `delete_front()` mempunyai kompleksitas waktu $O(1)$ karena node pertama dapat langsung dihapus tanpa traversal. Kompleksitas ruangnya juga $O(1)$. Pada fungsi `delete_back()`, kompleksitas waktunya adalah $O(n)$ karena program harus mencari node sebelum tail menggunakan traversal. Kompleksitas ruangnya tetap $O(1)$ karena tidak membutuhkan struktur data tambahan. Fungsi `delete_idx()` juga memiliki kompleksitas waktu $O(n)$ karena program perlu bergerak menuju node sebelum posisi target. Kompleksitas ruangnya tetap $O(1)$. Pada fungsi `display()`, kompleksitas waktunya adalah $O(n)$ karena seluruh node harus ditampilkan satu per satu hingga kembali ke head. Kompleksitas ruangnya adalah $O(1)$ karena tidak menggunakan memori tambahan yang signifikan. Fungsi `get()` dan `set()` memiliki kompleksitas waktu $O(n)$ karena keduanya membutuhkan traversal menuju index tertentu sebelum data diambil atau diubah. Kompleksitas ruang kedua fungsi tersebut tetap $O(1)$. Sementara itu fungsi `clear()` memiliki kompleksitas waktu $O(n)$ karena setiap node harus dihapus satu per satu hingga linked list kosong. Kompleksitas ruangnya tetap $O(1)$ karena proses penghapusan dilakukan langsung tanpa memerlukan struktur tambahan.

Seluruh operasi linked list mempertahankan hubungan circular antara tail dan head. Setiap kali node ditambahkan atau dihapus, pointer `tail->next` selalu diperbarui agar tetap menunjuk ke head. Dengan demikian struktur circular linked list tetap terjaga pada seluruh kondisi, baik ketika linked list kosong, memiliki satu node, maupun banyak node. Selain itu validasi index pada fungsi seperti `add_idx()`, `delete_idx()`, `get()`, dan `set()` memastikan program tidak mengakses memori di luar batas linked list. Penggunaan delete pada setiap proses penghapusan juga menjamin memori yang sudah tidak digunakan dapat dibebaskan kembali sehingga program menjadi lebih aman dan efisien.

Kode file **single_linked_list.cpp** merupakan implementasi dari struktur data *Circular Single Linked List*, yaitu linked list yang node terakhirnya kembali menunjuk ke node pertama sehingga membentuk lingkaran. Pada bagian awal program, file header dipanggil menggunakan sintaks `#include "single_linked_list.h"` pada baris [1] agar seluruh deklarasi struktur dan fungsi yang telah dibuat sebelumnya dapat digunakan kembali pada file implementasi ini. Selain itu, library `iostream` pada baris [2] digunakan untuk proses input dan output seperti menampilkan data linked list ke layar.

Fungsi `init()` pada baris [6] digunakan untuk menginisialisasi linked list agar berada dalam kondisi kosong. Pada bagian ini variabel `head` dan `tail` diubah menjadi `nullptr` pada baris [8] dan [9], yang menandakan belum terdapat node di dalam linked list. Sementara itu variabel `size` pada baris [11] diatur menjadi 0 untuk menunjukkan jumlah data awal masih kosong. Fungsi `is_empty()` pada baris [14] berfungsi untuk memeriksa apakah linked list masih kosong atau tidak. Pemeriksaan dilakukan menggunakan sintaks `return head == nullptr;` pada baris [16]. Jika `head` bernilai `nullptr`, maka fungsi akan menghasilkan nilai `true`, yang berarti belum terdapat node di dalam linked list. Pada fungsi `add_front()` di baris [19], program membuat node baru menggunakan sintaks `Node* newNode = new Node;` pada baris [21]. Sintaks ini berfungsi untuk memesan memori baru secara dinamis. Kemudian data yang dimasukkan pengguna disimpan ke dalam node melalui `newNode->data = inputValue;` pada baris [23]. Jika linked list masih kosong, maka node baru akan menjadi `head` dan `tail` sekaligus seperti terlihat pada baris [28] dan [29]. Selanjutnya sintaks `tail->next = head;` pada baris [30] menjadi bagian penting karena membuat node terakhir kembali menunjuk ke node pertama sehingga struktur linked list menjadi circular.

Fungsi `add_back()` pada baris [40] memiliki tujuan untuk menambahkan data di bagian belakang linked list. Ketika linked list tidak kosong, program menghubungkan node terakhir lama ke node baru menggunakan sintaks `tail->next = newNode;` pada baris [54]. Setelah itu posisi `tail` dipindahkan ke node baru pada

baris [55]. Bagian penting lainnya adalah `tail->next = head`; pada baris [56] yang memastikan hubungan circular tetap terjaga setelah penambahan data.

Pada fungsi `add_idx()` di baris [63], program memungkinkan penambahan data berdasarkan posisi index tertentu. Validasi index dilakukan terlebih dahulu pada baris [64] agar pengguna tidak memasukkan posisi yang melebihi ukuran linked list. Proses pencarian posisi node dilakukan menggunakan perulangan `for` pada baris [84] hingga [86]. Perulangan ini berjalan sampai node sebelum posisi tujuan ditemukan. Setelah itu node baru disisipkan menggunakan sintaks `newNode->next = currentNode->next`; pada baris [91] dan `currentNode->next = newNode`; pada baris [93]. Fungsi `delete_front()` pada baris [99] digunakan untuk menghapus node pertama. Jika linked list hanya memiliki satu node, maka memori node dihapus menggunakan sintaks `delete head`; pada baris [109]. Namun jika jumlah node lebih dari satu, maka head dipindahkan ke node berikutnya menggunakan `head = head->next`; pada baris [118]. Kemudian hubungan circular diperbarui melalui `tail->next = head`; pada baris [119]. Pada fungsi `delete_back()` di baris [127], program mencari node sebelum tail menggunakan perulangan `while` pada baris [142]. Setelah ditemukan, node terakhir dihapus menggunakan `delete tail`; pada baris [147]. Kemudian tail dipindahkan ke node sebelumnya pada baris [148], dan hubungan circular diperbaiki kembali melalui `tail->next = head`; pada baris [149]. Fungsi `delete_idx()` pada baris [155] digunakan untuk menghapus node berdasarkan posisi index tertentu. Program terlebih dahulu melakukan validasi index pada baris [156]. Selanjutnya pencarian node sebelum target dilakukan melalui perulangan `for` pada baris [174] hingga [176]. Node target kemudian dilewati menggunakan sintaks `currentNode->next = deletedNode->next`; pada baris [182], sehingga node tersebut tidak lagi terhubung dengan linked list sebelum akhirnya dihapus menggunakan `delete deletedNode`; pada baris [184]. Fungsi `display()` pada baris [190] digunakan untuk menampilkan seluruh isi linked list. Karena struktur yang digunakan adalah circular linked list, maka program memakai perulangan `do while` pada baris [201] hingga [207]. Perulangan ini akan terus berjalan sampai pointer kembali ke head, sehingga seluruh node dapat ditampilkan tanpa terjadi perulangan tak terbatas. Pada fungsi `get()` di baris [213], program mengambil data berdasarkan index tertentu. Proses perpindahan node dilakukan melalui perulangan `for` pada baris

[222] hingga [224]. Setelah posisi node ditemukan, data dikembalikan menggunakan sintaks `return currentNode->data;` pada baris [229].

Fungsi `set()` pada baris [232] digunakan untuk mengubah isi data pada index tertentu. Setelah posisi node ditemukan melalui proses perulangan pada baris [242] hingga [244], nilai data lama diganti menggunakan sintaks `currentNode->data = inputValue;` pada baris [247]. Fungsi `clear()` pada baris [249] digunakan untuk menghapus seluruh isi linked list. Penghapusan dilakukan secara berulang menggunakan `while (!is_empty())` pada baris [250], yang akan terus memanggil fungsi `delete_front()` sampai seluruh node habis terhapus. Setelah selesai, `head`, `tail`, dan `size` dikembalikan lagi ke kondisi awal kosong pada baris [254] hingga [257].

Pada Kode header file **single_linked_list.h** digunakan sebagai tempat deklarasi struktur data dan fungsi-fungsi yang akan dipakai pada program Circular Single Linked List. Pada bagian awal terdapat sintaks `#ifndef SINGLE_LINKED_LIST` di baris [1] dan `#define SINGLE_LINKED_LIST` pada baris [2]. Kedua sintaks ini berfungsi sebagai *header guard*, yaitu untuk mencegah file header dipanggil berulang kali saat proses kompilasi. Tanpa bagian ini, program dapat mengalami error karena deklarasi struktur atau fungsi dianggap dibuat lebih dari satu kali. Struktur `Node` pada baris [4] digunakan sebagai komponen utama dalam linked list. Di dalamnya terdapat variabel data pada baris [5] yang berfungsi menyimpan nilai bertipe integer. Selain itu terdapat pointer `next` pada baris [6] yang digunakan untuk menyimpan alamat node berikutnya. Pointer inilah yang membuat setiap node dapat saling terhubung membentuk linked list.

Struktur `SingleLinkedList` pada baris [9] digunakan sebagai wadah utama untuk mengatur seluruh operasi linked list. Variabel `head` pada baris [11] berfungsi menunjuk node pertama, sedangkan `tail` menunjuk node terakhir. Nilai awal keduanya diatur menjadi `nullptr` untuk menandakan linked list masih kosong. Pada baris [13], terdapat variabel `size` yang digunakan untuk menyimpan jumlah node yang ada di dalam linked list. Deklarasi fungsi `init()` pada baris [15] digunakan untuk melakukan inisialisasi awal linked list. Fungsi `is_empty()` pada baris [16] berfungsi

mengecek apakah linked list masih kosong atau sudah memiliki data. Kedua fungsi ini menjadi dasar penting sebelum melakukan operasi lain pada linked list.

Pada baris [18] hingga [20], terdapat fungsi `add_front()`, `add_back()`, dan `add_idx()`. Ketiga fungsi ini digunakan untuk menambahkan data ke dalam linked list. Fungsi `add_front()` menambahkan node di bagian depan, `add_back()` menambahkan node di bagian belakang, sedangkan `add_idx()` digunakan untuk menyisipkan node pada posisi index tertentu. Selanjutnya, pada baris [22] hingga [24], terdapat fungsi `delete_front()`, `delete_back()`, dan `delete_idx()`. Fungsi-fungsi tersebut digunakan untuk menghapus node dari linked list. Penghapusan dapat dilakukan dari bagian depan, belakang, maupun berdasarkan posisi index tertentu. Fungsi `display()` pada baris [26] digunakan untuk menampilkan seluruh isi linked list ke layar. Fungsi ini penting untuk memastikan data berhasil disimpan atau dihapus sesuai proses yang dilakukan pengguna.

Pada baris [28], fungsi `get()` digunakan untuk mengambil nilai data berdasarkan index tertentu. Sedangkan fungsi `set()` pada baris [30] digunakan untuk mengubah nilai data pada posisi tertentu di dalam linked list. Fungsi `clear()` pada baris [32] digunakan untuk menghapus seluruh node yang ada di linked list. Fungsi ini sangat penting karena membantu mencegah terjadinya *memory leak*, yaitu kondisi ketika memori yang sudah tidak digunakan masih tetap tersimpan dan tidak dibebaskan dari program.

Kode pada file **single_test.cpp** berfungsi sebagai program utama untuk menjalankan dan menguji implementasi Circular Single Linked List yang telah dibuat sebelumnya. Pada baris [1], program memanggil file header menggunakan sintaks `#include "single_linked_list.h"` agar seluruh struktur data dan fungsi yang telah dideklarasikan pada file header dapat digunakan di dalam program

utama. Kemudian pada baris [2], library `iostream` digunakan untuk mendukung proses input dan output seperti menampilkan data ke layar.

Sintaks `using namespace std;` pada baris [4] digunakan agar penulisan fungsi standar C++ seperti `cout` dan `endl` tidak perlu diawali dengan `std::`. Hal ini membuat penulisan kode menjadi lebih sederhana dan mudah dibaca.

Fungsi utama program dimulai pada baris [6] melalui `int main()`. Fungsi `main()` merupakan titik awal eksekusi program C++. Seluruh perintah yang berada di dalam fungsi ini akan dijalankan pertama kali ketika program dieksekusi.

Pada baris [8], program membuat sebuah objek bernama `numberList` dari struktur `SingleLinkedList` menggunakan sintaks `SingleLinkedList numberList;`. Objek ini nantinya digunakan untuk mengakses seluruh fungsi linked list seperti menambah, menghapus, dan menampilkan data.

Selanjutnya pada baris [10], fungsi `numberList.init();` dipanggil untuk melakukan inisialisasi awal linked list. Fungsi ini mengatur `head` dan `tail` menjadi kosong (`nullptr`) serta mengatur ukuran linked list menjadi 0. Proses ini penting agar linked list berada dalam kondisi siap digunakan sebelum dilakukan operasi lainnya.

Pada baris [12] dan [13], program menambahkan data ke bagian belakang linked list menggunakan fungsi `add_back()`. Nilai 10 dimasukkan terlebih dahulu, kemudian diikuti nilai 20. Setelah itu pada baris [14], fungsi `add_front(5)` digunakan untuk menambahkan data 5 di bagian depan linked list. Dengan proses tersebut, urutan data di dalam linked list menjadi 5 -> 10 -> 20. Fungsi `numberList.display();` pada baris [16] digunakan untuk menampilkan seluruh isi linked list ke layar. Fungsi ini akan membaca node satu per satu mulai dari `head` hingga kembali lagi ke `head` karena struktur yang digunakan adalah circular linked list.

SOAL 2 : Si Palui dan Relikui Siklus Resonansi

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui terjebak di dalam sebuah kuil kuno yang dikelilingi oleh N buah batu relikui yang membentuk formasi lingkaran sempurna. Untuk membuka pintu keluar, ia harus menghancurkan batu-batu tersebut dengan urutan tertentu menggunakan pancaran energi beruntun.

Pancaran energi bermula dari batu pertama, lalu melompat sejauh K langkah ke arah depan, lalu menghancurkan batu target tersebut. Namun, kuil ini memiliki anomali energi dinamis:

- Jika nilai ukiran pada batu yang baru saja dihancurkan adalah bilangan genap, maka lompatan berikutnya akan bertambah jauh ($K = K + 1$).
- Jika nilai ukiran pada batu yang dihancurkan adalah bilangan ganjil, maka lompatan berikutnya akan melemah ($K = K - 1$).
- Jika pada suatu titik energi lompatan habis atau negatif ($K \leq 0$), sistem akan mengalami **reset** dan mengembalikan nilai lompatan ke nilai K awal (saat pancaran energi pertama kali diinisialisasi).

Setelah seluruh batu hancur kecuali satu, batu terakhir itulah yang akan menjadi kunci pintu keluar.

Batasan

- $1 \leq N \leq 10^4$
- $1 \leq K \leq 10^9$
- $1 \leq A_i \leq 10^6$ (Nilai ukiran pada batu)
- Program wajib menggunakan implementasi struktur data yang dibuat dari Soal1

Format Input

N K
A1 A2 ... AN

Format Output

Sebuah bilangan bulat tunggal yang merupakan nilai ukiran pada batu terakhir yang tersisa.

Contoh Kasus

Input	Output
5 2 1 4 3 6 2	6
4 5 10 11 12 13	12

Penjelasan Contoh Kasus

Pada contoh pertama, susunan batu awal adalah $[1,4,3,6,2]$ dengan $K = 2$. Dari batu pertama, lompatan 2 langkah mendarat di batu 4 sehingga batu tersebut dihancurkan. Karena 4 genap, nilai K bertambah menjadi 3 dan susunan berubah menjadi $[1,3,6,2]$.

Selanjutnya, dengan $K = 3$, lompatan mendarat di batu 2. Batu 2 dihancurkan dan karena genap, K kembali bertambah menjadi 4. Susunan kini menjadi $[1,3,6]$.

Pada fase berikutnya dengan $K = 4$, lompatan mendarat di batu 1. Karena 1 ganjil, setelah dihancurkan nilai K berkurang menjadi 3, menyisakan $[3,6]$.

Terakhir, dengan $K = 3$, lompatan kembali mendarat di batu 3 sehingga batu tersebut dihancurkan. Struktur kini hanya menyisakan batu bernilai 6, sehingga keluaran akhirnya adalah 6.

A. Source Code

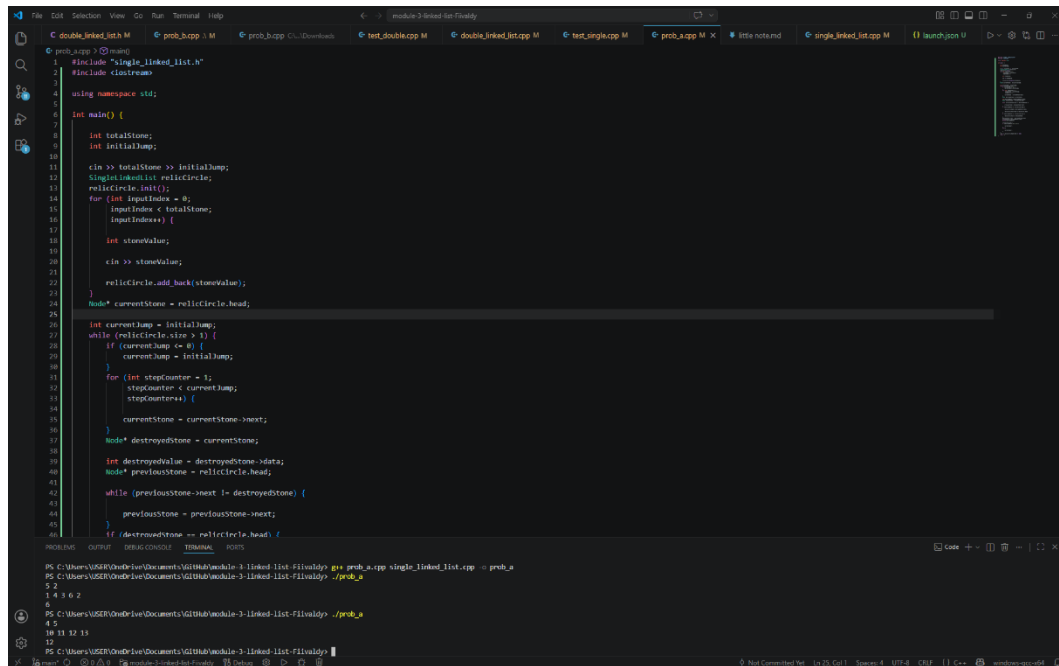
Tabel 18 Source Code File prob_a.cpp

1	<code>#include "single_linked_list.h"</code>
2	<code>#include <iostream></code>
3	
4	<code>using namespace std;</code>
5	
6	<code>int main() {</code>
7	
8	<code> int totalStone;</code>
9	<code> int initialJump;</code>
10	
11	<code> cin >> totalStone >> initialJump;</code>
12	<code> SingleLinkedList relicCircle;</code>
13	<code> relicCircle.init();</code>
14	<code> for (int inputIndex = 0;</code>
15	<code> inputIndex < totalStone;</code>
16	<code> inputIndex++) {</code>
17	
18	<code> int stoneValue;</code>
19	
20	<code> cin >> stoneValue;</code>
21	
22	<code> relicCircle.add_back(stoneValue);</code>
23	<code> }</code>
24	<code> Node* currentStone = relicCircle.head;</code>
25	
26	<code> int currentJump = initialJump;</code>
27	<code> while (relicCircle.size > 1) {</code>
28	<code> if (currentJump <= 0) {</code>
29	<code> currentJump = initialJump;</code>
30	<code> }</code>

31	for (int stepCounter = 1;
32	stepCounter < currentJump;
33	stepCounter++) {
34	
35	currentStone = currentStone->next;
36	}
37	Node* destroyedStone = currentStone;
38	
39	int destroyedValue = destroyedStone->data;
40	Node* previousStone = relicCircle.head;
41	
42	while (previousStone->next !=
43	destroyedStone) {
44	previousStone = previousStone->next;
45	}
46	if (destroyedStone == relicCircle.head) {
47	
48	relicCircle.head = destroyedStone-
49	>next;
50	relicCircle.tail->next =
51	relicCircle.head;
52	}
53	if (destroyedStone == relicCircle.tail) {
54	relicCircle.tail = previousStone;
55	}
56	previousStone->next = destroyedStone-
57	>next;
58	currentStone = destroyedStone->next;
	delete destroyedStone;

59	
60	relicCircle.size--;
61	if (destroyedValue % 2 == 0) {
62	
63	currentJump++;
64	}
65	else {
66	
67	currentJump--;
68	}
69	}
70	cout << relicCircle.head->data << endl;
71	return 0;
72	}

B. Output Program



```
1 #include "single_linked_list.h"
2 #include <iostream>
3
4 using namespace std;
5
6 int main() {
7     int totalStone;
8     int initialJump;
9
10    cin >> totalStone >> initialJump;
11    SingleLinkedList relicCircle;
12    relicCircle.init();
13    for (int inputIndex = 0;
14         inputIndex < totalStone;
15         inputIndex++) {
16        int stoneValue;
17        cin >> stoneValue;
18        relicCircle.add_back(stoneValue);
19    }
20    Node* currentStone = relicCircle.head;
21
22    int currentJump = initialJump;
23    while (relicCircle.size > 1) {
24        if (currentJump <= 0) {
25            currentJump = initialJump;
26        }
27        for (int stepCounter = 1;
28             stepCounter < currentJump;
29             stepCounter++) {
30            currentStone = currentStone->next;
31        }
32        Node* destroyedStone = currentStone;
33        int destroyedValue = destroyedStone->data;
34        Node* previousStone = relicCircle.head;
35        while (previousStone->next != destroyedStone) {
36            previousStone = previousStone->next;
37        }
38        if (destroyedStone == relicCircle.head) {
39            relicCircle.head = previousStone->next;
40        }
41        previousStone->next = previousStone->next->next;
42        delete destroyedStone;
43    }
44    cout << "The remaining stone is: " << relicCircle.head->data << endl;
45    return 0;
46 }
```

Output: 5 2
1 4 3 6 2
5
4 5
18 11 12 13
12

Gambar 6 Screenshot Output Program Soal 2

C. Pembahasan

Secara alur, program bekerja dengan cara menyusun seluruh batu ke dalam circular linked list, kemudian melakukan perpindahan sesuai nilai lompatan yang terus berubah berdasarkan aturan genap dan ganjil. Setiap kali pointer berhenti pada suatu batu, batu tersebut dihancurkan dan dihapus dari linked list. Proses ini berlangsung berulang sampai hanya tersisa satu batu terakhir sebagai hasil akhir program.

Kode pada file `prob_a.cpp` merupakan program simulasi penghancuran batu relik menggunakan struktur data Circular Single Linked List. Pada baris [1], file `single_linked_list.h` dipanggil agar seluruh fungsi dan struktur linked list yang telah dibuat sebelumnya dapat digunakan kembali dalam program. Kemudian library `iostream` pada baris [2] digunakan untuk proses input dan output data.

Fungsi utama program dimulai pada baris [6] melalui `int main()`. Pada bagian ini program pertama kali membaca jumlah batu dan nilai awal lompatan menggunakan `cin >> totalStone >> initialJump`; pada baris [11]. Variabel `totalStone` menyimpan jumlah batu relik, sedangkan `initialJump` digunakan sebagai nilai awal banyaknya langkah perpindahan. Pada baris [12], program membuat objek `relicCircle` dari struktur `SingleLinkedList`. Objek ini digunakan untuk menyimpan seluruh batu relik dalam bentuk circular linked list. Setelah itu fungsi `relicCircle.init()`; pada baris [13] dipanggil untuk memastikan linked list berada dalam kondisi kosong sebelum data dimasukkan. Proses input batu dilakukan menggunakan perulangan `for` pada baris [14] hingga [23]. Pada setiap perulangan, program membaca nilai batu menggunakan `cin >> stoneValue`; pada baris [20]. Nilai tersebut kemudian dimasukkan ke bagian belakang linked list menggunakan fungsi `relicCircle.add_back(stoneValue)`; pada baris [22]. Dengan cara ini seluruh batu tersusun membentuk lingkaran sesuai aturan soal. Pada baris [24], pointer `currentStone` diatur menuju `relicCircle.head`. Pointer ini digunakan untuk menandai posisi batu yang sedang aktif pada proses perpindahan. Kemudian nilai lompatan saat ini disimpan pada variabel `currentJump` pada baris [26].

Perulangan utama program berada pada baris [27] menggunakan kondisi `while (relicCircle.size > 1)`. Perulangan ini akan terus berjalan selama jumlah batu masih lebih dari satu. Artinya proses penghancuran batu dilakukan terus menerus sampai hanya tersisa satu batu terakhir. Pada baris [28] hingga [30], program memeriksa apakah nilai lompatan sudah habis atau bernilai kurang dari sama dengan nol. Jika kondisi tersebut terjadi, maka nilai lompatan dikembalikan lagi ke nilai awal menggunakan `currentJump = initialJump`; Hal ini sesuai aturan pada soal bahwa energi akan kembali ke nilai awal ketika sudah tidak dapat digunakan.

Proses perpindahan batu dilakukan pada baris [31] hingga [35] menggunakan perulangan `for`. Pointer `currentStone` dipindahkan maju menggunakan sintaks `currentStone = currentStone->next`; pada baris [35]. Perulangan dimulai dari angka 1 karena posisi awal pointer sudah dianggap sebagai langkah pertama. Setelah perpindahan selesai, batu tempat pointer berhenti akan menjadi target penghancuran. Pada baris [37], node target disimpan ke dalam pointer `destroyedStone`. Nilai batu tersebut juga disimpan pada variabel `destroyedValue` di baris [39] karena nantinya digunakan untuk menentukan perubahan nilai lompatan. Selanjutnya program mencari node sebelum target menggunakan perulangan `while` pada baris [42] hingga [45]. Langkah ini diperlukan agar hubungan antar node tetap tersambung setelah node target dihapus.

Jika batu yang dihancurkan merupakan head, maka posisi head dipindahkan ke node berikutnya menggunakan `relicCircle.head = destroyedStone->next`; pada baris [48]. Setelah itu hubungan circular diperbaiki melalui `relicCircle.tail->next = relicCircle.head`; pada baris [50]. Sementara itu jika batu target merupakan tail, maka posisi tail dipindahkan ke node sebelumnya menggunakan `relicCircle.tail = previousStone`; pada baris [54]. Pada baris [56], hubungan linked list diperbaiki menggunakan `previousStone->next = destroyedStone->next`; agar node yang dihancurkan tidak lagi terhubung dengan list. Kemudian pointer dipindahkan ke node berikutnya menggunakan `currentStone = destroyedStone->next`; pada baris [57]. Node target lalu dihapus dari memori menggunakan `delete destroyedStone`; pada baris [58] agar tidak terjadi kebocoran memori. Jumlah node dalam linked list dikurangi pada baris [60] menggunakan `relicCircle.size--`; Setelah penghancuran selesai, program memeriksa apakah nilai batu yang dihancurkan genap atau ganjil

pada baris [61]. Jika genap maka nilai lompatan ditambah satu menggunakan `currentJump++`; pada baris [63]. Namun jika ganjil maka nilai lompatan dikurangi satu menggunakan `currentJump--`; pada baris [67]. Ketika hanya tersisa satu batu, program menampilkan nilainya menggunakan `cout << relicCircle.head->data << endl`; pada baris [70]. Nilai tersebut merupakan batu terakhir yang bertahan setelah seluruh proses penghancuran selesai dilakukan.

SOAL 3

Pada file `double_linked_list.h` yang diberikan, implementasikan struktur `DoubleLinkedList`. beserta fungsi-fungsinya ke dalam file `double_linked_list.cpp`. Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap. Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

Tabel 19 Source Code File `double_linked_list.h`

1	<code>#ifndef DOUBLE_LINKED_LIST</code>
2	<code>#define DOUBLE_LINKED_LIST</code>
3	
4	<code>struct Node {</code>
5	<code> char data;</code>
6	<code> Node* next;</code>
7	<code> Node* prev;</code>
8	<code>};</code>
9	
10	<code>struct DoubleLinkedList {</code>
11	
12	<code> Node* head = nullptr;</code>
13	<code> Node* tail = nullptr;</code>
14	
15	<code> int size = 0;</code>
16	
17	<code> void init();</code>
18	<code> bool is_empty();</code>
19	
20	<code> void add_front(char inputValue);</code>

21	void add_back(char inputValue);
22	void add_idx(char inputValue, int indexPosition);
23	
24	void delete_front();
25	void delete_back();
26	void delete_idx(int indexPosition);
27	
28	void display();
29	
30	char get(int indexPosition);
31	
32	void set(char inputValue, int indexPosition);
33	
34	void clear();
35	};
36	
37	#endif

Tabel 20 Source Code File double_linked_list.h

1	#include "double_linked_list.h"
2	#include <iostream>
3	
4	using namespace std;
5	void DoubleLinkedList::init() {
6	
7	head = nullptr;
8	tail = nullptr;
9	
10	size = 0;
11	}


```

12 bool DoubleLinkedList::is_empty() {
13
14     return head == nullptr;
15 }
16
17 void DoubleLinkedList::add_front(char inputValue)
18 {
19     Node* newNode = new Node;
20     newNode->data = inputValue;
21     if (is_empty()) {
22         newNode->next = newNode;
23         newNode->prev = newNode;
24
25         head = newNode;
26         tail = newNode;
27     }
28     else {
29         newNode->next = head;
30         newNode->prev = tail;
31         head->prev = newNode;
32         tail->next = newNode;
33         head = newNode;
34     }
35
36     size++;
37 }
38
39 void DoubleLinkedList::add_back(char inputValue)
40 {
41     Node* newNode = new Node;

```

```

42
43     newNode->data = inputValue;
44     if (is_empty()) {
45
46         newNode->next = newNode;
47         newNode->prev = newNode;
48
49         head = newNode;
50         tail = newNode;
51     }
52     else {
53         newNode->next = head;
54         newNode->prev = tail;
55         tail->next = newNode;
56         head->prev = newNode;
57         tail = newNode;
58     }
59
60     size++;
61 }
62
63 void DoubleLinkedList::add_idx(char inputValue,
                                int indexPosition)
64 {
65     if (indexPosition < 0 ||
66         indexPosition > size) {
67
68         cout << "Index tidak valid\n";
69         return;
70     }
71     if (indexPosition == 0) {
72

```

73	add_front(inputValue);
74	return;
75	}
76	
77	if (indexPosition == size) {
78	
79	add_back(inputValue);
80	return;
81	}
82	
83	Node* currentNode = head;
84	for (int currentIndex = 0;
85	currentIndex < indexPosition;
86	currentIndex++) {
87	
88	currentNode = currentNode->next;
89	}
90	
91	Node* newNode = new Node;
92	
93	newNode->data = inputValue;
94	newNode->next = currentNode;
95	newNode->prev = currentNode->prev;
96	
97	currentNode->prev->next = newNode;
98	currentNode->prev = newNode;
99	
100	size++;
101	}
102	void DoubleLinkedList::delete_front() {
103	if (is_empty()) {
104	

```

105         cout << "Linked list kosong\n";
106         return;
107     }
108
109     if (head == tail) {
110
111         delete head;
112
113         head = nullptr;
114         tail = nullptr;
115     }
116     else {
117
118         Node* deletedNode = head;
119         head = head->next;
120         head->prev = tail;
121         tail->next = head;
122
123         delete deletedNode;
124     }
125
126     size--;
127 }
128 void DoubleLinkedList::delete_back() {
129     if (is_empty()) {
130
131         cout << "Linked list kosong\n";
132         return;
133     }
134     if (head == tail) {
135
136         delete tail;

```

```

137
138         head = nullptr;
139         tail = nullptr;
140     }
141     else {
142         Node* deletedNode = tail;
143         tail = tail->prev;
144         tail->next = head;
145         head->prev = tail;
146
147         delete deletedNode;
148     }
149
150     size--;
151 }
152 void DoubleLinkedList::delete_idx(int
indexPosition) {
153     if (indexPosition < 0 ||
154         indexPosition >= size) {
155
156         cout << "Index tidak valid\n";
157         return;
158     }
159     if (indexPosition == 0) {
160
161         delete_front();
162         return;
163     }
164     if (indexPosition == size - 1) {
165
166         delete_back();
167         return;

```

```

168     }
169
170     Node* currentNode = head;
171     for (int currentIndex = 0;
172         currentIndex < indexPosition;
173         currentIndex++) {
174
175         currentNode = currentNode->next;
176     }
177     currentNode->prev->next = currentNode->next;
178     currentNode->next->prev = currentNode->prev;
179
180     delete currentNode;
181
182     size--;
183 }
184 void DoubleLinkedList::display() {
185     if (is_empty()) {
186
187         cout << "Linked list kosong\n";
188         return;
189     }
190
191     Node* currentNode = head;
192
193     cout << "Isi Linked List : ";
194     do {
195
196         cout << currentNode->data << " ";
197
198         currentNode = currentNode->next;
199

```

```

200     } while (currentNode != head);
201
202     cout << endl;
203 }
204 char DoubleLinkedList::get(int indexPosition) {
205     if (indexPosition < 0 ||
206         indexPosition >= size) {
207
208         cout << "Index tidak valid\n";
209         return '\0';
210     }
211
212     Node* currentNode = head;
213     for (int currentIndex = 0;
214         currentIndex < indexPosition;
215         currentIndex++) {
216
217         currentNode = currentNode->next;
218     }
219
220     return currentNode->data;
221 }
222 void DoubleLinkedList::set(char inputValue,
223                             int indexPosition) {
224
225     if (indexPosition < 0 ||
226         indexPosition >= size) {
227
228         cout << "Index tidak valid\n";
229         return;
230     }
231

```

232	Node* currentNode = head;
233	for (int currentIndex = 0;
234	currentIndex < indexPosition;
235	currentIndex++) {
236	
237	currentNode = currentNode->next;
238	}
239	
240	currentNode->data = inputValue;
241	}
242	
243	void DoubleLinkedList::clear() {
244	while (!is_empty()) {
245	
246	delete_front();
247	}
248	
249	head = nullptr;
250	tail = nullptr;
251	
252	size = 0;
253	}

Tabel 21 Source Code File test_double.cpp

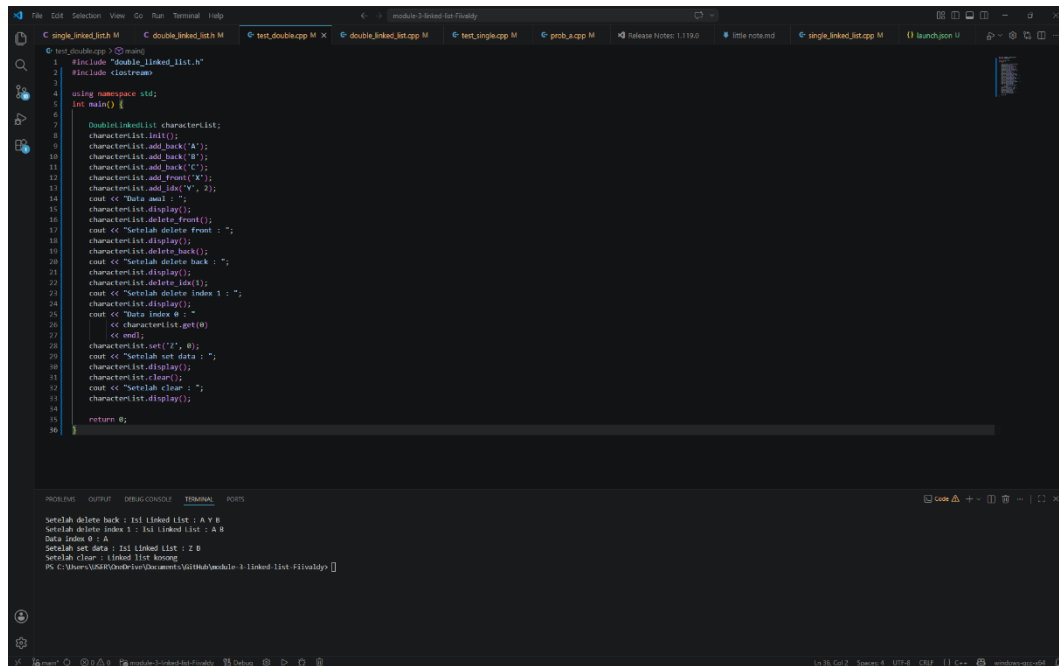
1	#include "double_linked_list.h"
2	#include <iostream>
3	
4	using namespace std;
5	int main() {
6	
7	DoubleLinkedList characterList;
8	characterList.init();


```

9      characterList.add_back('A');
10     characterList.add_back('B');
11     characterList.add_back('C');
12     characterList.add_front('X');
13     characterList.add_idx('Y', 2);
14     cout << "Data awal : ";
15     characterList.display();
16     characterList.delete_front();
17     cout << "Setelah delete front : ";
18     characterList.display();
19     characterList.delete_back();
20     cout << "Setelah delete back : ";
21     characterList.display();
22     characterList.delete_idx(1);
23     cout << "Setelah delete index 1 : ";
24     characterList.display();
25     cout << "Data index 0 : "
26           << characterList.get(0)
27           << endl;
28     characterList.set('Z', 0);
29     cout << "Setelah set data : ";
30     characterList.display();
31     characterList.clear();
32     cout << "Setelah clear : ";
33     characterList.display();
34
35     return 0;
36 }

```

B. Output Program



```
1 #include "double_linked_list.h"
2 #include <iostream>
3
4 using namespace std;
5 int main() {
6
7     DoubleLinkedList characterList;
8     characterList.init();
9     characterList.add_back("A");
10    characterList.add_back("B");
11    characterList.add_back("C");
12    characterList.add_front("D");
13    characterList.add_id("V", 2);
14    cout << "Data awal : ";
15    characterList.display();
16    characterList.delete_front();
17    cout << "Setelah delete front : ";
18    characterList.display();
19    characterList.delete_back();
20    cout << "Setelah delete back : ";
21    characterList.display();
22    characterList.delete_id(1);
23    cout << "Setelah delete index 1 : ";
24    characterList.display();
25    cout << "Data index 0 : ";
26    << characterList.get(0)
27    << endl;
28    characterList.set("Z", 0);
29    cout << "Setelah set data : ";
30    characterList.display();
31    characterList.clear();
32    cout << "Setelah clear : ";
33    characterList.display();
34
35    return 0;
36 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Setelah delete back : Isi linked list : A V 0
Setelah delete index 1 : Isi linked list : A 0
Data index 0 : A
Setelah set data : Isi linked list : Z 0
Setelah clear : linked list kosong
PS C:\Users\USIR\Desktop\Documents\GitHub\module-3-linked-list> g++ .\

Gambar 7 Screenshot Output Program Soal 3

C. Pembahasan

Implementasi DoubleLinkedList pada program menggunakan struktur data *Circular Doubly Linked List*, yaitu linked list yang setiap node memiliki dua hubungan sekaligus, yaitu menuju node berikutnya (next) dan node sebelumnya (prev). Selain itu node terakhir juga kembali terhubung ke node pertama sehingga membentuk struktur melingkar. Seluruh node pada program dialokasikan secara manual di dalam heap menggunakan sintaks `new Node`. Penggunaan heap dipilih karena jumlah data pada linked list bersifat dinamis sehingga ukuran penyimpanan dapat bertambah maupun berkurang selama program berjalan tanpa perlu menentukan kapasitas tetap sejak awal.

Pada setiap proses penambahan data seperti `add_front()`, `add_back()`, dan `add_idx()`, program membuat node baru menggunakan alokasi dinamis. Karena memori dibuat secara manual, maka node yang sudah tidak digunakan wajib dihapus menggunakan `delete` agar tidak terjadi *memory leak*. Oleh sebab itu fungsi penghapusan seperti `delete_front()`, `delete_back()`, `delete_idx()`, dan `clear()` memiliki peran penting untuk membebaskan memori yang sebelumnya telah dialokasikan di heap. Fungsi `clear()` bekerja dengan cara menghapus node satu per satu hingga linked list kosong. Dengan cara ini seluruh memori yang tidak lagi digunakan dapat dikembalikan ke sistem sehingga program menjadi lebih aman dan efisien.

Fungsi `init()` memiliki kompleksitas waktu $O(1)$ karena hanya mengatur nilai head, tail, dan size tanpa melakukan perulangan. Kompleksitas ruangnya juga $O(1)$ karena tidak menggunakan memori tambahan selain variabel yang sudah ada. Fungsi `is_empty()` mempunyai kompleksitas waktu $O(1)$ karena hanya melakukan satu kali pengecekan kondisi pada pointer head. Kompleksitas ruangnya juga tetap $O(1)$. Pada fungsi `add_front()` dan `add_back()`, kompleksitas waktunya adalah $O(1)$ karena penambahan node dilakukan langsung pada bagian depan atau belakang tanpa perlu menelusuri seluruh linked list. Kompleksitas ruangnya adalah $O(1)$ karena hanya membutuhkan satu node baru sebagai tambahan memori. Fungsi `add_idx()` memiliki kompleksitas waktu $O(n)$ karena program harus bergerak menuju posisi index tertentu sebelum menyisipkan node baru. Pada kondisi terburuk, traversal dapat berjalan hingga mendekati seluruh isi linked list.

Kompleksitas ruangnya tetap $O(1)$ karena hanya menggunakan satu node tambahan. Pada fungsi `delete_front()` dan `delete_back()`, kompleksitas waktunya adalah $O(1)$ karena node depan maupun belakang dapat langsung dihapus menggunakan bantuan pointer `head` dan `tail`. Kompleksitas ruang kedua fungsi tersebut tetap $O(1)$. Fungsi `delete_idx()` memiliki kompleksitas waktu $O(n)$ karena program harus melakukan traversal menuju posisi node target sebelum node dapat dihapus. Kompleksitas ruangnya tetap $O(1)$ karena tidak membutuhkan struktur data tambahan. Pada fungsi `display()`, kompleksitas waktunya adalah $O(n)$ karena seluruh node harus ditampilkan satu per satu hingga kembali ke `head`. Kompleksitas ruangnya tetap $O(1)$. Fungsi `get()` dan `set()` juga memiliki kompleksitas waktu $O(n)$ karena program perlu bergerak menuju index tertentu sebelum mengambil atau mengubah data. Kompleksitas ruang kedua fungsi tersebut tetap $O(1)$. Sementara itu fungsi `clear()` memiliki kompleksitas waktu $O(n)$ karena setiap node harus dihapus satu per satu sampai linked list kosong. Kompleksitas ruangnya adalah $O(1)$ karena penghapusan dilakukan langsung tanpa menggunakan struktur data tambahan.

Algoritma yang dirancang pada implementasi ini dapat dianggap valid karena setiap operasi selalu menjaga hubungan dua arah (`next` dan `prev`) antar node serta mempertahankan sifat circular antara `head` dan `tail`. Ketika node ditambahkan atau dihapus, pointer `next` dan `prev` selalu diperbarui sehingga hubungan antar node tetap benar dan tidak terputus. Validasi index pada fungsi seperti `add_idx()`, `delete_idx()`, `get()`, dan `set()` juga membantu mencegah akses data di luar batas linked list. Selain itu penggunaan `delete` pada setiap proses penghapusan memastikan memori yang sudah tidak digunakan dapat dibersihkan dengan benar sehingga program tidak mengalami kebocoran memori.

SOAL 4

Si Palui dan Tarik Tambang Dimensi

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui mengamati fenomena aneh di atas sebuah gelanggang melingkar yang berisi N karakter. Terdapat dua proyektor anomali, Alpha (dimulai dari head, bergerak maju menggunakan next) dan Beta (dimulai dari tail, bergerak mundur menggunakan prev).

Palui melakukan observasi selama R ronde. Pada setiap ronde:

1. Proyektor Alpha dipaksa melompat maju sebanyak X langkah. Proyektor Beta dipaksa melompat mundur sebanyak Y langkah (simultan).
2. Tabrakan Singular: Jika Alpha dan Beta mendarat di titik atau karakter yang sama persis secara bersamaan, karakter tersebut akan hancur dan lenyap dari lingkaran dimensi. Jika ini terjadi, Alpha berpindah ke karakter **next** dari karakter yang hancur, dan Beta berpindah ke karakter **prev** dari karakter yang hancur.
3. Resonansi Bersebelahan (Adjacent): Jika Alpha dan Beta saling bersebelahan langsung, sifat kuantum dari kedua karakter tersebut saling bertukar (data antara kedua node tersebut di-swap).

Tentukan sisa karakter pada lingkaran dimensi setelah R ronde selesai dieksekusi.

Batasan

- $1 \leq N \leq 10^5$
- $1 \leq R \leq 10^5$
- $1 \leq X, Y \leq 10^{18}$
- Struktur wajib menggunakan implementasi struktur data dari Soal 3.

Format Input

N R

C1 C2 ... CN

X1 Y1

X2 Y2

...

XR YR

Keterangan: C_i adalah satu karakter char tunggal.

Format Output

Karakter yang tersisa, dicetak berurutan mulai dari head hingga ke elemen terakhir, tanpa spasi. Jika dimensi menjadi kosong, cetak EMPTY

CONTOH KASUS

Input	Output
4 2 A B C D 2 3 1 1	BADC
5 3 V W X Y Z 100000 100000 2 2 5 5	XWZY

A. Source Code

Tabel 22 Source Code File prob_b.cpp

1	#include "double_linked_list.h"
2	#include <iostream>
3	
4	using namespace std;
5	void deleteNode(DoubleLinkedList &dll, Node*
6	target) {
7	if (dll.size == 1) {
8	delete target;
9	dll.head = nullptr;
10	dll.tail = nullptr;
11	dll.size = 0;
12	return;
13	}
14	
15	target->prev->next = target->next;
16	target->next->prev = target->prev;
17	
18	if (target == dll.head)
19	dll.head = target->next;
20	
21	if (target == dll.tail)
22	dll.tail = target->prev;
23	
24	delete target;
25	dll.size--;
26	}
27	
28	void swapAdjacent(DoubleLinkedList &dll, Node* a,
	Node* b) {

```

29     if (b->next == a) {
30         Node* temp = a;
31         a = b;
32         b = temp;
33     }
34
35     Node* before = a->prev;
36     Node* after = b->next;
37
38     before->next = b;
39     b->prev = before;
40
41     b->next = a;
42     a->prev = b;
43
44     a->next = after;
45     after->prev = a;
46
47     if (dll.head == a)
48         dll.head = b;
49
50     if (dll.tail == b)
51         dll.tail = a;
52 }
53
54 int main() {
55
56     DoubleLinkedList dll;
57     dll.init();
58
59     int N, R;
60     cin >> N >> R;

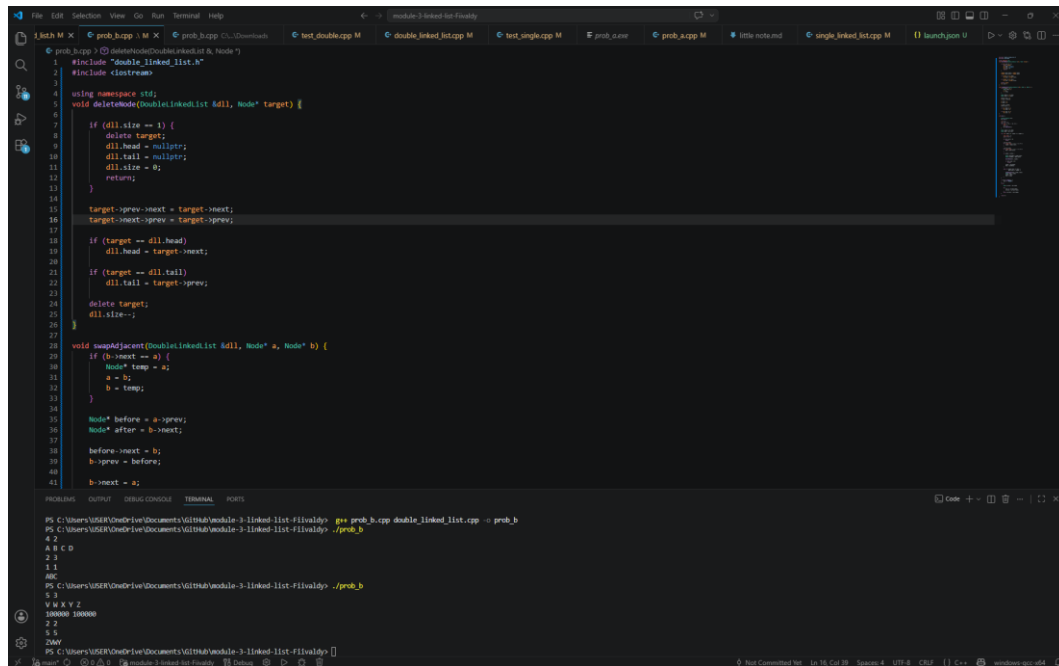
```


61	for (int i = 0; i < N; i++) {
62	char c;
63	cin >> c;
64	dll.add_back(c);
65	}
66	
67	Node* alpha = dll.head;
68	Node* beta = dll.tail;
69	
70	for (int ronde = 0; ronde < R; ronde++) {
71	
72	long long X, Y;
73	cin >> X >> Y;
74	
75	if (dll.size == 0)
76	break;
77	
78	X %= dll.size;
79	for (long long i = 0; i < X; i++) {
80	alpha = alpha->next;
81	}
82	
83	Y %= dll.size;
84	for (long long i = 0; i < Y; i++) {
85	beta = beta->prev;
86	}
87	
88	if (alpha == beta) {
89	
90	Node* nextAlpha = alpha->next;
91	Node* prevBeta = beta->prev;
92	

93	deleteNode(dll, alpha);
94	
95	if (dll.size == 0)
96	break;
97	
98	alpha = nextAlpha;
99	beta = prevBeta;
100	}
101	
102	else if (alpha->next == beta
103	beta->next == alpha) {
104	
105	swapAdjacent(dll, alpha, beta);
106	Node* temp = alpha;
107	alpha = beta;
108	beta = temp;
109	}
110	}
111	
112	if (dll.is_empty()) {
113	cout << "KOSONG";
114	}
115	else {
116	
117	Node* current = dll.head;
118	
119	do {
120	cout << current->data;
121	current = current->next;
122	}
123	while (current != dll.head);
124	}

125	
126	return 0;
127	}

B. Output Program



```
1 #include <iostream>
2 #include "double_linked_list.h"
3
4 using namespace std;
5 void deleteNode(DoubleLinkedList &dll, Node* target) {
6     if (dll.size == 1) {
7         delete target;
8         dll.head = nullptr;
9         dll.tail = nullptr;
10        dll.size = 0;
11        return;
12    }
13    target->prev->next = target->next;
14    target->next->prev = target->prev;
15
16    if (target == dll.head)
17        dll.head = target->next;
18    if (target == dll.tail)
19        dll.tail = target->prev;
20    delete target;
21    dll.size--;
22 }
23
24 void swapAdjacent(DoubleLinkedList &dll, Node* a, Node* b) {
25     if (a->next == b) {
26         Node* temp = a;
27         a = b;
28         b = temp;
29     }
30     Node* before = a->prev;
31     Node* after = b->next;
32     before->next = b;
33     b->prev = before;
34     b->next = a;
35     a->prev = after;
36 }
37
38 int main() {
39     DoubleLinkedList dll;
40     Node* a = new Node('A');
41     Node* b = new Node('B');
42     Node* c = new Node('C');
43     Node* d = new Node('D');
44     Node* e = new Node('E');
45     Node* f = new Node('F');
46     Node* g = new Node('G');
47     Node* h = new Node('H');
48     Node* i = new Node('I');
49     Node* j = new Node('J');
50     Node* k = new Node('K');
51     Node* l = new Node('L');
52     Node* m = new Node('M');
53     Node* n = new Node('N');
54     Node* o = new Node('O');
55     Node* p = new Node('P');
56     Node* q = new Node('Q');
57     Node* r = new Node('R');
58     Node* s = new Node('S');
59     Node* t = new Node('T');
60     Node* u = new Node('U');
61     Node* v = new Node('V');
62     Node* w = new Node('W');
63     Node* x = new Node('X');
64     Node* y = new Node('Y');
65     Node* z = new Node('Z');
66     dll.add(a);
67     dll.add(b);
68     dll.add(c);
69     dll.add(d);
70     dll.add(e);
71     dll.add(f);
72     dll.add(g);
73     dll.add(h);
74     dll.add(i);
75     dll.add(j);
76     dll.add(k);
77     dll.add(l);
78     dll.add(m);
79     dll.add(n);
80     dll.add(o);
81     dll.add(p);
82     dll.add(q);
83     dll.add(r);
84     dll.add(s);
85     dll.add(t);
86     dll.add(u);
87     dll.add(v);
88     dll.add(w);
89     dll.add(x);
90     dll.add(y);
91     dll.add(z);
92     dll.print();
93     return 0;
94 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL RUNS

```
PS C:\Users\USER\OneDrive\Documents\GitHub\module-3-linked-list-1\invalid> g++ prob_b.cpp double_linked_list.cpp -o prob_b
4 2
A B C D
2 3
1 1
000
5 1
V W X Y Z
100000 100000
2 2
5 1
20000
PS C:\Users\USER\OneDrive\Documents\GitHub\module-3-linked-list-1\invalid> ./prob_b
```

Gambar 8 Output Program Soal 4

C. Pembahasan

Program pada file `prob_b.cpp` digunakan untuk mensimulasikan pergerakan dua proyektor bernama Alpha dan Beta pada sebuah struktur data circular double linked list. Struktur data ini memungkinkan perpindahan node dilakukan ke arah kanan maupun kiri secara melingkar. Pada bagian awal program di baris [1]-[4], program melakukan import library dan menggunakan namespace standar C++. Library "`double_linked_list.h`" digunakan karena seluruh operasi linked list seperti penambahan node dan penghapusan node sudah dibuat pada file tersebut.

Pada baris [6]-[28], dibuat fungsi `deleteNode()` yang berfungsi untuk menghapus node tertentu secara langsung menggunakan pointer. Fungsi ini menerima parameter berupa objek linked list dan node target yang ingin dihapus. Pada kondisi di baris [8]-[14], program memeriksa apakah linked list hanya memiliki satu node. Jika benar, maka node langsung dihapus dan pointer head serta tail dikosongkan. Jika jumlah node lebih dari satu, maka pada baris [16]-[25] dilakukan penyambungan ulang antar node sebelum node target dihapus. Proses ini penting agar linked list tetap tersambung secara melingkar setelah penghapusan node dilakukan.

Selanjutnya pada baris [30]-[53], terdapat fungsi `swapAdjacent()` yang digunakan untuk menukar posisi dua node yang saling bersebelahan. Pada baris [31]-[35], program memastikan bahwa node a selalu berada sebelum node b. Jika ternyata posisi node terbalik, maka pointer ditukar terlebih dahulu. Setelah itu, pada baris [37]-[46], dilakukan pengaturan ulang hubungan pointer next dan prev antar node agar posisi kedua node benar-benar bertukar di dalam linked list. Kemudian pada baris [48]-[52], program memeriksa apakah node yang dipindahkan merupakan head atau tail, sehingga pointer utama linked list juga ikut diperbarui. Fungsi utama program dimulai pada baris [55]. Pada baris [57]-[58], dibuat objek `DoubleLinkedList` bernama `dll` dan dilakukan inisialisasi linked list menggunakan fungsi `init()`. Kemudian pada baris [60]-[61], program membaca input jumlah karakter (N) dan jumlah ronde simulasi (R).

Pada baris [62]-[66], program membaca karakter satu per satu lalu memasukkannya ke bagian belakang linked list menggunakan fungsi `add_back()`.

Dengan cara ini, urutan karakter pada input akan tetap sama seperti data awal yang diberikan pengguna.

Selanjutnya pada baris [68]-[69], ditentukan posisi awal kedua proyektor. Alpha dimulai dari node head, sedangkan Beta dimulai dari node tail. Kedua pointer ini nantinya akan bergerak selama simulasi berlangsung.

Perulangan utama simulasi terdapat pada baris [71]-[107]. Pada setiap ronde, program membaca nilai langkah Alpha (X) dan Beta (Y) pada baris [73]-[74]. Kemudian pada baris [76]-[77], program memeriksa apakah linked list sudah kosong. Jika kosong, maka simulasi dihentikan karena tidak ada lagi node yang dapat diproses. Pergerakan Alpha dilakukan pada baris [79]-[82]. Nilai langkah X terlebih dahulu dimodifikasi menggunakan operasi modulo terhadap ukuran linked list agar perpindahan tetap efisien walaupun jumlah langkah sangat besar. Setelah itu Alpha bergerak maju menggunakan pointer next. Hal yang sama dilakukan untuk Beta pada baris [84]-[87], namun Beta bergerak mundur menggunakan pointer prev.

Pada baris [89]-[101], program memeriksa kemungkinan terjadinya tabrakan. Jika Alpha dan Beta berada pada node yang sama, maka node tersebut akan dihapus menggunakan fungsi `deleteNode()`. Sebelum penghapusan dilakukan, pointer tujuan selanjutnya disimpan terlebih dahulu pada baris [91]-[92] agar Alpha dan Beta masih dapat melanjutkan simulasi setelah node dihapus. Setelah penghapusan selesai, posisi Alpha dan Beta diperbarui pada baris [99]-[100].

Jika tidak terjadi tabrakan, maka program akan memeriksa apakah Alpha dan Beta berada pada node yang saling bersebelahan pada baris [103]-[104]. Jika kondisi tersebut terpenuhi, maka fungsi `swapAdjacent()` dipanggil untuk menukar posisi kedua node tersebut. Setelah posisi node bertukar, pointer Alpha dan Beta juga ikut ditukar pada baris [106]-[107] agar proyektor tetap mengikuti node yang sama.

Bagian akhir program terdapat pada baris [110]-[121]. Pada baris [110]-[112], program memeriksa apakah linked list kosong. Jika kosong, maka program menampilkan tulisan "KOSONG". Namun jika masih terdapat node, maka program melakukan traversal mulai dari head pada baris [115]-[120] dan mencetak seluruh karakter secara berurutan hingga kembali lagi ke head. Hasil akhir inilah yang menjadi output dari simulasi program.

MODUL 4 : Sort

SORTING ALGORITHM

A. Source Code

Tabel 23 Source Code file algorithm.cpp

1	#include "sorting_algorithms.h"
2	#include <algorithm>
3	#include <chrono>
4	
5	using Clock = std::chrono::high_resolution_clock;
6	
7	// BUBBLE SORT
	void bubble_sort(std::vector<int>& data, Metrics&
8	m) {
9	int n = data.size();
10	auto start = Clock::now();
11	
12	for (int i = 0; i < n - 1; i++) {
13	for (int j = 0; j < n - i - 1; j++) {
14	
15	m.comparisons++;
16	
17	if (data[j] > data[j + 1]) {
18	std::swap(data[j], data[j + 1]);
19	m.swaps++;
20	}
21	}
22	}
23	
24	m.time_ms =
25	std::chrono::duration<double, std::milli>(
26	Clock::now() - start).count();

```

27 }
28
29 // SELECTION SORT
void selection_sort(std::vector<int>& data,
30 Metrics& m) {
31     int n = data.size();
32     auto start = Clock::now();
33
34     for (int i = 0; i < n - 1; i++) {
35
36         int minIndex = i;
37
38         for (int j = i + 1; j < n; j++) {
39
40             m.comparisons++;
41
42             if (data[j] < data[minIndex]) {
43                 minIndex = j;
44             }
45         }
46
47         if (minIndex != i) {
48             std::swap(data[i], data[minIndex]);
49             m.swaps++;
50         }
51     }
52
53     m.time_ms =
54         std::chrono::duration<double, std::milli>(
55             Clock::now() - start).count();
56 }
57

```



```

58 // INSERTION SORT
   void insertion_sort(std::vector<int>& data,
59 Metrics& m) {
60     int n = data.size();
61     auto start = Clock::now();
62
63     for (int i = 1; i < n; i++) {
64
65         int key = data[i];
66         int j = i - 1;
67
68         while (j >= 0) {
69
70             m.comparisons++;
71
72             if (data[j] > key) {
73                 data[j + 1] = data[j];
74                 m.shifts++;
75                 j--;
76             }
77             else {
78                 break;
79             }
80         }
81
82         data[j + 1] = key;
83     }
84
85     m.time_ms =
86         std::chrono::duration<double, std::milli>(
87             Clock::now() - start).count();
88 }

```

```

89
90 // MERGE SORT HELPER
91 void merge(std::vector<int>& data,
92           int left,
93           int mid,
94           int right,
95           Metrics& m) {
96
97     int n1 = mid - left + 1;
98     int n2 = right - mid;
99
100    std::vector<int> L(n1);
101    std::vector<int> R(n2);
102
103    for (int i = 0; i < n1; i++)
104        L[i] = data[left + i];
105
106    for (int j = 0; j < n2; j++)
107        R[j] = data[mid + 1 + j];
108
109    int i = 0;
110    int j = 0;
111    int k = left;
112
113    while (i < n1 && j < n2) {
114
115        m.comparisons++;
116
117        if (L[i] <= R[j]) {
118            data[k] = L[i];
119            i++;
120        }

```

```

121         else {
122             data[k] = R[j];
123             j++;
124         }
125
126         k++;
127     }
128
129     while (i < n1) {
130         data[k] = L[i];
131         i++;
132         k++;
133     }
134
135     while (j < n2) {
136         data[k] = R[j];
137         j++;
138         k++;
139     }
140 }
141
142 void merge_sort_recursive(std::vector<int>& data,
143                          int left,
144                          int right,
145                          Metrics& m) {
146
147     m.recursive_calls++;
148
149     if (left >= right)
150         return;
151
152     int mid = left + (right - left) / 2;

```

```

153
154     merge_sort_recursive(data, left, mid, m);
155     merge_sort_recursive(data, mid + 1, right, m);
156
157     merge(data, left, mid, right, m);
158 }
159
160 // MERGE SORT
161 void merge_sort(std::vector<int>& data, Metrics&
162 m) {
163     auto start = Clock::now();
164
165     if (!data.empty()) {
166         merge_sort_recursive(data, 0, data.size()
167 - 1, m);
168     }
169
170     m.time_ms =
171         std::chrono::duration<double, std::milli>(
172             Clock::now() - start).count();
173 }
174
175 // QUICK SORT HELPER
176 int partition(std::vector<int>& data,
177               int low,
178               int high,
179               Metrics& m) {
180
181     int pivot = data[high];
182     int i = low - 1;
183
184     for (int j = low; j < high; j++) {

```

```

183
184         m.comparisons++;
185
186         if (data[j] < pivot) {
187             i++;
188             std::swap(data[i], data[j]);
189             m.swaps++;
190         }
191     }
192
193     std::swap(data[i + 1], data[high]);
194     m.swaps++;
195
196     return i + 1;
197 }
198
199 void quick_sort_recursive(std::vector<int>& data,
200                          int low,
201                          int high,
202                          Metrics& m) {
203
204     m.recursive_calls++;
205
206     if (low < high) {
207
208         int pi = partition(data, low, high, m);
209
210         quick_sort_recursive(data, low, pi - 1,
211 m);
212         quick_sort_recursive(data, pi + 1, high,
211 m);
212     }

```

```

213 }
214
215 // QUICK SORT
216 void quick_sort(std::vector<int>& data, Metrics&
217 m) {
218     auto start = Clock::now();
219
220     if (!data.empty()) {
221         quick_sort_recursive(data, 0, data.size()
222 - 1, m);
223     }
224
225     m.time_ms =
226     std::chrono::duration<double, std::milli>(
227         Clock::now() - start).count();
228 }
229
230 // RADIX SORT HELPER
231 void counting_sort(std::vector<int>& data,
232 int exp,
233 Metrics& m) {
234
235     int n = data.size();
236
237     std::vector<int> output(n);
238     int count[10] = {0};
239
240     for (int i = 0; i < n; i++) {
241         count[(data[i] / exp) % 10]++;
242         m.array_accesses++;
243     }

```

```

243     for (int i = 1; i < 10; i++) {
244         count[i] += count[i - 1];
245     }
246
247     for (int i = n - 1; i >= 0; i--) {
248
249         int index = (data[i] / exp) % 10;
250
251         output[count[index] - 1] = data[i];
252         count[index]--;
253
254         m.array_accesses++;
255     }
256
257     for (int i = 0; i < n; i++) {
258         data[i] = output[i];
259         m.array_accesses++;
260     }
261 }
262
263 // RADIX SORT
264 void radix_sort(std::vector<int>& data, Metrics&
265 m) {
266     if (data.empty()) return;
267
268     auto start = Clock::now();
269
270     int maxVal = *std::max_element(data.begin(),
271 data.end());
272
273     for (int exp = 1; maxVal / exp > 0; exp *= 10)
274 {

```

272	counting_sort(data, exp, m);
273	}
274	
275	m.time_ms =
276	std::chrono::duration<double, std::milli>(
277	Clock::now() - start).count();
278	}

Tabel 24 Source Code File sorting_algorithms.h

1	#pragma once
2	#include <vector>
3	#include "metrics.h"
4	
	void bubble_sort (std::vector<int>& data,
5	Metrics& m);
	void selection_sort (std::vector<int>& data,
6	Metrics& m);
	void insertion_sort (std::vector<int>& data,
7	Metrics& m);
	void merge_sort (std::vector<int>& data,
8	Metrics& m);
	void radix_sort (std::vector<int>& data,
9	Metrics& m);
	void quick_sort (std::vector<int>& data,
10	Metrics& m);

Tabel 25 Source Code File metric.h

1	#include "sorting_algorithms.h"
2	#include <algorithm>
3	#include <chrono>
4	
5	using Clock = std::chrono::high_resolution_clock;

6	
7	// BUBBLE SORT
	void bubble_sort(std::vector<int>& data, Metrics&
8	m) {
9	int n = data.size();
10	auto start = Clock::now();
11	
12	for (int i = 0; i < n - 1; i++) {
13	for (int j = 0; j < n - i - 1; j++) {
14	
15	m.comparisons++;
16	
17	if (data[j] > data[j + 1]) {
18	std::swap(data[j], data[j + 1]);
19	m.swaps++;
20	}
21	}
22	}
23	
24	m.time_ms =
25	std::chrono::duration<double, std::milli>(
26	Clock::now() - start).count();

Tabel 26 Source Code File report.h

1	#pragma once
2	#include <vector>
3	#include "metrics.h"
4	
5	void printMetrics(const Metrics& m);
6	
7	void printTableHeader();

Tabel 27 Source Code File reporter.cpp

1	#include "reporter.h"
2	#include <iostream>
3	#include <iomanip>
4	
5	static const int W_ALG = 16;
6	static const int W_INPUT = 16;
7	static const int W_N = 8;
8	static const int W_STAT = 14;
9	static const int W_TIME = 16;
10	
11	void printTableHeader() {
12	std::cout << std::left
13	<< std::setw(W_ALG) << "Algorithm"
14	<< std::setw(W_INPUT) << "Input
15	Type"
16	<< std::setw(W_N) << "N"
17	<< std::setw(W_STAT) <<
18	"Comparisons"
19	<< std::setw(W_STAT) << "Swaps"
20	<< std::setw(W_STAT) << "Shifts"
21	<< std::setw(W_STAT) << "Rec.Calls"
22	<< std::setw(W_STAT) <<
23	"Arr.Access"
24	<< std::setw(W_TIME) << "Time(ms)"
25	<< "\n";
26	std::cout << std::string(W_ALG + W_INPUT + W_N
27	+ W_STAT*5 + W_TIME, '-') << "\n";
28	}
29	
30	void printMetrics(const Metrics& m) {
31	std::cout << std::left

	<< std::setw(W_ALG) <<
28	m.algorithm_name
	<< std::setw(W_INPUT) <<
29	m.input_type
30	<< std::setw(W_N) << m.n
	<< std::setw(W_STAT) <<
31	m.comparisons
32	<< std::setw(W_STAT) << m.swaps
33	<< std::setw(W_STAT) << m.shifts
	<< std::setw(W_STAT) <<
34	m.recursive_calls
	<< std::setw(W_STAT) <<
35	m.array_accesses
	<< std::fixed <<
36	std::setprecision(6)
37	<< std::setw(W_TIME) << m.time_ms
38	<< "\n";
39	}

Tabel 28 Source Code File main.cpp

1	#include <iostream>
2	#include <vector>
3	#include <functional>
4	#include <string>
5	
6	#include "metrics.h"
7	#include "data_generator.h"
8	#include "sorting_algorithms.h"
9	#include "reporter.h"
10	
11	static const std::vector<int> N_VALUES = { 100,
	1'000, 10'000 };

```

12 static const std::vector<InputType> INPUT_TYPES =
13 {
14     InputType::RANDOM,
15     InputType::SORTED,
16     InputType::REVERSE_SORTED,
17     InputType::NEARLY_SORTED,
18     InputType::DUPLICATES,
19 };
20
21 static const unsigned int SEED = 42;
22 using SortFn =
23     std::function<void(std::vector<int>&, Metrics&)>;
24
25 static const std::vector<std::pair<std::string,
26 SortFn>> ALGORITHMS = {
27     { "Bubble Sort", bubble_sort },
28     { "Selection Sort", selection_sort },
29     { "Insertion Sort", insertion_sort },
30     { "Merge Sort", merge_sort },
31     { "Quick Sort", quick_sort },
32     { "Radix Sort", radix_sort },
33 };
34
35 int main() {
36     std::cout <<
37     "=====
38     =\n";
39     std::cout << "          SORTING ALGORITHM -
40     ANALYSIS\n";

```

	std::cout	<<
	"=====	
37	=\n";	
38		
	for (const auto& [name, sortFn] : ALGORITHMS)	
39	{s	
40	std::cout << "\n>> " << name << "\n";	
41	printTableHeader();	
42		
43	for (int n : N_VALUES) {	
	for (const auto& inputType :	
44	INPUT_TYPES) {	
	std::vector<int> data =	
45	generateData(n, inputType, SEED);	
46		
47	Metrics m;	
48	m.algorithm_name = name;	
	m.input_type =	
49	inputTypeName(inputType);	
50	m.n = n;	
51		
52	sortFn(data, m);	
53	printMetrics(m);	
54	}	
55	}	
56	}	
57		
58	std::cout << "\nDone.\n";	
59	return 0;	
60	}	

Tabel 29 Source Code File data_generator.h

1	#pragma once
2	#include <vector>
3	#include <string>
4	
5	enum class InputType {
6	RANDOM,
7	SORTED,
8	REVERSE_SORTED,
9	NEARLY_SORTED,
10	DUPLICATES
11	};
12	
13	std::string inputTypeName(InputType type);
14	
15	std::vector<int> generateData(int n, InputType type, unsigned int seed = 42);

Tabel 30 Source Code File generator.cpp

1	#include "data_generator.h"
2	#include <algorithm>
3	#include <numeric>
4	#include <random>
5	#include <stdexcept>
6	
7	std::string inputTypeName(InputType type) {
8	switch (type) {
	case InputType::RANDOM: return
9	"Random";
	case InputType::SORTED: return
10	"Sorted";

	<pre> case InputType::REVERSE_SORTED: return 11 "Reverse Sorted"; case InputType::NEARLY_SORTED: return 12 "Nearly Sorted"; case InputType::DUPLICATES: return 13 "Many Duplicates"; default: return 14 "Unknown"; 15 } 16 } 17 std::vector<int> generateData(int n, InputType 18 type, unsigned int seed) { if (n <= 0) throw std::invalid_argument("N 19 harus > 0"); 20 21 std::mt19937 rng(seed); 22 std::vector<int> data(n); 23 24 switch (type) { 25 26 case InputType::RANDOM: { 27 std::uniform_int_distribution<int> 28 dist(1, n * 10); 29 for (auto& x : data) x = dist(rng); 30 break; 31 } 32 33 case InputType::SORTED: { 34 std::iota(data.begin(), data.end(), 35 1); 36 break; </pre>
--	---

```

35         }
36
37         case InputType::REVERSE_SORTED: {
38             std::iota(data.begin(), data.end(),
39 1);
40             std::reverse(data.begin(),
41 data.end());
42             break;
43         }
44
45         case InputType::NEARLY_SORTED: {
46             std::iota(data.begin(), data.end(),
47 1);
48             int swaps = std::max(1, n / 20);
49             std::uniform_int_distribution<int>
50 idx(0, n - 1);
51             for (int i = 0; i < swaps; i++) {
52                 std::swap(data[idx(rng)],
53 data[idx(rng)]);
54             }
55             break;
56         }
57
58         case InputType::DUPLICATES: {
59             int range = std::max(2, n / 10);
60             std::uniform_int_distribution<int>
61 dist(1, range);
62             for (auto& x : data) x = dist(rng);
63             break;
64         }
65     }

```


61	return data;
62	}

B. Output Program

>> Bubble Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Bubble Sort	Random	100	4950	2564	0	0	0	0.025500
Bubble Sort	Sorted	100	4950	0	0	0	0	0.003100
Bubble Sort	Reverse Sorted	100	4950	4950	0	0	0	0.037600
Bubble Sort	Nearly Sorted	100	4950	296	0	0	0	0.005100
Bubble Sort	Many Duplicates	100	4950	2315	0	0	0	0.015100
Bubble Sort	Random	1000	499500	243190	0	0	0	1.408100
Bubble Sort	Sorted	1000	499500	0	0	0	0	0.441800
Bubble Sort	Reverse Sorted	1000	499500	499500	0	0	0	2.724500
Bubble Sort	Nearly Sorted	1000	499500	25778	0	0	0	0.586900
Bubble Sort	Many Duplicates	1000	499500	240808	0	0	0	1.424000
Bubble Sort	Random	10000	49995000	25118148	0	0	0	149.790900
Bubble Sort	Sorted	10000	49995000	0	0	0	0	33.248400
Bubble Sort	Reverse Sorted	10000	49995000	49995000	0	0	0	266.830100
Bubble Sort	Nearly Sorted	10000	49995000	3119352	0	0	0	43.251300
Bubble Sort	Many Duplicates	10000	49995000	25093086	0	0	0	150.826500

Gambar 9 Hasil Pengujian Bubble Sort

>> Selection Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Selection Sort	Random	100	4950	95	0	0	0	0.007700
Selection Sort	Sorted	100	4950	0	0	0	0	0.006100
Selection Sort	Reverse Sorted	100	4950	50	0	0	0	0.009900
Selection Sort	Nearly Sorted	100	4950	4	0	0	0	0.006300
Selection Sort	Many Duplicates	100	4950	89	0	0	0	0.007700
Selection Sort	Random	1000	499500	995	0	0	0	0.782600
Selection Sort	Sorted	1000	499500	0	0	0	0	1.105500
Selection Sort	Reverse Sorted	1000	499500	500	0	0	0	0.781900
Selection Sort	Nearly Sorted	1000	499500	50	0	0	0	0.776500
Selection Sort	Many Duplicates	1000	499500	983	0	0	0	0.949100
Selection Sort	Random	10000	49995000	9992	0	0	0	85.182500
Selection Sort	Sorted	10000	49995000	0	0	0	0	84.525900
Selection Sort	Reverse Sorted	10000	49995000	5000	0	0	0	85.146000
Selection Sort	Nearly Sorted	10000	49995000	500	0	0	0	84.712600
Selection Sort	Many Duplicates	10000	49995000	9981	0	0	0	84.728800

Gambar 10 Hasil Pengujian Selection Sort

>> Insertion Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Insertion Sort	Random	100	2656	0	2564	0	0	0.002900
Insertion Sort	Sorted	100	99	0	0	0	0	0.000200
Insertion Sort	Reverse Sorted	100	4950	0	4950	0	0	0.002600
Insertion Sort	Nearly Sorted	100	395	0	296	0	0	0.000500
Insertion Sort	Many Duplicates	100	2412	0	2315	0	0	0.001800
Insertion Sort	Random	1000	244181	0	243190	0	0	0.106400
Insertion Sort	Sorted	1000	999	0	0	0	0	0.001300
Insertion Sort	Reverse Sorted	1000	499500	0	499500	0	0	0.228600
Insertion Sort	Nearly Sorted	1000	26775	0	25778	0	0	0.020600
Insertion Sort	Many Duplicates	1000	241801	0	240808	0	0	0.116000
Insertion Sort	Random	10000	25128135	0	25118148	0	0	10.711900
Insertion Sort	Sorted	10000	9999	0	0	0	0	0.012500
Insertion Sort	Reverse Sorted	10000	49995000	0	49995000	0	0	24.324700
Insertion Sort	Nearly Sorted	10000	3129351	0	3119352	0	0	1.583400
Insertion Sort	Many Duplicates	10000	25103078	0	25093086	0	0	12.004100

Gambar 11 Hasil Pengujian Insertion Sort

>> Merge Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Merge Sort	Random	100	535	0	0	199	0	0.013100
Merge Sort	Sorted	100	356	0	0	199	0	0.008700
Merge Sort	Reverse Sorted	100	316	0	0	199	0	0.008500
Merge Sort	Nearly Sorted	100	453	0	0	199	0	0.008400
Merge Sort	Many Duplicates	100	536	0	0	199	0	0.012800
Merge Sort	Random	1000	8690	0	0	1999	0	0.099400
Merge Sort	Sorted	1000	5044	0	0	1999	0	0.062800
Merge Sort	Reverse Sorted	1000	4932	0	0	1999	0	0.062300
Merge Sort	Nearly Sorted	1000	7628	0	0	1999	0	0.085700
Merge Sort	Many Duplicates	1000	8679	0	0	1999	0	0.093900
Merge Sort	Random	10000	120465	0	0	19999	0	1.015000
Merge Sort	Sorted	10000	69008	0	0	19999	0	0.769000
Merge Sort	Reverse Sorted	10000	64608	0	0	19999	0	0.943900
Merge Sort	Nearly Sorted	10000	111524	0	0	19999	0	0.712700
Merge Sort	Many Duplicates	10000	120461	0	0	19999	0	1.364500

Gambar 12 Hasil Pengujian Merge Sort

>> Quick Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Quick Sort	Random	100	678	394	0	129	0	0.002300
Quick Sort	Sorted	100	4950	5049	0	199	0	0.003500
Quick Sort	Reverse Sorted	100	4950	2549	0	199	0	0.003500
Quick Sort	Nearly Sorted	100	2638	2607	0	191	0	0.003900
Quick Sort	Many Duplicates	100	909	236	0	181	0	0.002000
Quick Sort	Random	1000	10537	6125	0	1341	0	0.027700
Quick Sort	Sorted	1000	499500	500499	0	1999	0	0.349300
Quick Sort	Reverse Sorted	1000	499500	250499	0	1999	0	0.282900
Quick Sort	Nearly Sorted	1000	35189	25081	0	1611	0	0.041900
Quick Sort	Many Duplicates	1000	12685	4920	0	1801	0	0.036600
Quick Sort	Random	10000	147606	77996	0	13315	0	0.382400
Quick Sort	Sorted	10000	49995000	50004999	0	19999	0	32.773400
Quick Sort	Reverse Sorted	10000	49995000	25004999	0	19999	0	27.840300
Quick Sort	Nearly Sorted	10000	622473	458547	0	16511	0	0.693600
Quick Sort	Many Duplicates	10000	181016	67083	0	18003	0	0.302600

Gambar 13 Hasil Pengujian Quick Sort

>> Radix Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Radix Sort	Random	100	0	0	0	0	900	0.002700
Radix Sort	Sorted	100	0	0	0	0	900	0.001800
Radix Sort	Reverse Sorted	100	0	0	0	0	900	0.002200
Radix Sort	Nearly Sorted	100	0	0	0	0	900	0.001800
Radix Sort	Many Duplicates	100	0	0	0	0	600	0.001400
Radix Sort	Random	1000	0	0	0	0	12000	0.015000
Radix Sort	Sorted	1000	0	0	0	0	12000	0.013200
Radix Sort	Reverse Sorted	1000	0	0	0	0	12000	0.016200
Radix Sort	Nearly Sorted	1000	0	0	0	0	12000	0.013100
Radix Sort	Many Duplicates	1000	0	0	0	0	9000	0.015300
Radix Sort	Random	10000	0	0	0	0	150000	0.177400
Radix Sort	Sorted	10000	0	0	0	0	150000	0.151800
Radix Sort	Reverse Sorted	10000	0	0	0	0	150000	0.151500
Radix Sort	Nearly Sorted	10000	0	0	0	0	150000	0.158400
Radix Sort	Many Duplicates	10000	0	0	0	0	120000	0.125600

Gambar 14 Hasil Pengujian Radix Sort

C. Pembahasan

Algoritma **Bubble Sort** bekerja dengan cara membandingkan dua data yang berdekatan kemudian menukarnya apabila urutannya salah. Proses ini dilakukan berulang kali sampai seluruh data tersusun dengan benar. Pada setiap putaran, nilai terbesar akan bergerak ke bagian akhir sehingga prosesnya mirip seperti gelembung yang naik ke permukaan.

Pada kode **sorting_algorithms.cpp**, proses Bubble Sort dimulai pada baris [8] melalui fungsi `bubble_sort`. Jumlah data disimpan ke variabel `n` pada baris [9], kemudian waktu mulai dihitung pada baris [10]. Perulangan utama pada baris [12] digunakan untuk menentukan jumlah putaran sorting, sedangkan perulangan di baris [13] digunakan untuk membandingkan elemen yang berdekatan. Setiap kali terjadi pengecekan, nilai `comparisons` ditambah pada baris [15]. Kondisi utama Bubble Sort terdapat pada baris [17], yaitu ketika elemen kiri lebih besar daripada elemen kanan maka kedua data ditukar menggunakan `std::swap` pada baris [18]. Setelah pertukaran terjadi, jumlah swap dicatat pada baris [19]. Setelah seluruh proses selesai, waktu eksekusi disimpan pada baris [23–24].

Selection Sort bekerja dengan cara mencari nilai terkecil dari bagian data yang belum terurut, kemudian menempatkannya ke posisi yang benar. Algoritma ini terus mengulang proses pencarian nilai minimum hingga seluruh data tersusun rapi.

Pada kode **sorting_algorithms.cpp**, fungsi `selection_sort` dimulai pada baris [28]. Variabel `minIndex` pada baris [35] digunakan untuk menyimpan posisi nilai terkecil sementara. Perulangan di baris [37] digunakan untuk mencari nilai paling kecil pada bagian data yang belum terurut. Setiap perbandingan dicatat pada baris [39]. Jika ditemukan nilai yang lebih kecil, posisi minimum diperbarui pada baris [41–42]. Setelah pencarian selesai, data ditukar pada baris [46] apabila posisi minimum berubah. Jumlah pertukaran kemudian dicatat pada baris [47]. Waktu eksekusi dihitung pada bagian akhir fungsi di baris [51–52].

Insertion Sort bekerja dengan cara mengambil satu data lalu menyisipkannya ke posisi yang sesuai pada bagian data yang sudah terurut. Cara kerjanya mirip seperti menyusun kartu satu per satu ke posisi yang benar.

Pada kode, **sorting_algorithms.cpp** fungsi `insertion_sort` dimulai pada baris [56]. Perulangan utama pada baris [60] mengambil data satu per satu mulai dari indeks kedua. Nilai yang akan dipindahkan disimpan ke variabel `key` pada baris [62]. Selanjutnya perulangan `while` pada baris [65] digunakan untuk menggeser data yang lebih besar dari `key`. Setiap pengecekan dicatat pada baris [67]. Jika data sebelumnya lebih besar, maka data digeser ke kanan pada baris [70], jumlah `shift` ditambah pada baris [71], lalu indeks mundur pada baris [72]. Setelah posisi yang tepat ditemukan, nilai `key` dimasukkan kembali pada baris [79]. Waktu eksekusi disimpan pada baris [83–84].

Merge Sort bekerja dengan cara membagi data menjadi bagian-bagian kecil, kemudian menggabungkannya kembali dalam keadaan terurut. Algoritma ini menggunakan konsep pemecahan data secara bertahap lalu penyatuan kembali.

Pada kode **sorting_algorithms.cpp**, proses penggabungan dilakukan dalam fungsi `merge` yang dimulai pada baris [88]. Data dibagi menjadi dua bagian menggunakan array sementara `L` dan `R` pada baris [95–96]. Perbandingan antar kedua bagian dilakukan pada baris [111], lalu elemen yang lebih kecil dimasukkan kembali ke array utama pada baris [113–120]. Fungsi rekursif utama berada pada `merge_sort_recursive` di baris [140]. Jumlah pemanggilan rekursif dicatat pada baris [145]. Data dibagi menjadi dua bagian menggunakan nilai tengah pada baris [150], lalu fungsi memanggil dirinya sendiri pada baris [152–153]. Setelah kedua bagian selesai diproses, data digabung kembali melalui fungsi `merge` pada baris [155]. Fungsi utama `merge_sort` dimulai pada baris [159] dan menjalankan proses rekursif pada baris [163].

Quick Sort bekerja dengan memilih satu nilai sebagai titik acuan atau `pivot`, kemudian memisahkan data yang lebih kecil dan lebih besar dari `pivot`. Setelah itu proses dilakukan kembali pada bagian kiri dan kanan sampai seluruh data terurut.

Pada kode **sorting_algorithms.cpp**, proses pembagian data dilakukan pada fungsi `partition` di baris [173]. Pivot dipilih dari elemen terakhir pada baris [177]. Perulangan pada baris [180] digunakan untuk membandingkan setiap data dengan pivot. Jika data lebih kecil dari pivot, maka posisi indeks dinaikkan pada baris [185] dan data ditukar pada baris [186]. Setelah seluruh data diperiksa, pivot dipindahkan ke posisi yang benar pada baris [191]. Fungsi rekursif `quick_sort_recursive` dimulai pada baris [198]. Jumlah pemanggilan rekursif dicatat pada baris [203]. Setelah posisi pivot ditemukan, bagian kiri dan kanan diproses kembali secara terpisah pada baris [209–210]. Fungsi utama `quick_sort` dimulai pada baris [216] dan menjalankan rekursi pada baris [220].

Radix Sort bekerja dengan mengurutkan data berdasarkan digit angka, dimulai dari digit paling belakang menuju digit paling depan. Algoritma ini tidak membandingkan angka secara langsung seperti algoritma sorting lainnya.

Pada kode **sorting_algorithms.cpp**, proses pengurutan per digit dilakukan pada fungsi `counting_sort` di baris [230]. Array `count` pada baris [236] digunakan untuk menghitung jumlah kemunculan digit tertentu. Perulangan pada baris [238] membaca digit dari setiap angka lalu mencatat jumlah akses array pada baris [240]. Setelah itu dilakukan perhitungan posisi digit pada baris [243–245]. Data kemudian dipindahkan ke array sementara `output` pada baris [251], lalu jumlah akses array ditambah pada baris [254]. Setelah proses selesai, data dikembalikan ke array utama pada baris [258–261]. Fungsi utama `radix_sort` dimulai pada baris [266]. Nilai terbesar dicari pada baris [271] untuk menentukan jumlah digit maksimum. Perulangan pada baris [273] menjalankan `counting_sort` untuk setiap digit sampai seluruh data terurut.

Pada file header **algoritma_sorting.h** terdapat deklarasi beberapa algoritma sorting yang nantinya akan digunakan pada program. File ini berfungsi sebagai penghubung agar fungsi sorting dapat dipanggil dari file lain tanpa perlu menuliskan ulang isi fungsi. Pada baris [1] digunakan `#pragma once` yang berfungsi agar file header hanya dibaca satu kali oleh compiler sehingga tidak terjadi duplikasi deklarasi. Kemudian pada baris [2] program memanggil library vector karena seluruh algoritma menggunakan `std::vector<int>` sebagai tempat penyimpanan data angka yang akan diurutkan. Pada baris [3] dipanggil file `metrics.h` yang digunakan untuk mencatat informasi pengujian seperti jumlah comparison, swap, shift, recursion call, array access, dan waktu eksekusi.

Algoritma pertama adalah Bubble Sort yang dideklarasikan pada baris [5]. Algoritma ini bekerja dengan cara membandingkan dua elemen yang berdekatan lalu menukarnya jika urutannya salah. Proses tersebut dilakukan berulang dari awal hingga akhir data sampai seluruh elemen tersusun dengan benar. Semakin besar data yang belum terurut, semakin banyak proses pertukaran yang dilakukan.

Selection Sort pada baris [6] bekerja dengan cara mencari nilai terkecil dari bagian data yang belum terurut, kemudian menempatkannya ke posisi yang benar. Proses ini dilakukan sedikit demi sedikit dari kiri ke kanan hingga seluruh data tersusun rapi. Algoritma ini memiliki jumlah swap yang relatif sedikit karena pertukaran hanya dilakukan setelah nilai minimum ditemukan.

Insertion Sort pada baris [7] bekerja seperti proses menyusun kartu secara manual. Algoritma mengambil satu elemen lalu menyisipkannya ke posisi yang benar pada bagian data yang sudah terurut. Jika data sudah hampir rapi, proses sorting dapat berjalan sangat cepat karena hanya membutuhkan sedikit perpindahan data.

Merge Sort pada baris [8] menggunakan metode membagi data menjadi bagian-bagian kecil terlebih dahulu, kemudian menggabungkannya kembali dalam keadaan terurut. Proses pembagian dilakukan terus menerus sampai data menjadi sangat kecil, lalu setiap bagian digabung kembali secara bertahap hingga seluruh data tersusun rapi. Algoritma ini dikenal stabil dan cukup cepat untuk data berukuran besar.

Radix Sort pada baris [9] bekerja dengan cara mengurutkan angka berdasarkan digitnya mulai dari digit paling belakang menuju digit paling depan. Algoritma ini tidak membandingkan angka secara langsung, melainkan mengelompokkan data berdasarkan nilai digit pada setiap tahap. Karena itu Radix Sort dapat bekerja sangat cepat pada data angka dengan jumlah digit yang tidak terlalu besar.

Quick Sort pada baris [10] bekerja dengan memilih satu elemen sebagai pivot atau titik acuan. Data kemudian dibagi menjadi dua bagian, yaitu elemen yang lebih kecil dari pivot dan elemen yang lebih besar dari pivot. Setelah itu proses yang sama dilakukan kembali pada masing-masing bagian sampai seluruh data terurut. Quick Sort terkenal sangat cepat pada kondisi normal, tetapi performanya dapat menurun jika pemilihan pivot kurang tepat.

Pada file header **metrics.h** dibuat sebuah struct bernama Metrics pada baris [5] yang berfungsi untuk menyimpan seluruh hasil pengukuran performa algoritma sorting. Struct ini digunakan agar setiap algoritma dapat mencatat informasi pengujian seperti jumlah comparison, swap, waktu eksekusi, dan operasi lainnya dalam satu tempat yang terorganisir.

Pada baris [1] digunakan `#pragma once` agar file header hanya diproses satu kali oleh compiler sehingga tidak terjadi duplikasi. Kemudian pada baris [2] program memanggil library string karena beberapa data seperti nama algoritma dan jenis input menggunakan tipe data string. Pada baris [3] digunakan library chrono yang nantinya dipakai untuk menghitung waktu eksekusi algoritma sorting.

Bagian utama dimulai pada baris [5] dengan pembuatan struct Metrics. Struct ini berfungsi seperti wadah yang menyimpan seluruh informasi hasil pengujian algoritma. Pada baris [6] dan [7], terdapat variabel `algorithm_name` dan `input_type` yang digunakan untuk menyimpan nama algoritma serta jenis data yang diuji, misalnya random, sorted, atau reverse sorted. Kemudian pada baris [8], variabel `n` digunakan untuk menyimpan jumlah data yang diproses.

Pada baris [10], variabel `comparisons` digunakan untuk menghitung berapa kali algoritma melakukan perbandingan antar elemen data. Nilai ini penting karena semakin banyak comparison biasanya waktu proses juga semakin besar. Pada baris [11], `swaps` digunakan untuk menghitung jumlah pertukaran posisi data selama proses sorting berlangsung. Selanjutnya pada baris [12], `shifts` digunakan khusus untuk algoritma seperti Insertion Sort yang memindahkan elemen dengan cara menggeser data, bukan menukar langsung.

Baris [13] berisi `recursive_calls` yang digunakan untuk mencatat jumlah pemanggilan fungsi rekursif pada algoritma seperti Merge Sort dan Quick Sort. Semakin besar jumlah recursive call, biasanya proses pembagian data juga semakin banyak. Kemudian pada baris [14], `array_accesses` digunakan untuk menghitung berapa kali algoritma mengakses array, khususnya pada algoritma seperti Counting Sort dan Radix Sort yang banyak melakukan pembacaan dan penulisan ulang data.

Pada baris [16], variabel `time_ms` digunakan untuk menyimpan waktu eksekusi algoritma dalam satuan milidetik. Variabel ini sangat penting untuk

membandingkan kecepatan masing-masing algoritma sorting pada berbagai kondisi data.

Bagian terakhir terdapat fungsi `reset()` pada baris [18] hingga [25]. Fungsi ini digunakan untuk mengembalikan seluruh nilai pengukuran menjadi 0 sebelum pengujian baru dimulai. Dengan adanya fungsi reset, setiap algoritma dapat diuji kembali tanpa membawa hasil pengukuran sebelumnya sehingga data pengujian tetap akurat dan rapi.

Pada file header **report.h** terdapat deklarasi fungsi yang digunakan untuk menampilkan hasil pengujian algoritma sorting ke layar. File ini berfungsi sebagai penghubung agar fungsi tampilan dapat dipanggil dari file lain tanpa perlu menuliskan ulang isi fungsi. Pada baris [1] digunakan `#pragma once` untuk memastikan file header hanya dibaca satu kali oleh compiler sehingga tidak terjadi duplikasi deklarasi fungsi.

Pada baris [2], program memanggil library vector. Walaupun pada potongan kode ini vector belum digunakan secara langsung, library tersebut biasanya disiapkan untuk kebutuhan pengolahan data dalam file implementasi terkait. Kemudian pada baris [3], file `metrics.h` dipanggil karena fungsi yang ada pada header ini menggunakan struct `Metrics` sebagai parameter utama untuk menampilkan hasil pengukuran algoritma.

Fungsi pertama dideklarasikan pada baris [5] yaitu `printMetrics(const Metrics& m);`. Fungsi ini digunakan untuk menampilkan seluruh hasil pengujian dari suatu algoritma sorting. Parameter `const Metrics& m` berarti fungsi menerima data `metrics` tanpa menyalin ulang isi struct sehingga penggunaan memori menjadi lebih efisien. Data yang biasanya ditampilkan melalui fungsi ini meliputi jumlah `comparison`, `swap`, `shift`, `recursion call`, `array access`, dan waktu eksekusi algoritma.

Kemudian pada baris [7] terdapat fungsi `printTableHeader();`. Fungsi ini digunakan untuk menampilkan bagian judul tabel sebelum hasil pengujian dicetak. Dengan adanya fungsi ini, tampilan output menjadi lebih rapi dan mudah dibaca karena setiap kolom seperti nama algoritma, jenis input, jumlah `comparison`, dan waktu eksekusi memiliki header yang jelas.

Secara keseluruhan, file header ini berfungsi untuk mengatur bagian tampilan hasil pengujian algoritma sorting. Pemisahan deklarasi fungsi seperti ini membuat struktur program menjadi lebih rapi, mudah dipahami, dan memudahkan pengelolaan kode ketika program semakin besar.

Pada file `reporter.cpp` ini terdapat implementasi fungsi yang digunakan untuk menampilkan hasil pengujian algoritma sorting dalam bentuk tabel yang rapi di terminal atau console. File ini bekerja sebagai bagian tampilan output sehingga hasil analisis seperti jumlah `comparison`, `swap`, dan waktu eksekusi dapat dibaca dengan lebih mudah.

Pada baris [1], file `reporter.h` dipanggil karena file ini merupakan implementasi dari deklarasi fungsi yang sudah dibuat sebelumnya pada header. Kemudian pada baris [2] digunakan library `iostream` untuk menampilkan teks ke layar menggunakan `std::cout`. Pada baris [3], program memanggil library `iomanip` yang berfungsi untuk mengatur tampilan output seperti lebar kolom dan jumlah angka desimal.

Baris [5] hingga [9] berisi konstanta ukuran kolom tabel. Misalnya `W_ALG` pada baris [5] digunakan untuk menentukan lebar kolom nama algoritma, `W_INPUT` pada baris [6] untuk jenis input data, dan `W_TIME` pada baris [9] untuk kolom waktu eksekusi. Dengan pengaturan ini, tampilan tabel menjadi lebih sejajar dan mudah dibaca walaupun isi datanya berbeda panjang.

Fungsi `printTableHeader()` dimulai pada baris [11]. Fungsi ini bertugas menampilkan bagian judul tabel sebelum hasil pengujian dicetak. Pada baris [12], digunakan `std::left` agar seluruh isi tabel rata kiri sehingga tampil lebih rapi. Selanjutnya pada baris [13] hingga [20], program menggunakan `std::setw()` untuk mengatur lebar setiap kolom sebelum mencetak judul seperti `Algorithm`, `Input Type`, `Comparisons`, `Swaps`, hingga `Time(ms)`. `std::setw(W_ALG)` pada baris [13] mengatur lebar kolom nama algoritma, `std::setw(W_STAT)` pada baris [16]-[19] mengatur lebar kolom statistik seperti `comparison` dan `swap`, sedangkan `std::setw(W_TIME)` pada baris [20] digunakan untuk kolom waktu eksekusi.

Pada baris [22], program mencetak garis pemisah menggunakan `std::string()` yang diisi karakter '-'. Garis ini berfungsi agar tampilan tabel terlihat lebih jelas dan terstruktur.

Fungsi berikutnya yaitu `printMetrics(const Metrics& m)` dimulai pada baris [25]. Fungsi ini digunakan untuk mencetak isi data hasil pengujian dari struct `Metrics`. Parameter `const Metrics& m` berarti data metrics dikirim tanpa disalin ulang sehingga lebih efisien.

Pada baris [26] hingga [35], setiap isi variabel dalam struct `Metrics` ditampilkan ke tabel menggunakan `std::setw()` agar posisinya tetap sejajar dengan header tabel. `m.algorithm_name` pada baris [27] menampilkan nama algoritma, `m.input_type` pada baris [28] menampilkan jenis input data, `m.comparisons` pada baris [30] menampilkan jumlah comparison, `m.swaps` pada baris [31] menampilkan

jumlah swap, dan `m.time_ms` pada baris [36] menampilkan waktu eksekusi algoritma.

Pada baris [35], digunakan `std::fixed` dan `std::setprecision(6)` untuk mengatur jumlah angka di belakang koma menjadi 6 digit. Hal ini membuat hasil waktu eksekusi tampil lebih presisi dan konsisten.

Pada file utama **main.cpp** program ini terdapat proses utama untuk menjalankan seluruh pengujian algoritma sorting secara otomatis. Program akan mencoba berbagai algoritma sorting menggunakan beberapa jenis data dan ukuran data, lalu hasil pengujiannya ditampilkan dalam bentuk tabel. Dengan struktur seperti ini, proses analisis performa algoritma menjadi lebih mudah dilakukan karena seluruh pengujian berjalan secara teratur dan konsisten.

Pada baris [1] hingga [4], program memanggil beberapa library bawaan C++ seperti `iostream`, `vector`, `functional`, dan `string`. Library tersebut digunakan untuk menampilkan output, menyimpan data dalam `vector`, menggunakan fungsi sebagai parameter, serta mengelola teks. Kemudian pada baris [6] hingga [9], program memanggil file header yang sebelumnya sudah dibuat, seperti `metrics.h`, `data_generator.h`, `sorting_algorithms.h`, dan `reporter.h`. File-file ini berisi fungsi dan struktur penting untuk proses pengujian algoritma sorting.

Baris [11] mendefinisikan `N_VALUES` yang berisi ukuran data yang akan diuji, yaitu 100, 1000, dan 10000. Dengan variasi ukuran data ini, program dapat membandingkan bagaimana performa algoritma berubah ketika jumlah data semakin besar.

Pada baris [13] hingga [19], dibuat daftar `INPUT_TYPES` yang berisi beberapa kondisi data untuk pengujian, yaitu:

- `RANDOM`
- `SORTED`
- `REVERSE_SORTED`
- `NEARLY_SORTED`
- `DUPLICATES`

Jenis data ini digunakan untuk melihat pengaruh bentuk data terhadap performa algoritma sorting.

Baris [21] mendefinisikan `SEED = 42` yang digunakan agar data random yang dihasilkan selalu konsisten pada setiap pengujian. Hal ini penting supaya hasil eksperimen lebih adil dan dapat dibandingkan secara akurat.

Pada baris [22], dibuat alias `SortFn` menggunakan `std::function`. Alias ini digunakan untuk menyimpan fungsi sorting dengan format parameter yang sama, yaitu menerima `vector<int>` dan `Metrics`.

Bagian penting berikutnya terdapat pada baris [24] hingga [31], yaitu daftar ALGORITHMS. Variabel ini menyimpan pasangan nama algoritma dan fungsi sorting yang akan dipanggil. Misalnya: "Bubble Sort" dipasangkan dengan fungsi `bubble_sort`, "Merge Sort" dipasangkan dengan `merge_sort`, "Radix Sort" dipasangkan dengan `radix_sort`.

Dengan cara ini, program dapat menjalankan seluruh algoritma menggunakan perulangan tanpa perlu menulis kode satu per satu.

Fungsi utama program dimulai pada baris [34] melalui `main()`. Pada baris [35] hingga [37], program menampilkan judul aplikasi ke layar agar output terlihat lebih jelas dan rapi.

Selanjutnya pada baris [39], program mulai melakukan perulangan untuk setiap algoritma sorting yang ada pada daftar ALGORITHMS. Variabel `name` menyimpan nama algoritma, sedangkan `sortFn` menyimpan fungsi sorting yang akan dijalankan. Pada baris [40], nama algoritma ditampilkan ke layar, lalu pada baris [41] fungsi `printTableHeader()` dipanggil untuk menampilkan header tabel hasil pengujian.

Baris [43] hingga [44] melakukan perulangan untuk setiap ukuran data dan setiap jenis input data. Artinya setiap algoritma akan diuji menggunakan seluruh kombinasi ukuran data dan kondisi data yang sudah ditentukan sebelumnya.

Pada baris [45], program membuat data pengujian menggunakan fungsi `generateData()`. Fungsi ini menghasilkan data sesuai ukuran dan jenis input yang dipilih, misalnya `random` atau `reverse sorted`.

Kemudian pada baris [47] hingga [50], dibuat objek `Metrics m` untuk menyimpan hasil pengukuran algoritma. Data seperti nama algoritma, jenis input, dan jumlah data disimpan ke dalam struct tersebut agar nantinya dapat ditampilkan pada tabel hasil pengujian.

Pada baris [52], fungsi sorting dijalankan melalui `sortFn(data, m);`. Di sinilah algoritma sorting benar-benar bekerja untuk mengurutkan data sekaligus mencatat statistik performanya seperti `comparison`, `swap`, dan waktu eksekusi.

Setelah proses sorting selesai, hasil pengukuran ditampilkan menggunakan `printMetrics(m);` pada baris [53]. Fungsi ini akan mencetak seluruh hasil pengujian dalam bentuk tabel yang rapi.

Pada baris [58], program menampilkan tulisan "Done." sebagai tanda bahwa seluruh proses pengujian telah selesai dilakukan, kemudian program berhenti pada baris [59] dengan return 0;

Pada file header **data_generator.h** berfungsi untuk mengatur proses pembuatan data pengujian pada algoritma sorting. File ini digunakan agar program dapat menghasilkan berbagai bentuk data seperti random, sorted, reverse sorted, nearly sorted, dan duplicates secara otomatis untuk keperluan analisis performa algoritma.

Pada baris [1], digunakan `#pragma once` untuk memastikan file header hanya dibaca satu kali oleh compiler sehingga tidak terjadi duplikasi deklarasi. Kemudian pada baris [2] dan [3], program memanggil library vector dan string. Library vector digunakan karena data pengujian disimpan dalam bentuk `std::vector<int>`, sedangkan string digunakan untuk mengubah tipe input menjadi teks yang dapat ditampilkan pada output program.

Bagian utama dimulai pada baris [5] dengan pembuatan enum class `InputType`. Enum ini digunakan untuk mendefinisikan beberapa jenis kondisi data yang akan dipakai dalam pengujian algoritma sorting. Dengan penggunaan enum, kode menjadi lebih rapi dan mudah dipahami karena setiap jenis data memiliki nama yang jelas.

Pada baris [6] terdapat `RANDOM` yang digunakan untuk menghasilkan data acak. Jenis data ini biasanya digunakan untuk melihat performa umum algoritma pada kondisi normal. Baris [7] berisi `SORTED` yang menghasilkan data sudah terurut menaik. Kondisi ini penting untuk menguji bagaimana algoritma bekerja ketika data sudah rapi.

Kemudian pada baris [8], terdapat `REVERSE_SORTED` yang menghasilkan data terurut menurun atau kebalikan dari urutan normal. Kondisi ini sering menjadi kasus terburuk bagi beberapa algoritma seperti Bubble Sort dan Insertion Sort karena banyak elemen harus dipindahkan.

Baris [9] berisi `NEARLY_SORTED` yang menghasilkan data hampir terurut. Biasanya hanya terdapat sedikit elemen yang salah posisi sehingga cocok untuk melihat performa algoritma pada data yang hampir rapi. Sedangkan pada baris [10], terdapat `DUPLICATES` yang menghasilkan banyak nilai kembar atau duplikat. Kondisi ini digunakan untuk menguji bagaimana algoritma menangani data dengan nilai yang sama dalam jumlah besar.

Pada baris [13], terdapat deklarasi fungsi `inputTypeName(InputType type);`. Fungsi ini digunakan untuk mengubah nilai enum seperti `RANDOM` atau `SORTED` menjadi teks biasa agar lebih mudah ditampilkan pada tabel hasil pengujian.

Selanjutnya pada baris [15], terdapat deklarasi fungsi `generateData()`. Fungsi ini berfungsi untuk membuat data pengujian sesuai ukuran dan jenis input yang dipilih. Parameter `n` digunakan untuk menentukan jumlah data, `type` menentukan bentuk data yang ingin dibuat, sedangkan `seed` digunakan agar data random yang dihasilkan tetap konsisten pada setiap pengujian. Dengan adanya fungsi ini, program dapat menghasilkan berbagai kondisi data secara otomatis tanpa perlu membuat data secara manual setiap kali pengujian dilakukan.

Pada file **data_generator.cpp** terdapat implementasi fungsi yang digunakan untuk membuat berbagai jenis data pengujian pada program sorting. File ini sangat penting karena seluruh eksperimen algoritma sorting bergantung pada data yang dihasilkan di sini. Dengan adanya file ini, program dapat membuat data random, sorted, reverse sorted, nearly sorted, dan duplicates secara otomatis sehingga pengujian menjadi lebih konsisten dan terstruktur.

Pada baris [1], file `data_generator.h` dipanggil karena file ini merupakan implementasi dari deklarasi fungsi yang sudah dibuat sebelumnya. Kemudian pada baris [2] hingga [5], program memanggil beberapa library tambahan seperti: `algorithm` untuk fungsi seperti `reverse()` dan `swap()`, `numeric` untuk fungsi `iota()`, `random` untuk menghasilkan angka acak, dan `stdexcept` untuk menangani error apabila input tidak valid.

Fungsi pertama dimulai pada baris [7] yaitu `inputTypeName(InputType type)`. Fungsi ini digunakan untuk mengubah nilai enum menjadi teks biasa agar lebih mudah ditampilkan pada tabel hasil pengujian. Pada baris [8] digunakan `switch` untuk memeriksa jenis input data yang diterima. Misalnya:

- `InputType::RANDOM` pada baris [9] dikembalikan menjadi teks "Random",
- `InputType::SORTED` pada baris [10] menjadi "Sorted",
- dan `InputType::DUPLICATES` pada baris [13] menjadi "Many Duplicates".

Jika jenis input tidak dikenali, maka pada baris [14] program mengembalikan teks "Unknown".

Fungsi utama pembuatan data dimulai pada baris [18] yaitu `generateData(int n, InputType type, unsigned int seed)`. Fungsi ini digunakan untuk membuat data sesuai ukuran dan jenis input yang dipilih. Parameter `n` menunjukkan jumlah data yang ingin dibuat, `type` menentukan bentuk data, dan `seed` digunakan agar data random yang dihasilkan tetap konsisten pada setiap pengujian.

Pada baris [19], program memeriksa apakah nilai `n` lebih kecil atau sama dengan 0. Jika iya, maka program akan menghasilkan error menggunakan `throw std::invalid_argument`. Hal ini dilakukan agar program tidak membuat data dengan ukuran yang tidak valid.

Baris [21] membuat objek `std::mt19937 rng(seed)` yang digunakan sebagai generator angka acak. Penggunaan seed membuat hasil data random tetap sama setiap kali program dijalankan sehingga hasil pengujian lebih adil dan mudah dibandingkan. Kemudian pada baris [22], dibuat vector data dengan ukuran n untuk menyimpan seluruh angka yang akan diuji.

Bagian utama berikutnya terdapat pada baris [24] yaitu `switch(type)`. Struktur ini digunakan untuk menentukan jenis data yang akan dibuat.

Pada baris [26] hingga [30], dibuat kondisi `RANDOM`. Program menggunakan `uniform_int_distribution` untuk menghasilkan angka acak dalam rentang 1 hingga $n * 10$. Kemudian pada baris [28], setiap elemen dalam vector diisi angka random menggunakan perulangan `for`.

Pada baris [32] hingga [35], dibuat kondisi `SORTED`. Program menggunakan fungsi `std::iota()` untuk mengisi vector dengan angka berurutan mulai dari 1. Hasilnya adalah data yang sudah terurut menaik.

Kemudian pada baris [37] hingga [41], dibuat kondisi `REVERSE_SORTED`. Program pertama-tama membuat data terurut menggunakan `iota()`, lalu pada baris [39] data dibalik menggunakan `std::reverse()`. Hasil akhirnya adalah data terurut menurun.

Pada baris [43] hingga [51], dibuat kondisi `NEARLY_SORTED`. Program awalnya membuat data terurut menggunakan `iota()`. Setelah itu pada baris [45], ditentukan jumlah swap sekitar 5% dari total data menggunakan $n / 20$. Kemudian pada baris [46], dibuat generator indeks acak. Selanjutnya pada baris [47]-[49], program menukar beberapa elemen secara acak menggunakan `std::swap()`. Hasil akhirnya adalah data yang sebagian besar sudah terurut namun masih memiliki sedikit posisi yang salah.

Pada baris [53] hingga [58], dibuat kondisi `DUPLICATES`. Program menentukan rentang angka yang lebih kecil menggunakan `range = n / 10`. Karena rentang nilainya kecil sementara jumlahnya besar, maka banyak angka yang akan muncul berulang. Setelah itu pada baris [56], setiap elemen diisi angka random dari rentang kecil tersebut sehingga menghasilkan banyak data duplikat. Pada baris [61], fungsi mengembalikan vector data yang sudah dibuat sesuai kondisi input yang dipilih.

SOAL 1

Untuk setiap algoritma, lakukan analisis terhadap aspek berikut:

Kompleksitas waktu:

- Best Case
- Average Case
- Worst Case

A. Bubble Sort

>> Bubble Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Bubble Sort	Random	100	4950	2564	0	0	0	0.025500
Bubble Sort	Sorted	100	4950	0	0	0	0	0.003100
Bubble Sort	Reverse Sorted	100	4950	4950	0	0	0	0.037600
Bubble Sort	Nearly Sorted	100	4950	296	0	0	0	0.005100
Bubble Sort	Many Duplicates	100	4950	2315	0	0	0	0.015100
Bubble Sort	Random	1000	499500	243190	0	0	0	1.408100
Bubble Sort	Sorted	1000	499500	0	0	0	0	0.441800
Bubble Sort	Reverse Sorted	1000	499500	499500	0	0	0	2.724500
Bubble Sort	Nearly Sorted	1000	499500	25778	0	0	0	0.586900
Bubble Sort	Many Duplicates	1000	499500	240808	0	0	0	1.424000
Bubble Sort	Random	10000	49995000	25118148	0	0	0	149.790900
Bubble Sort	Sorted	10000	49995000	0	0	0	0	33.248400
Bubble Sort	Reverse Sorted	10000	49995000	49995000	0	0	0	266.830100
Bubble Sort	Nearly Sorted	10000	49995000	3119352	0	0	0	43.251300
Bubble Sort	Many Duplicates	10000	49995000	25093086	0	0	0	150.826500

Gambar 15 Hasil Pengujian Bubble Sort

Bubble Sort merupakan metode pengurutan data yang bekerja dengan cara membandingkan dua angka yang berdekatan kemudian menukarnya apabila posisinya salah. Proses ini dilakukan terus menerus dari awal hingga akhir data sampai seluruh angka berada dalam urutan yang benar. Pada setiap putaran, angka terbesar akan bergerak ke bagian akhir sehingga lama-kelamaan data menjadi terurut seluruhnya.

Pada (best case) kondisi terbaik, yaitu ketika data sudah terurut dari awal, Bubble Sort tidak perlu melakukan pertukaran data. Hal ini terlihat pada hasil pengujian bagian Sorted dimana jumlah Swaps bernilai 0 pada semua ukuran data. Waktu yang dibutuhkan juga lebih kecil dibanding kondisi lainnya. Meskipun demikian, algoritma masih tetap melakukan pengecekan pada seluruh data sehingga jumlah Comparisons tetap besar.

Pada (worst case) kondisi terburuk, yaitu ketika data tersusun terbalik atau Reverse Sorted, Bubble Sort harus melakukan pertukaran hampir pada setiap pengecekan. Hal ini terlihat dari jumlah Swaps yang sangat tinggi dan bahkan sama besar dengan jumlah Comparisons. Waktu eksekusi juga menjadi paling lama dibanding kondisi lainnya. Semakin banyak data yang digunakan, waktu proses meningkat sangat besar karena hampir semua elemen harus dipindahkan berulang kali.

Pada kondisi rata-rata atau Random, performa Bubble Sort berada di tengah-tengah antara kondisi terbaik dan terburuk. Algoritma masih melakukan banyak pengecekan dan pertukaran data, tetapi tidak sebanyak pada kondisi reverse sorted. Data *Many Duplicates* dan *Nearly Sorted* juga menunjukkan hasil yang lebih baik

dibanding random karena sebagian data sudah berada cukup dekat dengan posisi yang benar sehingga jumlah pertukaran lebih sedikit.

Bubble Sort sangat dipengaruhi oleh kondisi awal data. Pada data yang sudah terurut, waktu proses menjadi jauh lebih cepat karena tidak terjadi pertukaran. Sebaliknya pada data terbalik, waktu eksekusi meningkat drastis karena banyak perpindahan data yang harus dilakukan. Selain itu, semakin besar jumlah data, waktu proses Bubble Sort meningkat sangat tajam. Hal ini terlihat pada ukuran data 10000 dimana waktu eksekusi mencapai lebih dari 100 ms.

B. Selection Sort

>> Selection Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Selection Sort	Random	100	4950	95	0	0	0	0.007700
Selection Sort	Sorted	100	4950	0	0	0	0	0.006100
Selection Sort	Reverse Sorted	100	4950	50	0	0	0	0.009900
Selection Sort	Nearly Sorted	100	4950	4	0	0	0	0.006300
Selection Sort	Many Duplicates	100	4950	89	0	0	0	0.007700
Selection Sort	Random	1000	499500	995	0	0	0	0.782600
Selection Sort	Sorted	1000	499500	0	0	0	0	1.105500
Selection Sort	Reverse Sorted	1000	499500	500	0	0	0	0.781900
Selection Sort	Nearly Sorted	1000	499500	50	0	0	0	0.776500
Selection Sort	Many Duplicates	1000	499500	983	0	0	0	0.949100
Selection Sort	Random	10000	49995000	9992	0	0	0	85.182500
Selection Sort	Sorted	10000	49995000	0	0	0	0	84.525900
Selection Sort	Reverse Sorted	10000	49995000	5000	0	0	0	85.146000
Selection Sort	Nearly Sorted	10000	49995000	500	0	0	0	84.712600
Selection Sort	Many Duplicates	10000	49995000	9981	0	0	0	84.728800

Gambar 16 Hasil Pengujian Selection Sort

Selection Sort merupakan metode pengurutan data yang bekerja dengan cara mencari nilai paling kecil dari kumpulan data yang belum terurut, lalu menempatkannya ke posisi yang benar. Setelah satu nilai terkecil ditemukan dan dipindahkan, proses yang sama dilakukan kembali pada bagian data berikutnya sampai seluruh data tersusun rapi. Cara kerjanya cukup sederhana karena setiap putaran hanya fokus mencari satu nilai minimum.

Pada (best case) kondisi terbaik, yaitu ketika data sudah terurut dari awal, Selection Sort tetap melakukan pengecekan terhadap seluruh data. Hal ini terlihat dari jumlah Comparisons yang tetap besar meskipun nilai Swaps bernilai 0. Waktu eksekusi juga tidak jauh berbeda dengan kondisi lainnya karena algoritma tetap harus mencari nilai terkecil pada setiap putaran walaupun posisi data sebenarnya sudah benar.

Pada (worst case) kondisi terburuk, yaitu saat data tersusun terbalik atau Reverse Sorted, jumlah pertukaran data meningkat karena hampir setiap putaran menemukan nilai terkecil di posisi yang jauh dari tempat seharusnya. Hal ini terlihat dari jumlah Swaps yang lebih tinggi dibanding kondisi sorted. Namun waktu proses Selection Sort masih relatif stabil karena jumlah pengecekannya tetap hampir sama pada semua kondisi data.

Pada kondisi rata-rata atau Random, Selection Sort tetap melakukan banyak pengecekan untuk mencari nilai terkecil. Jumlah pertukaran data berada di antara kondisi terbaik dan terburuk. Pada data Nearly Sorted, jumlah pertukaran menjadi sangat sedikit karena sebagian besar data sudah mendekati posisi yang benar.

Sementara pada Many Duplicates, pertukaran masih cukup banyak tetapi tidak sebanyak pada data acak penuh.

Performa Selection Sort cukup stabil pada berbagai jenis data. Hal ini terlihat dari waktu eksekusi yang tidak mengalami perubahan terlalu besar antara data random, sorted, maupun reverse sorted. Jumlah Comparisons juga selalu sama karena algoritma tetap memeriksa seluruh bagian data untuk mencari nilai terkecil. Selain itu, semakin besar jumlah data maka waktu proses meningkat cukup drastis. Pada ukuran data 10000, waktu eksekusi mencapai lebih dari 80 ms.

C. Insertion Sort

>> Insertion Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Insertion Sort	Random	100	2656	0	2564	0	0	0.002900
Insertion Sort	Sorted	100	99	0	0	0	0	0.000200
Insertion Sort	Reverse Sorted	100	4950	0	4950	0	0	0.002600
Insertion Sort	Nearly Sorted	100	395	0	296	0	0	0.000500
Insertion Sort	Many Duplicates	100	2412	0	2315	0	0	0.001800
Insertion Sort	Random	1000	244181	0	243190	0	0	0.106400
Insertion Sort	Sorted	1000	999	0	0	0	0	0.001300
Insertion Sort	Reverse Sorted	1000	499500	0	499500	0	0	0.228600
Insertion Sort	Nearly Sorted	1000	26775	0	25778	0	0	0.020600
Insertion Sort	Many Duplicates	1000	241801	0	240808	0	0	0.116000
Insertion Sort	Random	10000	25128135	0	25118148	0	0	10.711900
Insertion Sort	Sorted	10000	9999	0	0	0	0	0.012500
Insertion Sort	Reverse Sorted	10000	49995000	0	49995000	0	0	24.324700
Insertion Sort	Nearly Sorted	10000	3129351	0	3119352	0	0	1.583400
Insertion Sort	Many Duplicates	10000	25103078	0	25093086	0	0	12.004100

Gambar 17 Hasil Pengujian Insertion Sort

Insertion Sort merupakan metode pengurutan data yang bekerja dengan cara mengambil satu data kemudian menyisipkannya ke posisi yang benar pada bagian data yang sudah terurut. Proses ini dilakukan sedikit demi sedikit dari kiri ke kanan sampai seluruh data tersusun rapi. Cara kerjanya mirip seperti menyusun kartu di tangan, dimana setiap kartu baru dimasukkan ke posisi yang sesuai.

Pada (best case) kondisi terbaik, yaitu ketika data sudah terurut dari awal (Sorted), Insertion Sort bekerja sangat cepat. Hal ini terlihat dari jumlah Comparisons dan Shifts yang sangat kecil dibanding kondisi lainnya. Pada ukuran data 10000 misalnya, jumlah comparison hanya sekitar 9999 dan shift bernilai 0. Waktu eksekusinya juga menjadi yang paling kecil karena algoritma tidak perlu memindahkan data.

Pada kondisi terburuk, yaitu ketika data tersusun terbalik (Reverse Sorted), Insertion Sort harus menggeser hampir seluruh data pada setiap langkah. Hal ini menyebabkan jumlah Shifts dan Comparisons meningkat sangat besar. Pada data ukuran 10000, jumlah shift mencapai hampir 50 juta dan waktu proses menjadi jauh lebih lama dibanding kondisi sorted. Semakin besar ukuran data, semakin besar pula peningkatan waktu eksekusinya.

Pada kondisi rata-rata atau Random, performa Insertion Sort berada di tengah antara kondisi terbaik dan terburuk. Algoritma masih melakukan cukup banyak pergeseran data, tetapi tidak sebanyak pada reverse sorted. Pada data Nearly Sorted, performanya jauh lebih baik karena sebagian besar data sudah berada dekat posisi yang benar sehingga perpindahan data menjadi lebih sedikit. Sementara pada

Many Duplicates, jumlah pergeseran juga lebih rendah dibanding random karena banyak nilai yang sama.

Insertion Sort sangat dipengaruhi oleh kondisi awal data. Ketika data sudah terurut atau hampir terurut, algoritma dapat bekerja dengan sangat cepat karena hanya sedikit perpindahan data yang diperlukan. Namun ketika data tersusun terbalik, waktu proses meningkat drastis akibat banyaknya pergeseran yang terjadi. Selain itu, semakin besar jumlah data maka waktu eksekusi juga meningkat cukup tajam, terutama pada kondisi reverse sorted.

D. Merge Sort

>> Merge Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Merge Sort	Random	100	535	0	0	199	0	0.013100
Merge Sort	Sorted	100	356	0	0	199	0	0.008700
Merge Sort	Reverse Sorted	100	316	0	0	199	0	0.008500
Merge Sort	Nearly Sorted	100	453	0	0	199	0	0.008400
Merge Sort	Many Duplicates	100	536	0	0	199	0	0.012800
Merge Sort	Random	1000	8690	0	0	1999	0	0.099400
Merge Sort	Sorted	1000	5044	0	0	1999	0	0.062800
Merge Sort	Reverse Sorted	1000	4932	0	0	1999	0	0.062300
Merge Sort	Nearly Sorted	1000	7628	0	0	1999	0	0.085700
Merge Sort	Many Duplicates	1000	8679	0	0	1999	0	0.093900
Merge Sort	Random	10000	120465	0	0	19999	0	1.015000
Merge Sort	Sorted	10000	69008	0	0	19999	0	0.769000
Merge Sort	Reverse Sorted	10000	64608	0	0	19999	0	0.943900
Merge Sort	Nearly Sorted	10000	111524	0	0	19999	0	0.712700
Merge Sort	Many Duplicates	10000	120461	0	0	19999	0	1.364500

Gambar 18 Hasil Pengujian Merge Sort

Merge Sort merupakan metode pengurutan data yang bekerja dengan cara membagi data menjadi bagian-bagian kecil, kemudian mengurutkannya dan menggabungkannya kembali menjadi satu urutan yang benar. Proses pembagian dilakukan terus menerus sampai data menjadi sangat kecil, lalu setiap bagian digabung kembali secara bertahap hingga seluruh data selesai diurutkan. Cara kerja ini membuat Merge Sort mampu menangani data dalam jumlah besar dengan cukup stabil.

Pada (best case), yaitu ketika data sudah terurut (Sorted), Merge Sort tetap melakukan proses pembagian dan penggabungan data seperti biasa. Walaupun demikian, jumlah Comparisons menjadi lebih sedikit dibanding kondisi random. Hal ini terlihat pada ukuran data 10000 dimana jumlah comparison lebih rendah dibanding beberapa kondisi lainnya. Waktu eksekusinya juga termasuk cepat dan stabil.

Pada (worst case), yaitu ketika data tersusun terbalik (Reverse Sorted), performa Merge Sort tidak mengalami penurunan yang terlalu besar. Jumlah comparison memang meningkat dibanding kondisi sorted, tetapi waktu proses tetap relatif mendekati kondisi lainnya. Hal ini menunjukkan bahwa susunan awal data tidak terlalu mempengaruhi cara kerja Merge Sort.

Pada kondisi rata-rata atau Random, Merge Sort tetap bekerja dengan cukup konsisten. Jumlah comparison dan waktu eksekusi berada di sekitar kondisi lainnya tanpa perbedaan yang terlalu jauh. Pada data Nearly Sorted, waktu proses bahkan sedikit lebih cepat karena beberapa bagian data sudah cukup dekat dengan urutan yang benar. Sementara pada Many Duplicates, jumlah comparison hampir sama

dengan data random karena proses pembagian dan penggabungan tetap berjalan seperti biasa.

Merge Sort memiliki performa yang sangat stabil pada semua kondisi data. Perbedaan waktu eksekusi antara data sorted, reverse sorted, random, maupun duplicates tidak terlalu besar. Selain itu, jumlah Rec.Calls selalu sama untuk ukuran data yang sama karena proses pembagian data dilakukan secara tetap. Ketika ukuran data meningkat dari 100 menjadi 10000, waktu proses memang bertambah, tetapi peningkatannya tidak sebesar Bubble Sort atau Insertion Sort.

E. Quick Sort

>> Quick Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Quick Sort	Random	100	678	394	0	129	0	0.002300
Quick Sort	Sorted	100	4950	5049	0	199	0	0.003500
Quick Sort	Reverse Sorted	100	4950	2549	0	199	0	0.003500
Quick Sort	Nearly Sorted	100	2638	2607	0	191	0	0.003900
Quick Sort	Many Duplicates	100	909	236	0	181	0	0.002000
Quick Sort	Random	1000	10537	6125	0	1341	0	0.027700
Quick Sort	Sorted	1000	499500	500499	0	1999	0	0.349300
Quick Sort	Reverse Sorted	1000	499500	250499	0	1999	0	0.282900
Quick Sort	Nearly Sorted	1000	35189	25081	0	1611	0	0.041900
Quick Sort	Many Duplicates	1000	12685	4920	0	1801	0	0.036600
Quick Sort	Random	10000	147606	77996	0	13315	0	0.382400
Quick Sort	Sorted	10000	49995000	50004999	0	19999	0	32.773400
Quick Sort	Reverse Sorted	10000	49995000	25004999	0	19999	0	27.840300
Quick Sort	Nearly Sorted	10000	622473	458547	0	16511	0	0.693600
Quick Sort	Many Duplicates	10000	181016	67083	0	18003	0	0.302600

Gambar 19 Hasil Pengujian Quick Sort

Quick Sort merupakan metode pengurutan data yang bekerja dengan cara memilih satu nilai sebagai acuan, kemudian memisahkan data yang lebih kecil dan lebih besar dari nilai tersebut. Setelah data terbagi, proses yang sama dilakukan kembali pada bagian kiri dan kanan sampai seluruh data tersusun dengan benar. Cara kerja ini membuat Quick Sort mampu mengurutkan data dengan sangat cepat pada kondisi tertentu.

Pada kondisi terbaik, yaitu ketika pembagian data terjadi dengan seimbang, Quick Sort dapat bekerja sangat cepat. Hal ini biasanya terlihat pada data random dimana jumlah Comparisons, Swaps, dan waktu eksekusi jauh lebih kecil dibanding kondisi lainnya. Pada ukuran data 10000 misalnya, waktu proses random hanya sekitar 0.38 ms, jauh lebih cepat dibanding sorted maupun reverse sorted. Hal ini menunjukkan bahwa Quick Sort bekerja sangat baik ketika data terbagi secara merata.

Pada kondisi terburuk, yaitu ketika data sudah terurut (Sorted) atau terurut terbalik (Reverse Sorted), performa Quick Sort menurun sangat drastis. Hal ini terjadi karena pembagian data menjadi tidak seimbang sehingga proses pengulangan menjadi jauh lebih banyak. Pada hasil pengujian terlihat jumlah Comparisons, Swaps, dan Rec.Calls meningkat sangat besar. Waktu eksekusi juga menjadi paling lama dibanding kondisi lainnya. Pada ukuran data 10000, waktu proses bahkan mencapai lebih dari 27 ms hingga 32 ms.

Pada kondisi rata-rata atau Random, Quick Sort menunjukkan performa yang sangat baik. Jumlah perbandingan dan pertukaran data masih cukup rendah sehingga proses berjalan cepat. Pada data Nearly Sorted, performanya masih cukup

baik walaupun lebih lambat dibanding random karena sebagian data mulai menyebabkan pembagian yang kurang seimbang. Sementara pada Many Duplicates, waktu proses tetap tergolong cepat meskipun jumlah comparison dan swap masih cukup tinggi.

Berdasarkan hasil pengujian, terlihat bahwa performa Quick Sort sangat dipengaruhi oleh kondisi awal data. Pada data random, algoritma bekerja sangat cepat dan efisien bahkan untuk ukuran data besar. Namun pada data sorted dan reverse sorted, waktu proses meningkat sangat drastis karena pembagian data menjadi buruk. Selain itu, jumlah Rec.Calls juga meningkat besar pada kondisi tersebut. Walaupun memiliki kondisi terburuk yang cukup lambat, secara keseluruhan Quick Sort tetap menjadi salah satu algoritma sorting tercepat pada kondisi data normal atau acak.

F. Radix Sort

>> Radix Sort									
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)	
Radix Sort	Random	100	0	0	0	0	900	0.002700	
Radix Sort	Sorted	100	0	0	0	0	900	0.001800	
Radix Sort	Reverse Sorted	100	0	0	0	0	900	0.002200	
Radix Sort	Nearly Sorted	100	0	0	0	0	900	0.001800	
Radix Sort	Many Duplicates	100	0	0	0	0	600	0.001400	
Radix Sort	Random	1000	0	0	0	0	12000	0.015000	
Radix Sort	Sorted	1000	0	0	0	0	12000	0.013200	
Radix Sort	Reverse Sorted	1000	0	0	0	0	12000	0.016200	
Radix Sort	Nearly Sorted	1000	0	0	0	0	12000	0.013100	
Radix Sort	Many Duplicates	1000	0	0	0	0	9000	0.015300	
Radix Sort	Random	10000	0	0	0	0	150000	0.177400	
Radix Sort	Sorted	10000	0	0	0	0	150000	0.151800	
Radix Sort	Reverse Sorted	10000	0	0	0	0	150000	0.151500	
Radix Sort	Nearly Sorted	10000	0	0	0	0	150000	0.158400	
Radix Sort	Many Duplicates	10000	0	0	0	0	120000	0.125600	

Gambar 20 Hasil Pengujian Radix Sort

Radix Sort merupakan metode pengurutan data yang bekerja dengan cara mengelompokkan angka berdasarkan digitnya, dimulai dari digit paling belakang menuju digit paling depan. Algoritma ini tidak membandingkan angka satu per satu seperti Bubble Sort atau Quick Sort, tetapi langsung mengelompokkan angka sesuai nilai digit pada setiap tahap. Karena itu proses pengurutan dapat berjalan cepat dan stabil pada data angka dalam jumlah besar.

Pada kondisi terbaik, Radix Sort bekerja ketika jumlah digit pada data relatif sedikit sehingga proses pengelompokan digit tidak perlu dilakukan terlalu banyak kali. Dari hasil pengujian terlihat bahwa waktu eksekusi pada beberapa kondisi seperti Sorted dan Nearly Sorted cukup kecil. Selain itu nilai Arr.Access juga lebih rendah pada data Many Duplicates karena kemungkinan angka yang digunakan memiliki digit yang lebih pendek atau variasi digit yang lebih sedikit.

Pada kondisi terburuk, performa Radix Sort dipengaruhi oleh banyaknya digit angka yang harus diproses, bukan karena data terurut atau terbalik. Semakin panjang digit suatu angka dan semakin banyak kemiripan d, semakin banyak tahap pengelompokan yang dilakukan sehingga jumlah Arr.Access meningkat. Hal ini terlihat pada ukuran data 10000 dimana Arr.Access mencapai 150000 pada sebagian besar kondisi input. Walaupun demikian, peningkatan waktu proses masih tergolong stabil dibanding algoritma sorting lainnya.

Pada kondisi rata-rata atau Random, Radix Sort tetap menunjukkan performa yang konsisten. Hal ini terlihat dari perbedaan waktu eksekusi antar jenis input yang tidak terlalu jauh. Berbeda dengan algoritma lain, nilai Comparisons, Swaps, Shifts, dan Rec.Calls seluruhnya bernilai 0 pada semua pengujian. Hal ini

terjadi karena implementasi Radix Sort pada kode tidak menggunakan proses perbandingan data, pertukaran data langsung, pergeseran data, maupun rekursi. Aktivitas utama algoritma hanya terlihat pada `Arr.Access` karena proses sorting dilakukan melalui akses dan penyusunan ulang array berdasarkan digit angka.

Berdasarkan hasil pengujian, terlihat bahwa Radix Sort memiliki performa yang sangat stabil pada berbagai kondisi data seperti random, sorted, reverse sorted, nearly sorted, maupun many duplicates. Perbedaan waktu eksekusi antar kondisi sangat kecil dibanding algoritma lain seperti Bubble Sort atau Quick Sort. Selain itu, kenaikan waktu proses saat jumlah data bertambah juga masih cukup teratur. Faktor utama yang mempengaruhi performa Radix Sort adalah jumlah digit angka yang diproses dan kemiripan dari digit pada setiap data, bukan susunan awal data. Dari hasil tersebut dapat disimpulkan bahwa Radix Sort sangat cocok digunakan untuk pengurutan data angka dalam jumlah besar karena mampu bekerja cepat dan konsisten pada berbagai kondisi input data.

SOAL 2

Analisis Karakteristik pada setiap algoritma:

- Apakah algoritma bersifat stable atau tidak.
- Apakah algoritma termasuk in-place atau membutuhkan memori tambahan.
- Mekanisme utama algoritma dalam melakukan pengurutan

A. Bubble Sort

Bubble Sort merupakan algoritma pengurutan yang bekerja dengan cara membandingkan dua data yang posisinya berdekatan. Jika urutannya salah, maka kedua data tersebut ditukar. Proses ini dilakukan terus menerus dari awal hingga akhir data sampai semua angka tersusun dengan benar. Cara kerjanya menggeser angka terbesar sedikit demi sedikit ke bagian belakang pada setiap putaran.

Bubble Sort termasuk algoritma yang bersifat *stable*. Jika ada dua data yang nilainya sama, urutan asli kedua data tersebut tidak berubah setelah proses sorting selesai. Sebagai contoh, misalnya terdapat dua data dengan nilai 5 yang berasal dari posisi berbeda. Setelah diurutkan, kedua angka 5 tersebut akan tetap berada pada urutan yang sama seperti sebelumnya. Hal ini penting pada beberapa kondisi tertentu, misalnya ketika data memiliki informasi tambahan yang urutannya harus tetap dipertahankan.

Selain itu, Bubble Sort juga termasuk algoritma *in-place*. Artinya, proses pengurutan dilakukan langsung pada data utama tanpa membutuhkan tempat penyimpanan tambahan yang besar. Algoritma ini hanya memakai sedikit variabel sementara saat melakukan pertukaran data. Karena itu penggunaan memorinya kecil dan lebih hemat dibanding beberapa algoritma lain yang memerlukan array tambahan.

Mekanisme utama Bubble Sort adalah melakukan pengecekan berulang pada pasangan data yang berdekatan. Misalnya terdapat data:

5 3 8 1

Pada langkah pertama:

- 5 dibandingkan dengan 3 → ditukar menjadi 3 5 8 1
- 5 dibandingkan dengan 8 → tidak ditukar
- 8 dibandingkan dengan 1 → ditukar menjadi 3 5 1 8

Pada akhir putaran pertama, angka terbesar yaitu 8 sudah berada di posisi paling belakang. Proses ini terus diulang sampai semua angka berada pada posisi yang benar.

Untuk kompleksitas waktu, Bubble Sort termasuk algoritma yang cukup lambat jika digunakan pada jumlah data besar. Hal ini terjadi karena algoritma harus melakukan pengecekan berulang kali terhadap banyak pasangan data. Semakin

banyak jumlah data, semakin banyak juga proses perbandingan dan pertukaran yang dilakukan.

Pada kondisi terbaik atau *best case*, Bubble Sort dapat bekerja lebih cepat apabila data sudah terurut dari awal. Kompleksitas waktunya dapat mendekati:

$$O(n)$$

Artinya jumlah proses bertambah secara searah dengan jumlah data. Misalnya jika jumlah data dua kali lebih banyak, maka proses juga bertambah sekitar dua kali lipat. Kondisi ini biasanya terjadi apabila algoritma menggunakan optimasi untuk berhenti lebih awal ketika data sudah rapi. Namun pada kode pengujian yang digunakan, perulangan tetap berjalan penuh sehingga jumlah comparison masih tetap besar walaupun data sudah terurut.

Pada kondisi rata-rata maupun kondisi terburuk, kompleksitas waktunya menjadi:

$$O(n^2)$$

Artinya jumlah proses meningkat sangat cepat ketika jumlah data bertambah. Sebagai gambaran sederhana:

- jika data berjumlah 100, maka proses comparison dilakukan 4950
- jika data menjadi 1000, maka proses comparison dilakukan 499500

Inilah alasan mengapa Bubble Sort menjadi lambat pada data besar. Kondisi terburuk biasanya terjadi ketika data tersusun terbalik dari besar ke kecil karena hampir semua data harus dipindahkan berkali-kali.

Untuk kompleksitas ruang, Bubble Sort memiliki penggunaan memori yang sangat kecil yaitu:

$$O(1)$$

Artinya jumlah memori tambahan yang digunakan tetap kecil walaupun jumlah data semakin besar. Algoritma hanya membutuhkan beberapa variabel sementara untuk membantu proses pertukaran data.

Bubble Sort memiliki kelebihan berupa cara kerja yang sederhana, mudah dipahami, dan hemat memori. Namun kelemahannya adalah waktu proses menjadi sangat lambat ketika jumlah data besar karena terlalu banyak melakukan pengecekan dan pertukaran data.

B. Selection Sort

Selection Sort merupakan algoritma pengurutan yang bekerja dengan cara mencari nilai paling kecil dari bagian data yang belum terurut, kemudian menempatkannya ke posisi yang benar. Proses ini dilakukan berulang kali dari awal hingga akhir data sampai seluruh elemen tersusun secara berurutan. Pada setiap putaran, algoritma memilih satu nilai minimum lalu menukarnya dengan elemen pada posisi saat ini.

Selection Sort termasuk algoritma yang tidak *stable*. Hal ini karena proses pertukaran data dapat mengubah urutan asli elemen yang memiliki nilai sama. Sebagai contoh, apabila terdapat dua data dengan nilai yang sama tetapi berada pada posisi berbeda, proses swap dapat menyebabkan posisi keduanya berubah setelah sorting selesai. Oleh sebab itu urutan awal elemen yang sama tidak selalu dipertahankan.

Selection Sort juga termasuk algoritma *in-place* karena proses pengurutan dilakukan langsung pada array utama tanpa menggunakan tempat penyimpanan tambahan dalam jumlah besar. Algoritma hanya menggunakan beberapa variabel sederhana seperti indeks minimum dan variabel sementara saat pertukaran data dilakukan. Karena itu penggunaan memorinya relatif kecil.

Mekanisme utama Selection Sort adalah mencari nilai terkecil dari bagian data yang belum terurut. Setelah nilai terkecil ditemukan, elemen tersebut dipindahkan ke posisi yang benar melalui proses pertukaran data. Kemudian algoritma melanjutkan pencarian pada bagian data berikutnya sampai seluruh data selesai diurutkan. Sebagai contoh, tersedia data:

5 3 8 1

Langkah algoritma pada data sederhana di atas :

- algoritma mencari nilai terkecil yaitu 1
- lalu menukarnya dengan angka pertama,
- hasil menjadi:

1 3 8 5

Kemudian proses diulang pada sisa data sampai seluruh elemen tersusun.

Untuk kompleksitas waktu, Selection Sort memiliki jumlah proses yang relatif tetap pada berbagai kondisi data karena algoritma selalu melakukan pencarian nilai minimum pada seluruh bagian data yang belum terurut.

Pada kondisi terbaik, rata-rata, maupun terburuk, kompleksitas waktunya adalah:

$$O(n^2)$$

Hal ini terjadi karena algoritma menggunakan dua perulangan bersarang:

- perulangan pertama menentukan posisi elemen,
- perulangan kedua mencari nilai minimum.

Akibatnya jumlah comparison tetap besar walaupun kondisi data sudah terurut.

Berdasarkan hasil pengujian:

- Pada 100 data dilakukan 4950 comparison,
- Pada 1000 data dilakukan 499500 comparison,
- Pada 10000 data dilakukan 49995000 comparison.

Data tersebut menunjukkan bahwa jumlah comparison selalu sama untuk setiap jenis input seperti random, sorted, maupun reverse sorted. Hal ini membuktikan bahwa kondisi awal data tidak mempengaruhi jumlah pengecekan yang dilakukan oleh Selection Sort.

Meskipun jumlah comparison selalu besar, jumlah swap pada Selection Sort cenderung lebih sedikit dibanding Bubble Sort. Hal ini karena dalam satu putaran algoritma hanya melakukan maksimal satu kali pertukaran data setelah nilai minimum ditemukan. Untuk kompleksitas ruang, Selection Sort memiliki penggunaan memori sebesar:

$$O(1)$$

Karena hanya menggunakan sedikit variabel tambahan selama proses sorting berlangsung dan tidak membutuhkan array tambahan. Oleh sebab itu Selection Sort termasuk algoritma yang hemat memori, tetapi kurang efisien dalam waktu proses untuk data berukuran besar.

C. Insertion Sort

Insertion Sort merupakan algoritma pengurutan yang bekerja dengan cara mengambil satu elemen lalu menyisipkannya ke posisi yang sesuai pada bagian data yang sudah terurut. Proses ini dilakukan secara bertahap dari kiri ke kanan sampai seluruh data tersusun dengan benar. Pada setiap langkah, algoritma membandingkan elemen yang sedang diproses dengan elemen sebelumnya, kemudian menggeser data apabila posisinya belum sesuai.

Insertion Sort termasuk algoritma yang bersifat *stable*. Hal ini karena elemen yang memiliki nilai sama tidak akan saling bertukar posisi selama proses sorting berlangsung. Dengan demikian urutan awal data yang bernilai sama tetap dipertahankan setelah pengurutan selesai.

Insertion Sort juga termasuk algoritma *in-place* karena proses pengurutan dilakukan langsung pada array utama tanpa membutuhkan tempat penyimpanan tambahan dalam jumlah besar. Algoritma hanya menggunakan sedikit variabel tambahan seperti key untuk menyimpan sementara nilai yang sedang diproses. Oleh sebab itu penggunaan memorinya relatif kecil.

Mekanisme utama Insertion Sort adalah mengambil satu elemen dari bagian data yang belum terurut, kemudian menyisipkannya ke posisi yang benar pada bagian data yang sudah terurut. Jika terdapat elemen yang lebih besar, maka elemen tersebut digeser ke kanan sampai posisi yang sesuai ditemukan. Sebagai contoh:

5 3 8 1

Langkah pertama:

- angka 3 dibandingkan dengan 5,
- karena 3 lebih kecil, maka 5 digeser ke kanan,
- hasil menjadi:

3 5 8 1

Kemudian proses dilanjutkan pada angka berikutnya sampai seluruh data selesai diurutkan.

Untuk kompleksitas waktu, performa Insertion Sort sangat dipengaruhi oleh kondisi awal data. Pada kondisi terbaik atau *best case*, kompleksitas waktunya adalah:

$$O(n)$$

Kondisi ini terjadi ketika data sudah terurut. Pada keadaan tersebut, algoritma hanya perlu melakukan satu kali comparison untuk setiap elemen tanpa melakukan pergeseran data.

Berdasarkan hasil pengujian:

- Pada data 100 dilakukan 99 comparison,
- Pada data 1000 dilakukan 999 comparison,
- Pada data 10000 dilakukan 9999 comparison,
- Jumlah shift bernilai 0.

Data tersebut menunjukkan bahwa jumlah proses bertambah searah dengan jumlah data sehingga waktu eksekusi relatif cepat pada kondisi sorted.

Pada kondisi rata-rata dan kondisi terburuk, kompleksitas waktunya adalah:

$$O(n^2)$$

Kondisi terburuk terjadi ketika data tersusun terbalik. Dalam keadaan tersebut, setiap elemen harus dibandingkan dan digeser berkali-kali sampai mencapai posisi yang benar.

Berdasarkan hasil pengujian:

- Pada 10000 data reverse sorted dilakukan sekitar 49.995.000 comparison
- Jumlah shift mencapai sekitar 49.995.000.

Hal ini menunjukkan bahwa jumlah proses meningkat sangat besar ketika ukuran data bertambah dan kondisi data semakin tidak terurut.

Pada kondisi random, jumlah comparison dan shift berada di antara kondisi terbaik dan terburuk. Sedangkan pada nearly sorted, jumlah proses lebih kecil dibanding random karena sebagian besar data sudah berada dekat posisi yang benar.

Untuk kompleksitas ruang, Insertion Sort memiliki penggunaan memori sebesar:

$$O(1)$$

Karena hanya menggunakan sedikit variabel tambahan selama proses pengurutan berlangsung dan tidak membutuhkan array tambahan. Oleh sebab itu Insertion Sort termasuk algoritma yang hemat memori dan cukup efisien untuk data kecil atau data yang hampir terurut.

D. Merge Sort

Merge Sort merupakan algoritma pengurutan yang bekerja dengan cara membagi data menjadi beberapa bagian kecil, kemudian mengurutkannya dan menggabungkannya kembali hingga seluruh data tersusun dengan benar. Proses pembagian dilakukan terus menerus sampai ukuran data menjadi sangat kecil, kemudian setiap bagian digabung kembali secara bertahap dalam keadaan terurut.

Merge Sort termasuk algoritma yang bersifat **stable**. Hal ini karena elemen yang memiliki nilai sama tetap mempertahankan urutan awalnya setelah proses sorting selesai. Pada saat proses penggabungan data, elemen dengan nilai sama akan diambil sesuai urutan kemunculannya sehingga posisi relatif data tetap terjaga.

Merge Sort bukan algoritma **in-place** karena membutuhkan memori tambahan selama proses penggabungan data. Pada implementasi kode, algoritma membuat array sementara untuk menyimpan bagian kiri dan kanan data sebelum digabung kembali. Oleh sebab itu penggunaan memorinya lebih besar dibanding algoritma seperti Bubble Sort atau Insertion Sort yang bekerja langsung pada array utama.

Mekanisme utama Merge Sort adalah membagi data menjadi dua bagian, kemudian setiap bagian diproses kembali sampai tersisa satu elemen. Setelah itu data digabung kembali sambil dibandingkan satu per satu sehingga hasil akhirnya menjadi urut.

Sebagai contoh:

8 3 5 1

Data pertama kali dibagi menjadi:

8 3

5 1

Kemudian dibagi lagi:

8

3

5

1

Setelah itu data digabung kembali secara terurut:

3 8

1 5

Lalu digabung menjadi:

1 3 5 8

Untuk kompleksitas waktu, Merge Sort memiliki performa yang stabil pada semua kondisi data karena proses pembagian dan penggabungan tetap dilakukan dengan cara yang sama, baik data sudah terurut maupun belum.

Pada Merge Sort, proses pengurutan dilakukan dengan cara membagi data menjadi dua bagian secara terus menerus, kemudian setiap bagian digabung kembali dalam keadaan terurut. Karena proses pembagian dan penggabungan selalu dilakukan dengan pola yang sama, jumlah proses pada Merge Sort cenderung stabil pada berbagai kondisi data. Hal ini membuat performa Merge Sort tidak terlalu dipengaruhi oleh kondisi awal data seperti random, sorted, maupun reverse sorted.

Berdasarkan hasil pengujian, jumlah Rec.Calls selalu sama untuk ukuran data yang sama karena proses pembagian data berlangsung secara tetap. Selain itu, waktu eksekusi antara data random, sorted, reverse sorted, dan nearly sorted juga tidak memiliki perbedaan yang terlalu besar. Jumlah Comparisons meningkat secara bertahap seiring bertambahnya ukuran data, namun peningkatannya masih jauh lebih rendah dibanding algoritma seperti Bubble Sort atau Insertion Sort. Sebagai contoh, pada ukuran data 10000 jumlah comparison berada di kisaran sekitar 120 ribu hingga 127 ribu, sedangkan Bubble Sort dan Insertion Sort dapat mencapai puluhan juta comparison. Hasil tersebut menunjukkan bahwa Merge Sort lebih efisien untuk pengolahan data dalam jumlah besar.

Untuk kompleksitas ruang, Merge Sort memiliki penggunaan memori sebesar:

$$O(n)$$

Hal ini karena algoritma membutuhkan array tambahan untuk menyimpan sementara bagian kiri dan kanan data selama proses penggabungan berlangsung. Selain itu, proses rekursi juga menambah penggunaan memori selama algoritma dijalankan.

E. Quick Sort

Quick Sort merupakan algoritma pengurutan yang bekerja dengan cara memilih satu elemen sebagai titik acuan atau pivot, kemudian membagi data menjadi dua bagian, yaitu data yang lebih kecil dari pivot dan data yang lebih besar dari pivot. Setelah proses pembagian selesai, algoritma akan melakukan proses yang sama kembali pada bagian kiri dan kanan sampai seluruh data tersusun dengan benar. Cara kerja ini membuat Quick Sort mampu mengurutkan data dengan sangat cepat pada kondisi tertentu.

Quick Sort termasuk algoritma yang **tidak stable**. Hal ini karena selama proses pertukaran data, elemen yang memiliki nilai sama dapat berubah posisi. Akibatnya urutan awal data yang bernilai sama tidak selalu dipertahankan setelah proses sorting selesai.

Quick Sort termasuk algoritma **in-place** karena proses pengurutan dilakukan langsung pada array utama tanpa membutuhkan array tambahan dalam jumlah besar. Algoritma hanya menggunakan beberapa variabel tambahan untuk proses pembagian data dan rekursi. Namun karena menggunakan rekursi, tetap terdapat penggunaan memori tambahan pada stack pemanggilan fungsi.

Mekanisme utama Quick Sort adalah memilih satu elemen sebagai pivot, lalu memindahkan elemen yang lebih kecil ke bagian kiri dan elemen yang lebih besar ke bagian kanan. Setelah pivot berada pada posisi yang benar, proses yang sama dilakukan kembali pada bagian kiri dan kanan data sampai seluruh elemen selesai diurutkan.

Sebagai contoh:

8 3 5 1

Pivot yang akan dipilih adalah 1, maka seluruh data yang lebih kecil dari pivot akan berada di kiri dan yang lebih besar berada di kanan. Setelah proses pembagian selesai, algoritma akan melanjutkan pengurutan pada bagian data yang belum terurut sampai hasil akhirnya menjadi:

1 3 5 8

Untuk kompleksitas waktu, performa Quick Sort sangat dipengaruhi oleh keseimbangan proses pembagian data.

Pada kondisi terbaik atau **best case**, kompleksitas waktunya adalah:

$$O(n \log n)$$

Kondisi ini terjadi ketika pivot membagi data menjadi dua bagian yang relatif seimbang pada setiap proses rekursi. Dalam keadaan tersebut jumlah proses menjadi lebih sedikit sehingga waktu eksekusi berjalan sangat cepat.

Berdasarkan hasil pengujian:

- pada data random, waktu eksekusi Quick Sort termasuk sangat kecil,
- jumlah comparison dan recursive call meningkat secara lebih teratur,
- bahkan pada ukuran data 10000 waktu proses masih jauh lebih cepat dibanding Bubble Sort dan Insertion Sort.

Hal ini menunjukkan bahwa Quick Sort sangat efisien pada kondisi pembagian data yang seimbang. Pada kondisi terburuk atau **worst case**, kompleksitas waktunya menjadi:

$$O(n^2)$$

Kondisi ini terjadi ketika pivot selalu berada pada posisi paling kecil atau paling besar sehingga pembagian data menjadi tidak seimbang. Pada implementasi kode yang digunakan, pivot dipilih dari elemen terakhir array. Oleh sebab itu kondisi sorted dan reverse sorted menyebabkan proses pembagian menjadi buruk.

Berdasarkan hasil pengujian:

- jumlah Comparisons meningkat sangat besar pada data sorted dan reverse sorted,
- jumlah Rec.Calls juga meningkat drastis,
- waktu eksekusi menjadi jauh lebih lama dibanding data random.

Pada ukuran data 10000 data random memiliki waktu proses sekitar kurang dari 1 ms, sedangkan sorted dan reverse sorted mencapai puluhan ms. Hal ini menunjukkan bahwa performa Quick Sort dapat menurun sangat besar apabila pembagian data tidak seimbang.

Untuk kompleksitas ruang, Quick Sort memiliki penggunaan memori berbeda tergantung kondisi proses rekursinya.

Pada kondisi terbaik, kompleksitas ruangnya adalah

$$O(\log n)$$

karena kedalaman rekursi relatif kecil akibat pembagian data yang seimbang.

Sedangkan pada kondisi terburuk, kompleksitas ruangnya menjadi

$$O(n)$$

karena rekursi terjadi sangat dalam akibat pembagian data yang tidak seimbang.

Dari karakteristik tersebut, Quick Sort memiliki kelebihan berupa kecepatan yang sangat tinggi pada kondisi normal atau data acak. Namun performanya dapat menurun drastis pada kondisi tertentu apabila pemilihan pivot menghasilkan pembagian data yang tidak seimbang.

F. Radix Sort

Radix Sort merupakan algoritma pengurutan yang bekerja dengan cara mengelompokkan angka berdasarkan digitnya, dimulai dari digit paling belakang menuju digit paling depan. Algoritma ini tidak membandingkan angka secara langsung seperti Bubble Sort atau Quick Sort, melainkan menyusun data berdasarkan nilai digit pada setiap tahap. Setelah semua digit selesai diproses, data akan berada dalam urutan yang benar.

Radix Sort termasuk algoritma yang bersifat **stable**. Hal ini karena elemen yang memiliki nilai digit sama tetap mempertahankan urutan awalnya setelah proses pengelompokan dilakukan. Pada implementasi yang menggunakan Counting Sort sebagai proses bantu, urutan data dengan nilai sama tetap dijaga sehingga posisi relatif elemen tidak berubah.

Radix Sort bukan algoritma **in-place** karena membutuhkan memori tambahan selama proses sorting berlangsung. Pada implementasi kode, algoritma menggunakan array tambahan seperti output dan count untuk membantu proses pengelompokan digit. Oleh sebab itu penggunaan memorinya lebih besar dibanding algoritma seperti Bubble Sort atau Selection Sort yang bekerja langsung pada array utama.

Mekanisme utama Radix Sort adalah memproses angka berdasarkan posisi digit. Algoritma pertama-tama mengurutkan data berdasarkan digit satuan, kemudian digit puluhan, ratusan, dan seterusnya sampai seluruh digit selesai diproses.

Sebagai contoh terdapat data:

329 145 210 478

Tahap 1

Digit satuan:

- 329 → 9
- 145 → 5
- 210 → 0
- 478 → 8

Data kemudian disusun berdasarkan digit satuannya:

210 145 478 329

karena urutan digit satuannya adalah:

0 5 8 9

Tahap 2

Dari hasil sebelumnya:

210 145 478 329

Digit puluhan:

- 210 → 1
- 145 → 4
- 478 → 7
- 329 → 2

Kemudian data diurutkan kembali berdasarkan digit puluhan:

210 329 145 478

karena urutan digit puluhannya adalah:

1 2 4 7

Tahap 3

Dari hasil sebelumnya:

210 329 145 478

Digit ratusan:

- 210 → 2
- 329 → 3
- 145 → 1
- 478 → 4

Kemudian data diurutkan kembali berdasarkan digit ratusan:

145 210 329 478

Karena urutan digit ratusannya adalah:

1 2 3 4

Setelah seluruh digit selesai diproses dari belakang ke depan, data akhirnya menjadi terurut sepenuhnya.

Untuk kompleksitas waktu, performa Radix Sort dipengaruhi oleh jumlah digit yang dimiliki data, bukan oleh kondisi urutan data seperti sorted atau reverse sorted.

Pada kondisi terbaik atau **best case**, kompleksitas waktunya adalah:

$$O(d(n + k))$$

dengan:

- d = jumlah digit maksimum
- n = jumlah data
- k = jumlah kemungkinan digit

Kondisi terbaik terjadi ketika jumlah digit pada data relatif sedikit sehingga proses pengelompokan digit tidak perlu dilakukan terlalu banyak.

Berdasarkan hasil pengujian:

- waktu eksekusi antar kondisi data tidak memiliki perbedaan besar,
- nilai Comparisons, Swaps, Shifts, dan Rec.Calls selalu bernilai 0,
- aktivitas utama terlihat pada Arr.Access karena proses sorting dilakukan melalui akses array.

Hal ini menunjukkan bahwa performa Radix Sort relatif stabil pada berbagai kondisi input data. Namun, Pada kondisi terburuk, jumlah proses pada Radix Sort meningkat ketika data memiliki digit yang lebih panjang karena algoritma harus melakukan lebih banyak tahap pengelompokan digit. Setiap digit diproses satu per satu mulai dari digit paling belakang hingga digit paling depan. Oleh sebab itu, semakin banyak digit pada suatu angka, semakin banyak juga proses pengelompokan yang harus dilakukan. Berdasarkan hasil pengujian, ketika ukuran data meningkat dari 100 menjadi 10000, nilai Arr.Access juga meningkat sangat besar karena proses akses array dilakukan berulang pada setiap tahap digit. Meskipun demikian, peningkatan waktu proses masih terlihat cukup stabil dan tidak meningkat sedrastis algoritma dengan kompleksitas waktu $O(n^2)$ seperti Bubble Sort atau Selection Sort.

Hal ini menunjukkan bahwa performa Radix Sort lebih dipengaruhi oleh jumlah digit angka dibanding kondisi awal data. Untuk kompleksitas ruang, Radix Sort memiliki penggunaan memori sebesar:

$$O(n + k)$$

karena algoritma membutuhkan array tambahan untuk menyimpan hasil sementara dan proses penghitungan digit.

Dari karakteristik tersebut, Radix Sort memiliki kelebihan berupa performa yang stabil dan cepat untuk pengolahan data angka dalam jumlah besar. Namun algoritma ini membutuhkan memori tambahan dan hanya cocok digunakan pada data yang dapat diproses berdasarkan digit, seperti bilangan bulat.

SOAL 3

Analisis operasi:

- Jumlah comparison.
- Jumlah swap.
- Jumlah shift (khusus algoritma tertentu seperti Insertion Sort).
- Jumlah recursion call.
- Jumlah array accessed.
- Pengaruh bentuk data terhadap jumlah operasi.

A. Bubble Sort

Pada Bubble Sort, bentuk awal data sangat mempengaruhi jumlah operasi yang terjadi selama proses pengurutan. Hal ini terlihat jelas dari perbedaan jumlah Swap dan waktu eksekusi pada setiap jenis input data. Pada kondisi Sorted, data sudah berada dalam urutan yang benar sehingga tidak terjadi pertukaran elemen sama sekali. Akibatnya nilai Swap bernilai 0 pada seluruh ukuran data. Meskipun demikian, jumlah Comparison tetap sangat besar karena implementasi algoritma tetap melakukan seluruh proses pengecekan tanpa penghentian lebih awal. Sebaliknya pada kondisi Reverse Sorted, hampir setiap perbandingan menyebabkan pertukaran data sehingga jumlah swap menjadi yang paling tinggi. Sebagai contoh pada ukuran data 10000, jumlah swap mencapai sekitar 49.995.000 dan waktu eksekusi meningkat hingga sekitar 266 ms. Hal ini menunjukkan bahwa kondisi data yang terbalik membuat Bubble Sort bekerja jauh lebih berat.

Pada kondisi Random, jumlah swap berada di tengah karena sebagian data sudah dekat dengan posisi yang benar dan sebagian lainnya masih perlu dipindahkan. Sementara pada Nearly Sorted, jumlah swap jauh lebih kecil dibanding random karena sebagian besar data sudah hampir terurut. Pada data Many Duplicates, jumlah swap juga cukup besar, namun masih lebih rendah dibanding random dan reverse sorted karena banyak elemen memiliki nilai yang sama sehingga tidak selalu perlu ditukar.

Jumlah Comparison pada Bubble Sort selalu sama untuk ukuran data yang sama. Hal ini terjadi karena algoritma menggunakan dua perulangan bersarang yang tetap berjalan penuh sampai seluruh proses selesai. Berdasarkan hasil pengujian, pada ukuran data 100 dilakukan sekitar 4950 comparison, pada 1000 data sekitar 499500 comparison, dan pada 10000 data mencapai sekitar 49.995.000 comparison. Dari data tersebut terlihat bahwa jumlah comparison meningkat sangat cepat ketika ukuran data bertambah. Peningkatan inilah yang menyebabkan waktu eksekusi Bubble Sort menjadi sangat lambat pada data besar.

Jumlah Swap menjadi faktor utama yang membedakan performa antar kondisi data. Pada sorted, swap bernilai 0 karena tidak ada elemen yang perlu dipindahkan. Pada reverse sorted, hampir seluruh elemen harus dipindahkan berkali-kali sehingga swap mencapai nilai maksimum. Semakin besar jumlah swap,

semakin lama pula waktu proses yang dibutuhkan karena algoritma harus terus melakukan pertukaran posisi data.

Bubble Sort tidak menggunakan proses pergeseran data seperti Insertion Sort sehingga nilai Shift selalu 0 pada seluruh pengujian. Selain itu algoritma ini juga tidak menggunakan rekursi sehingga Rec.Calls bernilai 0. Pada implementasi kode yang digunakan, Arr.Access juga tidak dihitung sehingga nilainya tetap 0 walaupun sebenarnya algoritma tetap melakukan akses array saat membaca dan menukar data.

Dari keseluruhan hasil pengujian, terlihat bahwa kompleksitas waktu Bubble Sort pada kondisi rata-rata dan kondisi terburuk adalah:

$$O(n^2)$$

Karena jumlah comparison dan swap meningkat secara kuadrat terhadap jumlah data. Hal ini terlihat dari lonjakan waktu proses ketika ukuran data berubah dari 1000 menjadi 10000. Pada kondisi terbaik, kompleksitas waktunya sebenarnya dapat menjadi:

$$O(n)$$

Apabila algoritma menggunakan optimasi penghentian lebih awal ketika data sudah terurut. Namun pada implementasi pengujian ini, optimasi tersebut tidak digunakan sehingga jumlah comparison tetap besar walaupun data sudah rapi. Dari hasil tersebut dapat disimpulkan bahwa Bubble Sort mudah dipahami dan sederhana, tetapi kurang efisien untuk data berukuran besar karena jumlah operasinya meningkat sangat cepat ketika jumlah data bertambah.

B. Selection Sort

Pada Selection Sort, bentuk awal data tidak terlalu mempengaruhi jumlah proses utama yang dilakukan algoritma. Hal ini terlihat dari nilai Comparison yang selalu sama pada setiap jenis input dengan ukuran data yang sama. Baik data dalam kondisi Random, Sorted, Reverse Sorted, Nearly Sorted, maupun Many Duplicates, jumlah comparison tetap tidak berubah. Sebagai contoh, pada ukuran data 100 dilakukan sekitar 4950 comparison, pada 1000 data sekitar 499500 comparison, dan pada 10000 data mencapai sekitar 49.995.000 comparison. Hal ini terjadi karena Selection Sort tetap mencari nilai terkecil pada seluruh bagian data yang belum terurut pada setiap putaran, tanpa memperhatikan apakah data sudah rapi atau belum.

Walaupun, jumlah comparison selalu sama, bentuk data tetap mempengaruhi jumlah Swap dan waktu eksekusi. Pada kondisi Sorted, swap bernilai 0 karena seluruh elemen sudah berada di posisi yang benar sehingga tidak diperlukan pertukaran data. Pada kondisi Nearly Sorted, jumlah swap juga sangat kecil karena hanya sedikit elemen yang perlu dipindahkan. Sebaliknya pada Random, Reverse Sorted, dan Many Duplicates, jumlah swap meningkat karena lebih banyak elemen yang berada di posisi yang salah. Namun jumlah swap pada Selection Sort tetap jauh lebih kecil dibanding Bubble Sort karena algoritma ini hanya melakukan maksimal satu pertukaran pada setiap putaran setelah nilai minimum ditemukan.

Sebagai contoh pada ukuran data 10000:

- Random menghasilkan sekitar 9992 swap,
- Reverse Sorted sekitar 5000 swap,
- Many Duplicates sekitar 9981 swap,
- Sorted tetap 0 swap.

Data tersebut menunjukkan bahwa jumlah swap pada Selection Sort meningkat secara lebih stabil dan tidak sebesar Bubble Sort yang dapat mencapai puluhan juta swap.

Selection Sort tidak menggunakan proses pergeseran data sehingga nilai Shift selalu 0 pada seluruh pengujian. Selain itu algoritma ini juga tidak menggunakan rekursi sehingga Rec.Calls bernilai 0. Pada implementasi kode yang

digunakan, Arr.Access juga tidak dihitung sehingga nilainya tetap 0 walaupun sebenarnya algoritma tetap melakukan akses array saat membaca dan menukar data.

Dari hasil pengujian terlihat bahwa waktu eksekusi Selection Sort meningkat cukup besar ketika ukuran data bertambah. Hal ini terutama disebabkan oleh jumlah comparison yang selalu sangat besar pada setiap kondisi data. Kompleksitas waktu Selection Sort pada kondisi terbaik, rata-rata, maupun terburuk tetap sama yaitu:

$$O(n^2)$$

Karena algoritma tetap melakukan pencarian nilai minimum pada seluruh bagian data yang belum terurut. Berbeda dengan Bubble Sort atau Insertion Sort yang dapat menjadi lebih cepat pada data yang hampir terurut, performa Selection Sort cenderung tetap karena jumlah comparison tidak berubah. Dari hasil tersebut dapat disimpulkan bahwa Selection Sort memiliki penggunaan swap yang lebih sedikit dan penggunaan memori yang kecil, tetapi masih kurang efisien untuk data berukuran besar karena jumlah comparison meningkat sangat cepat seiring bertambahnya jumlah data.

C. Insertion Sort

Pada Insertion Sort, bentuk awal data sangat mempengaruhi jumlah operasi yang dilakukan selama proses pengurutan. Hal ini terjadi karena algoritma bekerja dengan cara menyisipkan elemen ke posisi yang benar pada bagian data yang sudah terurut. Jika data sudah hampir rapi, proses perpindahan data menjadi jauh lebih sedikit. Sebaliknya jika data tersusun terbalik, hampir seluruh elemen harus digeser berulang kali sehingga jumlah operasi meningkat sangat besar.

Pengaruh bentuk data terlihat jelas pada jumlah Comparison dan Shift. Pada kondisi Sorted, jumlah comparison sangat kecil karena setiap elemen langsung berada pada posisi yang benar sehingga algoritma hanya melakukan satu pengecekan sederhana pada setiap langkah. Sebagai contoh, pada ukuran data 100 hanya dilakukan 99 comparison, pada 1000 data sekitar 999 comparison, dan pada 10000 data sekitar 9999 comparison. Selain itu jumlah Shift bernilai 0 karena tidak ada elemen yang perlu dipindahkan. Akibatnya waktu eksekusi menjadi sangat cepat.

Sebaliknya pada kondisi Reverse Sorted, jumlah comparison dan shift meningkat sangat besar karena setiap elemen harus dibandingkan dan digeser hingga ke bagian depan array. Pada ukuran data 10000, jumlah comparison mencapai sekitar 49.995.000 dan jumlah shift juga mencapai sekitar 49.995.000. Hal ini membuat waktu eksekusi meningkat drastis hingga sekitar 24 ms. Dari data tersebut terlihat bahwa kondisi data yang terbalik merupakan kondisi paling berat bagi Insertion Sort.

Pada kondisi Random, jumlah comparison dan shift berada di tengah karena sebagian elemen perlu dipindahkan dan sebagian lainnya sudah cukup dekat dengan posisi yang benar. Sedangkan pada Nearly Sorted, jumlah operasi jauh lebih kecil dibanding random karena sebagian besar data sudah hampir terurut. Pada Many Duplicates, jumlah comparison dan shift masih cukup besar, namun sedikit lebih rendah dibanding random karena banyak elemen memiliki nilai yang sama sehingga perpindahan data tidak selalu diperlukan.

Berbeda dengan Bubble Sort atau Selection Sort, Insertion Sort tidak menggunakan proses Swap. Pada implementasi kode yang digunakan, perpindahan

data dilakukan dengan cara menggeser elemen ke kanan sehingga nilai Swap selalu 0.

Operasi utama pada algoritma ini justru terlihat pada bagian Shift karena setiap elemen yang dipindahkan dihitung sebagai pergeseran data.

Insertion Sort juga tidak menggunakan rekursi sehingga Rec.Calls bernilai 0 pada seluruh pengujian. Selain itu Arr.Access tidak dihitung pada implementasi kode sehingga nilainya tetap 0 walaupun algoritma tetap melakukan akses terhadap array selama proses penyisipan data berlangsung.

Dari keseluruhan hasil pengujian terlihat bahwa kompleksitas waktu Insertion Sort sangat dipengaruhi oleh kondisi awal data. Pada kondisi terbaik atau ketika data sudah terurut, kompleksitas waktunya adalah:

$$O(n)$$

Karena algoritma hanya melakukan sedikit comparison tanpa pergeseran data.

Sedangkan pada kondisi rata-rata dan kondisi terburuk, kompleksitas waktunya menjadi:

$$O(n^2)$$

Karena jumlah comparison dan shift meningkat sangat besar ketika data semakin tidak terurut. Dari hasil pengujian dapat disimpulkan bahwa Insertion Sort sangat efisien untuk data kecil atau data yang sudah hampir terurut, namun performanya menurun cukup drastis pada data besar yang tidak terurut.

D. Merge Sort

Pada Merge Sort, bentuk awal data tidak terlalu mempengaruhi jumlah operasi yang dilakukan algoritma. Hal ini terjadi karena Merge Sort selalu membagi data menjadi beberapa bagian kecil lalu menggabungkannya kembali dengan cara yang sama, baik data dalam keadaan random, sorted, reverse sorted, maupun nearly sorted. Akibatnya performa algoritma cenderung stabil pada berbagai kondisi data dan tidak mengalami perbedaan yang terlalu ekstrem seperti Bubble Sort atau Insertion Sort.

Pengaruh bentuk data tetap terlihat pada jumlah Comparison, namun perbedaannya tidak terlalu besar. Pada kondisi Sorted dan Reverse Sorted, jumlah comparison justru sedikit lebih kecil dibanding Random atau Many Duplicates. Sebagai contoh pada ukuran data 10000:

- Random menghasilkan sekitar 120465 comparison,
- Sorted sekitar 69008 comparison,
- Reverse Sorted sekitar 64608 comparison,
- Nearly Sorted mencapai sekitar 111524 comparison.

Data tersebut menunjukkan bahwa jumlah comparison pada Merge Sort tetap meningkat ketika ukuran data bertambah, tetapi peningkatannya jauh lebih rendah dibanding algoritma dengan kompleksitas $O(n^2)$. Hal ini membuat waktu eksekusi Merge Sort tetap relatif stabil walaupun jumlah data menjadi besar.

Pada implementasi Merge Sort yang digunakan, nilai Swap selalu 0 karena algoritma tidak melakukan pertukaran data secara langsung antar elemen. Proses pengurutan dilakukan dengan cara menggabungkan kembali bagian data yang sudah dipisahkan sebelumnya. Selain itu Merge Sort juga tidak menggunakan proses pergeseran data seperti Insertion Sort sehingga nilai Shift tetap 0 pada seluruh pengujian.

Salah satu operasi utama pada Merge Sort terlihat pada bagian Rec.Calls. Nilai ini cukup besar karena algoritma bekerja menggunakan rekursi untuk membagi data menjadi bagian-bagian kecil. Semakin besar jumlah data, semakin banyak pemanggilan fungsi rekursif yang dilakukan. Sebagai contoh:

- Pada ukuran data 100 terdapat sekitar 199 recursive call,
- Pada 1000 data sekitar 1999 recursive call,

- Pada 10000 data mencapai sekitar 19999 recursive call.

Hal ini menunjukkan bahwa jumlah rekursi meningkat secara teratur mengikuti pertambahan ukuran data. Pada implementasi kode yang digunakan, Arr.Access tidak dihitung sehingga nilainya tetap 0 walaupun sebenarnya Merge Sort tetap melakukan akses array selama proses pembagian dan penggabungan data berlangsung.

Dari keseluruhan hasil pengujian terlihat bahwa kompleksitas waktu Merge Sort pada kondisi terbaik, rata-rata, maupun terburuk tetap sama yaitu:

$$O(n \log n)$$

Hal ini terjadi karena proses pembagian dan penggabungan data selalu dilakukan dengan pola yang sama tanpa dipengaruhi secara besar oleh bentuk awal data. Dibanding algoritma seperti Bubble Sort atau Selection Sort, jumlah comparison Merge Sort jauh lebih kecil ketika ukuran data sangat besar. Sebagai contoh pada ukuran data 10000, comparison Merge Sort hanya berada di kisaran ratusan ribu, , sedangkan Bubble Sort dapat mencapai puluhan juta comparison.

E. Quick Sort

Pada Quick Sort, bentuk awal data sangat mempengaruhi jumlah operasi dan waktu eksekusi algoritma. Hal ini terjadi karena Quick Sort bekerja dengan memilih satu elemen sebagai pivot untuk membagi data menjadi dua bagian. Jika pembagian data terjadi dengan seimbang, proses sorting berjalan sangat cepat. Namun jika pembagian tidak seimbang, jumlah operasi meningkat sangat besar dan performa algoritma menurun drastis.

Pengaruh bentuk data terlihat jelas pada jumlah Comparison, Swap, dan Rec.Calls. Pada kondisi Random, jumlah comparison relatif kecil dibanding kondisi sorted atau reverse sorted. Sebagai contoh pada ukuran data 10000, data random menghasilkan sekitar 147666 comparison dengan waktu proses sekitar 0.38 ms. Hal ini menunjukkan bahwa pembagian data berlangsung cukup seimbang sehingga proses sorting berjalan efisien.

Sebaliknya pada kondisi Sorted dan Reverse Sorted, jumlah comparison meningkat sangat besar hingga sekitar 49.995.000 comparison. Waktu eksekusinya juga meningkat drastis menjadi sekitar 27 ms hingga 32 ms. Kondisi ini terjadi karena implementasi Quick Sort pada kode menggunakan elemen terakhir sebagai pivot. Ketika data sudah terurut atau terbalik, pivot selalu berada pada posisi yang buruk sehingga pembagian data menjadi tidak seimbang. Akibatnya jumlah proses rekursi dan comparison meningkat sangat besar.

Pada kondisi Nearly Sorted, performa Quick Sort masih lebih baik dibanding sorted penuh, tetapi jumlah operasi tetap cukup tinggi karena sebagian pembagian data masih belum seimbang. Sementara pada Many Duplicates, jumlah comparison dan swap lebih rendah dibanding random karena banyak elemen memiliki nilai yang sama sehingga perpindahan data tidak sebanyak kondisi lainnya.

Jumlah Swap pada Quick Sort cukup besar karena algoritma terus menukar posisi elemen selama proses partition berlangsung. Pada kondisi sorted, jumlah swap bahkan lebih besar dibanding reverse sorted. Sebagai contoh pada ukuran data 10000:

- Sorted menghasilkan sekitar 5.000.499 swap,
- Reverse Sorted sekitar 2.500.499 swap,
- Random sekitar 77.996 swap.

Hal ini menunjukkan bahwa kondisi data sangat mempengaruhi banyaknya perpindahan elemen pada Quick Sort.

Quick Sort tidak menggunakan proses pergeseran data seperti Insertion Sort sehingga nilai Shift selalu 0 pada seluruh pengujian. Namun algoritma ini menggunakan rekursi secara intensif sehingga nilai Rec.Calls cukup besar. Pada ukuran data 10000:

- Random menghasilkan sekitar 13315 recursive call,
- Sorted dan Reverse Sorted mencapai sekitar 19999 recursive call.

Jumlah rekursi yang tinggi pada sorted dan reverse sorted menunjukkan bahwa pembagian data menjadi sangat tidak seimbang sehingga proses pemanggilan fungsi berlangsung jauh lebih banyak.

Pada implementasi kode yang digunakan, Arr.Access tidak dihitung sehingga nilainya tetap 0 walaupun sebenarnya Quick Sort tetap melakukan akses array selama proses comparison dan partition berlangsung. Dari keseluruhan hasil pengujian terlihat bahwa kompleksitas waktu Quick Sort sangat bergantung pada kualitas pembagian data. Pada kondisi terbaik dan rata-rata, kompleksitas waktunya adalah:

$$O(n \log n)$$

Karena data terbagi cukup seimbang sehingga jumlah proses meningkat secara lebih stabil.

Sedangkan pada kondisi terburuk, kompleksitas waktunya menjadi:

$$O(n^2)$$

Karena pembagian data menjadi sangat tidak seimbang dan menyebabkan jumlah comparison, swap, serta recursive call meningkat drastis.

F. Radix Sort

Pada Radix Sort, bentuk awal data tidak terlalu mempengaruhi jumlah operasi yang dilakukan selama proses pengurutan. Hal ini berbeda dengan Bubble Sort atau Quick Sort yang sangat dipengaruhi kondisi data seperti sorted atau reverse sorted. Radix Sort bekerja berdasarkan digit angka, bukan berdasarkan perbandingan antar elemen. Oleh sebab itu jumlah proses lebih dipengaruhi oleh banyaknya digit dan jumlah data dibanding urutan awal data.

Berdasarkan hasil pengujian, nilai Comparison, Swap, Shift, dan Rec.Calls selalu bernilai 0 pada seluruh kondisi data. Hal ini terjadi karena Radix Sort memang tidak menggunakan proses perbandingan antar elemen seperti algoritma sorting lainnya. Algoritma ini juga tidak melakukan pertukaran data secara langsung antar elemen sehingga nilai swap tetap 0. Selain itu Radix Sort tidak menggunakan proses pergeseran data seperti Insertion Sort dan tidak menggunakan rekursi sehingga nilai shift dan recursion call juga tetap 0 pada seluruh pengujian.

Operasi utama pada Radix Sort terlihat pada bagian Arr.Access. Nilai ini menunjukkan seberapa sering algoritma mengakses array selama proses pengelompokan digit berlangsung. Pada implementasi kode, setiap tahap sorting digit melakukan pembacaan dan penulisan ulang data ke array sementara sehingga jumlah array access menjadi sangat besar. Sebagai contoh:

- Pada ukuran data 100, nilai Arr.Access berada di kisaran 600 hingga 900,
- Pada ukuran data 1000 meningkat menjadi sekitar 9000 hingga 12000,
- Pada ukuran data 10000 mencapai sekitar 120000 hingga 150000.

Perbedaan nilai tersebut dipengaruhi oleh jumlah digit angka yang diproses. Data dengan digit lebih panjang membutuhkan lebih banyak tahap pengelompokan sehingga jumlah akses array meningkat. Hal ini terlihat pada kondisi Many Duplicates yang memiliki nilai Arr.Access lebih rendah dibanding kondisi lainnya karena kemungkinan jumlah digit yang diproses lebih sedikit.

Walaupun jumlah array access cukup besar, waktu eksekusi Radix Sort tetap relatif stabil dan sangat cepat. Pada ukuran data 10000, waktu proses hanya berada di sekitar 0.12 ms hingga 0.17 ms. Perbedaan waktu antar kondisi data juga sangat kecil. Hal ini menunjukkan bahwa performa Radix Sort tidak terlalu dipengaruhi

bentuk awal data, melainkan lebih dipengaruhi jumlah digit pada angka yang diproses.

Kompleksitas waktu Radix Sort adalah:

$$O(d(n + k))$$

dengan:

- d sebagai jumlah digit maksimum,
- n sebagai jumlah data,
- k sebagai jumlah kemungkinan digit.

Karena jumlah digit biasanya tidak terlalu besar, Radix Sort mampu bekerja sangat cepat pada data angka dalam jumlah besar. Selain itu kompleksitas waktunya relatif stabil pada kondisi terbaik, rata-rata, maupun terburuk karena proses pengelompokan digit selalu dilakukan dengan pola yang sama.

Untuk penggunaan memori, Radix Sort membutuhkan array tambahan selama proses sorting berlangsung sehingga algoritma ini tidak termasuk in-place. Kompleksitas ruangnya adalah:

$$O(n + k)$$

Karena algoritma memerlukan array sementara dan array penghitung digit untuk membantu proses pengurutan.

Dari hasil pengujian dapat disimpulkan bahwa Radix Sort sangat efisien untuk pengolahan data angka dalam jumlah besar karena waktu prosesnya stabil dan tidak dipengaruhi secara besar oleh kondisi awal data.

SOAL 4

Analisis performa:

- Perbandingan waktu eksekusi antar algoritma.
- Algoritma mana yang paling cepat untuk dataset kecil dan besar.
- Pengaruh distribusi data terhadap performa algoritma.
- Hubungan antara teori kompleksitas dan hasil eksperimen yang diperoleh.

Berdasarkan hasil pengujian yang diperoleh, terlihat bahwa setiap algoritma sorting memiliki performa yang berbeda tergantung ukuran data dan kondisi dataset yang digunakan. Untuk dataset kecil seperti 100 data, hampir seluruh algoritma memiliki waktu eksekusi yang sangat cepat sehingga perbedaannya tidak terlalu terlihat. Namun ketika ukuran data meningkat menjadi 1000 hingga 10000, perbedaan performa antar algoritma mulai terlihat dengan jelas. Algoritma sederhana seperti Bubble Sort, Selection Sort, dan Insertion Sort mengalami peningkatan waktu eksekusi yang sangat besar karena jumlah operasi meningkat drastis. Sebaliknya Merge Sort, Quick Sort, dan Radix Sort tetap mampu mempertahankan waktu proses yang relatif cepat meskipun jumlah data bertambah besar.

Berdasarkan hasil eksperimen, Radix Sort menjadi algoritma dengan performa paling cepat pada hampir seluruh pengujian data besar. Hal ini terlihat dari waktu eksekusinya yang tetap sangat kecil bahkan pada ukuran data 10000. Merge Sort dan Quick Sort juga menunjukkan performa yang sangat baik pada dataset besar karena jumlah operasi yang dilakukan jauh lebih sedikit dibanding algoritma sederhana. Sementara itu Bubble Sort menjadi algoritma paling lambat terutama pada kondisi reverse sorted karena jumlah comparison dan swap meningkat hingga puluhan juta operasi. Selection Sort juga memiliki performa lambat karena jumlah comparison selalu besar walaupun jumlah swap relatif sedikit. Insertion Sort berada di tengah karena mampu bekerja sangat cepat pada data yang sudah hampir terurut, namun performanya menurun cukup drastis pada data random dan reverse sorted.

Distribusi atau bentuk awal data juga memberikan pengaruh besar terhadap performa beberapa algoritma. Bubble Sort dan Insertion Sort sangat dipengaruhi oleh kondisi data. Ketika data sudah terurut atau nearly sorted, jumlah operasi menjadi jauh lebih sedikit sehingga waktu proses lebih cepat. Sebaliknya pada reverse sorted, kedua algoritma tersebut harus melakukan banyak perpindahan data sehingga waktu eksekusi meningkat drastis. Quick Sort juga sangat dipengaruhi distribusi data karena pemilihan pivot pada implementasi kode menggunakan elemen terakhir array. Akibatnya kondisi sorted dan reverse sorted menghasilkan pembagian data yang buruk dan menyebabkan jumlah comparison serta recursion

call meningkat sangat besar. Berbeda dengan ketiga algoritma tersebut, Merge Sort dan Radix Sort cenderung lebih stabil karena mekanisme kerjanya tidak terlalu dipengaruhi urutan awal data.

Hasil eksperimen yang diperoleh juga sesuai dengan teori kompleksitas waktu masing-masing algoritma. Bubble Sort, Selection Sort, dan Insertion Sort memiliki kompleksitas waktu:

$$O(n^2)$$

sehingga ketika ukuran data meningkat, jumlah operasi bertambah sangat cepat dan menyebabkan waktu eksekusi meningkat drastis. Hal ini terlihat jelas pada pengujian data 10000 yang menghasilkan waktu proses jauh lebih besar dibanding algoritma lainnya.

Merge Sort dan Quick Sort pada kondisi normal memiliki kompleksitas waktu:

$$O(n \log n)$$

karena proses pengurutan dilakukan dengan pembagian data yang lebih efisien. Oleh sebab itu waktu eksekusinya tetap relatif cepat walaupun jumlah data besar. Namun pada Quick Sort, hasil eksperimen juga menunjukkan kondisi worst case yang sesuai teori ketika data sorted atau reverse sorted menyebabkan kompleksitas waktunya berubah menjadi:

$$O(n^2)$$

karena pembagian data menjadi tidak seimbang.

Sedangkan Radix Sort memiliki kompleksitas waktu:

$$O(d(n + k))$$

yang membuat performanya sangat stabil pada data angka dengan jumlah digit yang tidak terlalu besar. Hasil pengujian membuktikan bahwa Radix Sort mampu mempertahankan waktu eksekusi paling cepat dan tidak terlalu dipengaruhi bentuk awal data.

Selain itu bentuk distribusi data juga menjadi faktor penting karena beberapa algoritma sangat sensitif terhadap kondisi awal data, sedangkan algoritma lain mampu mempertahankan performa yang lebih stabil.

MODUL 5 : Search

Searching Algorithm

A. Source Code

Tabel 31 Source Code File searching_algorithms.h

1	#include "searching_algorithms.h"
2	#include <chrono>
3	
4	using Clock = std::chrono::high_resolution_clock;
5	
	void linear_search(const std::vector<int>& data,
6	int target, Metrics& m) {
7	auto start = Clock::now();
8	

Tabel 32 Source Code File searching_algorithms.cpp

1	#include "searching_algorithms.h"
2	#include <chrono>
3	
4	using Clock = std::chrono::high_resolution_clock;
5	
	void linear_search(const std::vector<int>& data,
6	int target, Metrics& m) {
7	auto start = Clock::now();
8	
9	m.comparisons = 0;
10	m.found_index = -1;
11	
12	for (size_t i = 0; i < data.size(); i++) {
13	m.comparisons++;
14	
15	if (data[i] == target) {

```

16         m.found_index = i;
17         break;
18     }
19 }
20
21     m.time_us = std::chrono::duration<long double,
22 std::micro>(
23         Clock::now() - start
24     ).count();
25
26 void binary_search(const std::vector<int>& data,
27 int target, Metrics& m) {
28     auto start = Clock::now();
29
30     m.comparisons = 0;
31     m.found_index = -1;
32
33     int left = 0;
34     int right = data.size() - 1;
35
36     while (left <= right) {
37         int mid = (left + right) / 2;
38
39         m.comparisons++;
40
41         if (data[mid] == target) {
42             m.found_index = mid;
43             break;
44         }
45         else if (data[mid] < target) {
46             left = mid + 1;

```

46	}
47	else {
48	right = mid - 1;
49	}
50	}
51	
	m.time_us = std::chrono::duration<long double,
52	std::micro>(
53	Clock::now() - start
54	).count();
55	}

Tabel 33 Source Code File metrics.h

1	#pragma once
2	#include <string>
3	#include <chrono>
4	
5	struct Metrics {
6	std::string algorithm_name;
7	std::string input_type;
8	int n;
9	
10	long long comparisons = 0;
11	long long found_index = -1; // -1 for not found
12	
13	long double time_us = 0.0;
14	
15	void reset() {
16	comparisons = 0;
17	found_index = -1;
18	time_us = 0.0;
19	}

20	};
21	

Tabel 34 Source Code File reporter.h

1	#pragma once
2	#include "metrics.h"
3	
4	void print_table_header();
5	void print_metrics(const Metrics& m);
6	void print_separator();

Tabel 35 Source Code File reporter.cpp

1	#include "reporter.h"
2	#include <iostream>
3	#include <iomanip>
4	
5	const int W_ALG = 16;
6	const int W_CASE = 10;
7	const int W_N = 10;
8	const int W_CMP = 14;
9	const int W_IDX = 14;
10	const int W_TIME = 16;
11	
12	void print_separator() {
	std::cout << std::string(W_ALG + W_CASE + W_N
13	+ W_CMP + W_IDX + W_TIME, '-') << "\n";
14	}
15	
16	void print_table_header() {
17	print_separator();
18	std::cout << std::left
19	<< std::setw(W_ALG) << "Algorithm"


```

4
5 enum class SearchCase {
6     BEST,
7     AVERAGE,
8     WORST
9 };
10
11 std::string search_case_name(SearchCase sc);
12
13 std::vector<int> generate_sorted_data(int n);
14
15 int generate_target(const std::vector<int>& data,
16                    SearchCase sc,
17                    bool is_binary,
18                    unsigned int seed = 42);
19

```

Tabel 37 Source Code File data_generator.cpp

```

1 #include "data_generator.h"
2 #include <numeric>
3 #include <random>
4 #include <stdexcept>
5
6 std::string search_case_name(SearchCase sc) {
7     switch (sc) {
8         case SearchCase::BEST:    return "Best ";
9         case SearchCase::AVERAGE: return "Average";
10        case SearchCase::WORST:    return "Worst ";
11        default:                   return "Unknown";
12    }
13 }
14

```

```

15 | std::vector<int> generate_sorted_data(int n) {
    |     if (n <= 0) throw std::invalid_argument("N
16 | harus > 0");
17 |     std::vector<int> data(n);
18 |     std::iota(data.begin(), data.end(), 1);
19 |     return data;
20 | }
21 |
22 | int generate_target(const std::vector<int>& data,
23 |                   SearchCase sc,
24 |                   bool is_binary,
25 |                   unsigned int seed)
26 | {
27 |     int n = (int)data.size();
28 |
29 |     switch (sc) {
30 |         case SearchCase::BEST:
    |             return is_binary ? data[n / 2] :
31 | data[0];
32 |
33 |         case SearchCase::AVERAGE: {
34 |             std::mt19937 rng(seed);
    |             std::uniform_int_distribution<int>
35 | dist(0, n - 1);
36 |             return data[dist(rng)];
37 |         }
38 |
39 |         case SearchCase::WORST:
40 |             return data[n - 1] + 9999;
41 |
42 |         default:
43 |             return -1;

```


44	}
45	}
46	

B. Output Program

```
>> Linear Search
```

Algorithm	Case	N	Comparisons	Found Index	Time(us)
Linear Search	Best	1000	1	0	0.100
Linear Search	Average	1000	375	374	2.700
Linear Search	Worst	1000	1000	Not Found	4.500
Linear Search	Best	10000	1	0	0.100
Linear Search	Average	10000	3746	3745	16.800
Linear Search	Worst	10000	10000	Not Found	45.800
Linear Search	Best	100000	1	0	0.100
Linear Search	Average	100000	37455	37454	162.700
Linear Search	Worst	100000	100000	Not Found	439.900
Linear Search	Best	1000000	1	0	0.100
Linear Search	Average	1000000	374541	374540	2043.900
Linear Search	Worst	1000000	1000000	Not Found	7421.300

Gambar 21 Hasil Pengujian Linear Search

```
>> Binary Search
```

Algorithm	Case	N	Comparisons	Found Index	Time(us)
Binary Search	Best	1000	9	500	0.200
Binary Search	Average	1000	3	374	0.100
Binary Search	Worst	1000	10	Not Found	0.400
Binary Search	Best	10000	13	5000	0.200
Binary Search	Average	10000	13	3745	0.300
Binary Search	Worst	10000	14	Not Found	0.500
Binary Search	Best	100000	16	50000	0.200
Binary Search	Average	100000	16	37454	0.400
Binary Search	Worst	100000	17	Not Found	0.500
Binary Search	Best	1000000	19	500000	0.300
Binary Search	Average	1000000	15	374540	1.100
Binary Search	Worst	1000000	20	Not Found	0.400

Gambar 22 Hasil Pengujian Binary Search

C. Pembahasan

Pada bagian file **searching_algorithms.cpp** dilakukan `#include` terhadap file **searching_algorithms.h** dan library `<chrono>` pada baris [1] dan [2]. File header digunakan agar deklarasi fungsi dapat dikenali oleh program lain, sedangkan library `<chrono>` digunakan untuk mengukur waktu eksekusi algoritma. Kemudian pada baris [4], dibuat alias `Clock` untuk `std::chrono::high_resolution_clock`. Tujuan penggunaan alias ini adalah agar penulisan kode menjadi lebih singkat dan mudah dibaca tanpa harus menuliskan nama yang panjang berulang kali.

Fungsi `linear_search` dimulai pada baris [6]. Pada baris [7], program menyimpan waktu awal eksekusi menggunakan `Clock::now()`. Langkah ini diperlukan karena program ingin menghitung berapa lama proses pencarian berlangsung. Setelah itu pada baris [9] dan [10], nilai `comparisons` diatur menjadi 0 dan `found_index` diatur menjadi -1. Nilai 0 digunakan karena sebelum pencarian dimulai belum ada `comparison` yang dilakukan, sedangkan -1 digunakan sebagai tanda awal bahwa data belum ditemukan.

Proses utama `linear search` terjadi pada `Loop for` di baris [12]. Algoritma memeriksa setiap elemen mulai dari indeks pertama hingga indeks terakhir. Pada baris [13], setiap kali satu elemen diperiksa maka `comparisons` ditambah satu. Penambahan ini dilakukan agar program dapat mengetahui berapa banyak perbandingan yang terjadi selama proses pencarian. Kemudian pada baris [15], program membandingkan isi array dengan nilai `target`. Jika nilai tersebut sama, maka indeks tempat data ditemukan disimpan ke `found_index` pada baris [16]. Setelah data ditemukan, program langsung menghentikan `loop` menggunakan `break` pada baris [17]. Penggunaan `break` penting karena setelah `target` ditemukan, tidak ada alasan untuk memeriksa elemen berikutnya. Dengan menghentikan `loop` lebih awal, jumlah operasi dapat dikurangi.

Cara kerja `linear search` dapat dibuktikan langsung dari alur *loop*-nya. Misalnya terdapat data [10, 20, 30, 40] dan `target` 30. Program pertama memeriksa 10, lalu 20, kemudian 30. Ketika nilai 30 ditemukan, indeks 2 disimpan ke `found_index` dan `loop` berhenti.

Pada baris [21]-[23], program menghitung waktu eksekusi menggunakan selisih waktu sekarang dengan waktu awal yang disimpan sebelumnya. Hasilnya

disimpan dalam satuan ms (microseconds) ke variabel `time_us`. Langkah ini dilakukan agar program dapat membandingkan kecepatan masing-masing algoritma secara lebih jelas.

Fungsi `binary_search` dimulai pada baris [26]. Bagian awal fungsi hampir sama dengan `linear search`, yaitu menyimpan waktu awal eksekusi pada baris [27], lalu mengatur nilai awal `comparisons` dan `found_index` pada baris [29] dan [30]. Perbedaannya mulai terlihat pada penggunaan variabel `left` dan `right` di baris [32] dan [33]. Kedua variabel ini digunakan untuk menentukan batas kiri dan kanan area pencarian. Ide utama `binary search` adalah mempersempit area pencarian sedikit demi sedikit hingga target ditemukan.

Pada baris [35], program menggunakan loop `while` yang akan terus berjalan selama batas kiri masih lebih kecil atau sama dengan batas kanan. Di dalam loop, program menghitung indeks tengah menggunakan rumus pada baris [36]. Nilai tengah ini sangat penting karena `binary search` selalu memulai pemeriksaan dari elemen tengah, bukan dari awal data seperti `linear search`.

Pada baris [40], `comparison` kembali ditambah satu setiap kali elemen tengah diperiksa. Kemudian pada baris [42], program memeriksa apakah nilai tengah sama dengan target. Jika sama, maka indeks tengah disimpan ke `found_index` pada baris [43] dan pencarian dihentikan menggunakan `break` pada baris [44].

Jika nilai tengah tidak sama dengan target, program akan menentukan arah pencarian berikutnya. Pada baris [46]-[48], jika nilai tengah lebih kecil dari target maka batas kiri dipindahkan ke `mid + 1`. Hal ini dilakukan karena target pasti berada di sebelah kanan data tengah. Sebaliknya pada baris [50]-[52], jika nilai tengah lebih besar dari target maka batas kanan dipindahkan ke `mid - 1` karena target pasti berada di sebelah kiri. Mekanisme inilah yang membuat `binary search` jauh lebih cepat dibandingkan `linear search` karena setiap langkah langsung membuang setengah area pencarian.

Sebagai contoh, jika data berisi [10, 20, 30, 40, 50, 60, 70] dan target yang dicari adalah 60, maka program pertama memeriksa nilai tengah 40. Karena 60 lebih besar dari 40, maka bagian kiri langsung diabaikan dan pencarian dilanjutkan

hanya pada bagian kanan. Selanjutnya nilai tengah akan diperiksa hingga akhirnya 60 ditemukan. Binary search tidak perlu memeriksa seluruh data satu per satu.

Fungsi binary search di baris [56]-[58], waktu eksekusi kembali dihitung menggunakan cara yang sama seperti linear search. Dengan adanya perhitungan waktu dan comparison.

File **metrics.h** berfungsi sebagai tempat penyimpanan hasil pengujian algoritma pencarian. Struct Metrics digunakan agar semua informasi yang berkaitan dengan proses pengujian dapat dikumpulkan dalam satu tempat sehingga lebih mudah digunakan dan dikelola oleh program lain.

Pada baris [1], digunakan `#pragma once` untuk mencegah file header dimuat berulang kali oleh compiler. Hal ini penting agar tidak terjadi duplikasi definisi ketika file dipanggil di beberapa bagian program.

Pada baris [2] dan [3], program memanggil library yang diperlukan untuk mendukung penggunaan tipe data string dan pengukuran waktu. Library tersebut dibutuhkan karena struct menyimpan nama algoritma, jenis input, dan hasil waktu eksekusi algoritma.

Struct dimulai pada baris [6]. Struct dipilih karena program membutuhkan beberapa variabel yang saling berkaitan untuk menyimpan hasil pengujian. Dengan menggunakan struct, semua data dapat dikumpulkan menjadi satu objek sehingga lebih rapi dibanding membuat banyak variabel terpisah.

Pada baris [7] dan [8], program menyimpan informasi mengenai nama algoritma dan jenis pengujian yang dilakukan. Data ini penting karena program melakukan beberapa kondisi pengujian seperti best case, average case, dan worst case sehingga hasilnya perlu dibedakan.

Pada baris [9], terdapat variabel yang digunakan untuk menyimpan ukuran data yang diuji. Informasi ini diperlukan karena performa algoritma sangat dipengaruhi oleh banyaknya data yang diproses.

Pada baris [11], program menyimpan jumlah comparison yang dilakukan selama pencarian. Nilai awal dibuat 0 karena sebelum algoritma berjalan belum ada comparison yang terjadi. Tipe data yang digunakan dipilih agar mampu menampung jumlah comparison yang sangat besar ketika ukuran data meningkat.

Pada baris [12], program menyimpan indeks tempat target ditemukan. Nilai awal diberikan -1 sebagai penanda bahwa target belum ditemukan. Nilai ini dipilih karena indeks array normal dimulai dari 0, sehingga -1 dapat digunakan sebagai tanda khusus bahwa data tidak ada di dalam array.

Pada baris [14], program menyimpan waktu eksekusi algoritma dalam satuan microseconds. Tipe data yang digunakan dibuat lebih presisi karena waktu

eksekusi algoritma sangat kecil sehingga membutuhkan satuan pengukuran yang cukup kecil.

Pada baris [16]-[20], terdapat fungsi untuk mengembalikan semua hasil pengujian ke kondisi awal. Fungsi ini penting agar hasil pengujian sebelumnya tidak mempengaruhi pengujian berikutnya. Tanpa proses reset, nilai comparison atau waktu sebelumnya masih bisa tersimpan sehingga hasil baru menjadi tidak akurat.

File **searching_algorithms.h** berfungsi sebagai file deklarasi fungsi algoritma pencarian. Pada baris [1], kembali digunakan `#pragma once` agar file tidak dimuat lebih dari satu kali. Pada baris [2], program memanggil library vector karena data pencarian disimpan dalam bentuk vector integer. Kemudian pada baris [3], file ini memanggil `metrics.h` karena kedua algoritma membutuhkan struct `Metrics` untuk menyimpan hasil pengujian.

Pada baris [6] dan [7], program mendeklarasikan fungsi `linear search` dan `binary search`. Kedua fungsi menerima tiga hasil dari data yang akan dicari, target yang ingin ditemukan, dan objek `metrics` untuk menyimpan hasil pengujian seperti jumlah `comparison`, indeks data yang ditemukan, dan waktu eksekusi algoritma.

File **reporter.h** dan **reporter.cpp** digunakan untuk menampilkan hasil pengujian algoritma pencarian dalam bentuk tabel yang rapi. Kedua file ini berfungsi sebagai bagian output program, yaitu menampilkan informasi seperti nama algoritma, jenis pengujian, jumlah comparison, indeks data yang ditemukan, dan waktu eksekusi.

Pada file **reporter.h**, baris [1] menggunakan `#pragma once` untuk mencegah file dimuat berulang kali oleh compiler. Kemudian pada baris [2], file memanggil `metrics.h` karena fungsi-fungsi pada `reporter` membutuhkan struct `Metrics` untuk mengambil data hasil pengujian algoritma.

Pada baris [4]-[6], program mendeklarasikan tiga fungsi yang akan digunakan untuk menampilkan tabel hasil pengujian. Fungsi pertama digunakan untuk menampilkan bagian header tabel, fungsi kedua digunakan untuk menampilkan isi data hasil pengujian, dan fungsi ketiga digunakan untuk membuat garis pemisah agar tampilan tabel lebih rapi dan mudah dibaca.

Pada file **reporter.cpp**, baris [1] memanggil file `reporter.h` agar deklarasi fungsi yang sudah dibuat dapat digunakan kembali. Kemudian pada baris [2] dan [3], program memanggil library yang digunakan untuk menampilkan output ke layar dan mengatur format tampilan tabel.

Pada baris [6]-[11], program membuat beberapa konstanta ukuran kolom tabel. Setiap konstanta digunakan untuk menentukan lebar masing-masing bagian tabel seperti kolom nama algoritma, jenis kasus, ukuran data, jumlah comparison, indeks data, dan waktu eksekusi. Tujuan penggunaan konstanta ini adalah agar tampilan tabel tetap rapi walaupun isi data memiliki panjang yang berbeda-beda. Tanpa pengaturan lebar kolom, hasil output bisa terlihat berantakan dan sulit dibaca.

Fungsi `print_separator()` dimulai pada baris [13]. Fungsi ini digunakan untuk menampilkan garis horizontal sebagai pembatas tabel. Pada baris [14], panjang garis dihitung dari penjumlahan seluruh lebar kolom yang sudah ditentukan sebelumnya. Setelah itu program menampilkan karakter “-” secara berulang sehingga menghasilkan garis pemisah. Proses ini membantu membuat tampilan tabel lebih terstruktur dan mudah dipahami.

Fungsi `print_table_header()` dimulai pada baris [17]. Pada baris [18], program memanggil fungsi pemisah terlebih dahulu agar bagian atas tabel memiliki

garis pembatas. Kemudian pada baris [19]-[25], program menampilkan nama setiap kolom tabel seperti nama algoritma, jenis kasus, ukuran data, jumlah comparison, indeks data yang ditemukan, dan waktu eksekusi. Penggunaan format pada bagian ini bertujuan agar semua teks rata kiri dan berada pada posisi kolom yang sesuai. Setelah header selesai ditampilkan, pada baris [27] program kembali memanggil fungsi pemisah agar bagian header terpisah jelas dari isi tabel.

Fungsi `print_metrics()` dimulai pada baris [30]. Fungsi ini digunakan untuk menampilkan hasil pengujian yang tersimpan di dalam objek `Metrics`. Pada baris [31]-[33], program terlebih dahulu memeriksa nilai `found_index`. Jika nilainya -1, maka program menampilkan tulisan “Not Found”. Namun, jika target ditemukan, maka program mengubah nilai indeks menjadi string agar dapat ditampilkan ke layar.

Pada baris [35]-[42], program menampilkan seluruh hasil pengujian ke dalam tabel. Data seperti nama algoritma, jenis pengujian, ukuran data, jumlah comparison, indeks data, dan waktu eksekusi ditampilkan sesuai kolom masing-masing. Pada baris [41], program mengatur agar angka desimal waktu eksekusi ditampilkan hingga tiga angka di belakang koma.

File **data_generator.h** dan **data_generator.cpp** digunakan untuk membuat data pengujian yang akan dipakai oleh algoritma search. Kedua file ini berfungsi untuk menghasilkan kumpulan data terurut serta menentukan target pencarian sesuai kondisi pengujian seperti best case, average case, dan worst case.

Pada file **data_generator.h**, pada baris [2] dan [3], program memanggil library yang diperlukan untuk penggunaan vector dan string.

Pada baris [6]-[10], program membuat enum class bernama SearchCase. Enum ini digunakan untuk mewakili jenis kondisi pengujian, yaitu best case, average case, dan worst case. Penggunaan enum dipilih agar program lebih mudah dibaca dan mengurangi kemungkinan kesalahan penulisan dibanding menggunakan angka atau string secara langsung. Dengan enum, program dapat langsung memahami kondisi pengujian yang sedang digunakan.

Pada baris [12], program mendeklarasikan fungsi yang digunakan untuk mengubah nilai enum menjadi teks. Fungsi ini diperlukan agar nama kondisi pengujian dapat ditampilkan dengan mudah pada tabel hasil output.

Pada baris [14], program mendeklarasikan fungsi untuk membuat data terurut dengan ukuran tertentu. Data terurut diperlukan terutama untuk binary search karena algoritma tersebut hanya dapat bekerja dengan benar jika data sudah dalam keadaan terurut.

Pada baris [16]-[19], program mendeklarasikan fungsi untuk menentukan target pencarian berdasarkan kondisi pengujian yang dipilih. Fungsi ini menerima kumpulan data, jenis kasus pengujian, penanda apakah algoritma yang digunakan adalah binary search atau bukan, serta nilai seed untuk random generator. Tujuan adanya seed adalah agar hasil pengacakan dapat tetap konsisten ketika program dijalankan kembali.

Pada file **data_generator.cpp**, baris [1] memanggil file **data_generator.h** agar semua deklarasi fungsi dapat digunakan kembali. Kemudian pada baris [2]-[4], program memanggil library tambahan yang digunakan untuk mengisi data otomatis, membuat angka acak, dan menangani error.

Fungsi **search_case_name()** dimulai pada baris [7]. Fungsi ini menggunakan **switch** untuk menentukan teks yang sesuai dengan kondisi pengujian. Jika nilai **enum** adalah **best case** maka program mengembalikan teks “Best”, jika **average case** maka mengembalikan “Average”, dan jika **worst case** maka mengembalikan “Worst”. Pada baris [12], terdapat kondisi default yang mengembalikan “Unknown”. Bagian ini digunakan sebagai pengaman apabila nilai data **enum** yang diterima tidak sesuai dengan kondisi yang tersedia.

Fungsi **generate_sorted_data()** dimulai pada baris [15]. Pada baris [16], program terlebih dahulu memeriksa apakah ukuran data lebih kecil atau sama dengan nol. Jika kondisi tersebut terjadi, program akan menghasilkan error karena tidak masuk akal membuat data dengan ukuran nol atau negatif. Pemeriksaan ini dilakukan untuk menjaga agar program tetap berjalan dengan benar dan menghindari data yang tidak valid.

Pada baris [17], program membuat **vector** dengan ukuran sesuai nilai **n**. Kemudian pada baris [18], program menggunakan fungsi untuk mengisi **vector** secara berurutan mulai dari angka 1 hingga **n**. Proses ini membuat data menjadi terurut secara otomatis. Sebagai contoh, jika $n = 5$, maka data yang dihasilkan adalah [1, 2, 3, 4, 5]. Cara ini dipilih karena lebih singkat dan lebih efisien dibanding mengisi data menggunakan **loop manual**.

Fungsi **generate_target()** dimulai pada baris [22]. Fungsi ini bertugas menentukan target pencarian berdasarkan kondisi pengujian. Pada baris [27]-[28], jika kondisi yang dipilih adalah **best case**, maka target akan disesuaikan dengan jenis algoritma yang digunakan. Jika algoritmanya **binary search**, maka target diambil dari elemen tengah data karena **binary search** memulai pencarian dari bagian tengah. Sedangkan jika algoritmanya **linear search**, target diambil dari elemen pertama karena **linear search** memulai pemeriksaan dari awal data. Pemilihan target seperti ini dilakukan agar benar-benar menghasilkan kondisi **best case** pada masing-masing algoritma.

Pada baris [30]-[34], program menangani kondisi average case. Di bagian ini program menggunakan random generator untuk memilih salah satu elemen data secara acak sebagai target pencarian. Proses ini dilakukan karena average case bertujuan menggambarkan kondisi pencarian umum yang tidak selalu berada di awal atau akhir data. Dengan memilih target secara acak, posisi target bisa berada di berbagai bagian data sehingga hasil pengujian menjadi lebih realistis.

Pada baris [36]-[37], program menangani kondisi worst case. Target dibuat dengan mengambil nilai terakhir data lalu menambahkan angka besar sehingga target dipastikan tidak ada di dalam data. Sebagai contoh, jika data terakhir bernilai 1000, maka target menjadi 10999. Cara ini dipilih agar algoritma benar-benar melakukan pemeriksaan maksimal sebelum menyimpulkan bahwa data tidak ditemukan.

Pada baris [39]-[40], program memberikan nilai -1 sebagai kondisi default apabila jenis pengujian tidak dikenali. Bagian ini digunakan sebagai pengaman tambahan agar fungsi tetap mengembalikan nilai walaupun kondisi yang diterima tidak sesuai.

File **main.cpp** merupakan program utama yang digunakan untuk menjalankan pengujian algoritma search. Program ini bertugas mengatur seluruh proses pengujian mulai dari menentukan ukuran data, memilih jenis kasus pengujian, menjalankan algoritma pencarian, hingga menampilkan hasilnya ke dalam bentuk tabel. Dengan kata lain, file ini menjadi pusat pengendali yang menghubungkan seluruh file sebelumnya seperti `metrics`, `data_generator`, `searching_algorithms`, dan `reporter`.

Pada baris [1]-[4], program memanggil beberapa library standar C++. Library tersebut digunakan untuk menampilkan output, menyimpan kumpulan data, menggunakan fungsi sebagai variabel, dan mendukung penggunaan string. Setiap library dipilih sesuai kebutuhan program agar fitur-fitur yang digunakan dapat berjalan dengan benar.

Pada baris [6]-[9], program memanggil file header yang sebelumnya sudah dibuat. File `metrics.h` digunakan untuk menyimpan hasil pengujian, `data_generator.h` digunakan untuk membuat data dan target pencarian, `searching_algorithms.h` berisi algoritma pencarian, sedangkan `reporter.h` digunakan untuk menampilkan hasil pengujian dalam bentuk tabel. Pemisahan file seperti ini dilakukan agar setiap bagian program memiliki tugas masing-masing sehingga struktur kode menjadi lebih rapi dan mudah dipahami.

Pada baris [12], program membuat kumpulan ukuran data yang akan diuji. Nilai yang digunakan adalah 1000, 10000, 100000, dan 1000000. Pemilihan ukuran data yang terus meningkat dilakukan agar program dapat melihat bagaimana performa algoritma berubah ketika jumlah data semakin besar. Dengan cara ini, perbedaan efisiensi linear search dan binary search dapat terlihat lebih jelas.

Pada baris [14]-[18], program menentukan jenis kasus pengujian yang akan digunakan, yaitu best case, average case, dan worst case. Ketiga kondisi ini dipilih karena masing-masing menggambarkan situasi pencarian yang berbeda. Best case menunjukkan kondisi terbaik, average case menunjukkan kondisi umum, sedangkan worst case menunjukkan kondisi paling sial bagi algoritma.

Pada baris [20], program membuat nilai seed tetap untuk random generator. Penggunaan seed dilakukan agar hasil pengacakan pada average case tetap

konsisten setiap kali program dijalankan. Dengan begitu, hasil pengujian dapat dibandingkan dengan lebih adil karena target acak yang dipilih tidak berubah-ubah.

Pada baris [22], program membuat alias `SearchFn` menggunakan `std::function`. Tujuan bagian ini adalah agar fungsi pencarian dapat diperlakukan seperti data biasa dan disimpan di dalam vector. Cara ini membuat program lebih fleksibel karena linear search dan binary search dapat diproses menggunakan mekanisme yang sama tanpa perlu menulis kode terpisah untuk masing-masing algoritma.

Pada baris [24]-[27], program membuat kumpulan algoritma yang akan diuji. Setiap pasangan data berisi nama algoritma dan fungsi pencarian yang sesuai. Dengan cara ini, program dapat melakukan pengujian terhadap beberapa algoritma secara otomatis menggunakan loop. Proses berpikir pada bagian ini adalah mengurangi pengulangan kode agar program lebih singkat dan lebih mudah dikembangkan apabila nanti ingin menambahkan algoritma lain.

Fungsi utama program dimulai pada baris [29]. Pada baris [30]-[33], program menampilkan judul program ke layar agar output lebih mudah dibaca dan terlihat rapi.

Pada baris [35], program mulai melakukan loop untuk setiap algoritma yang ada di dalam kumpulan algoritma. Program mengambil nama algoritma dan fungsi pencariannya sekaligus. Cara ini memungkinkan linear search dan binary search diuji menggunakan proses yang sama.

Pada baris [36] dan [37], program menampilkan nama algoritma yang sedang diuji lalu memanggil fungsi untuk menampilkan header tabel hasil pengujian.

Pada baris [39], program memeriksa apakah algoritma yang sedang digunakan adalah binary search. Pemeriksaan ini penting karena best case binary search berbeda dengan linear search. Binary search memiliki best case ketika target berada di tengah data, sedangkan linear search memiliki best case ketika target berada di awal data.

Pada baris [41], program mulai melakukan loop untuk setiap ukuran data yang ada pada `N_VALUES`. Kemudian pada baris [42], program membuat data

terurut sesuai ukuran yang sedang diuji. Data dibuat terurut karena binary search hanya dapat bekerja dengan benar jika data dalam keadaan terurut.

Pada baris [44], program kembali melakukan loop untuk setiap jenis kasus pengujian. Kemudian pada baris [45], program menentukan target pencarian menggunakan fungsi generator target berdasarkan jenis kasus yang dipilih.

Pada baris [47]-[50], program membuat objek Metrics untuk menyimpan hasil pengujian. Setelah itu program mengisi nama algoritma, jenis kasus, dan ukuran data. Langkah ini penting agar setiap hasil pengujian memiliki informasi lengkap ketika ditampilkan ke tabel.

Pada baris [52], program menjalankan algoritma pencarian menggunakan fungsi yang sedang dipilih. Fungsi tersebut menerima data, target, dan objek metrics sehingga hasil pencarian dapat langsung disimpan. Setelah proses pencarian selesai, pada baris [53] program menampilkan hasil pengujian menggunakan fungsi reporter.

Pada baris [56], program mencetak garis pemisah setelah seluruh kasus untuk satu ukuran data selesai diuji. Hal ini dilakukan agar tabel output lebih rapi dan lebih mudah dibaca. Pada baris [60]-[61], program menampilkan tulisan bahwa proses pengujian selesai lalu mengembalikan nilai 0.

Linear Search

Linear search adalah algoritma pencarian yang bekerja dengan cara memeriksa data satu per satu mulai dari elemen pertama hingga data ditemukan atau seluruh data selesai dicek. Dari hasil pengujian yang diperoleh, terlihat bahwa jumlah perbandingan dan waktu eksekusi sangat dipengaruhi oleh posisi data yang dicari. Semakin jauh posisi data dari awal array, maka semakin banyak proses pemeriksaan yang dilakukan. Karena itu, linear search memiliki tiga kondisi kompleksitas waktu yaitu best case, average case, dan worst case.

Pada kondisi **best case**, data yang dicari berada tepat di posisi pertama. Berdasarkan hasil pengujian, untuk semua ukuran data seperti 1000, 10000, 100000, hingga 1000000, jumlah perbandingan selalu bernilai 1. Hal ini terjadi karena algoritma langsung menemukan data pada pemeriksaan pertama sehingga tidak perlu memeriksa elemen lainnya. Waktu eksekusi juga sangat kecil dan hampir tidak berubah walaupun jumlah data semakin besar. Dari kondisi ini dapat dibuktikan bahwa waktu pencarian tidak dipengaruhi oleh ukuran data karena prosesnya selalu hanya satu langkah. Oleh sebab itu kompleksitas waktunya adalah:

$$O(1)$$

Kompleksitas ini konstan karena berapapun jumlah data, proses pencarian tetap hanya membutuhkan satu kali pemeriksaan apabila data berada di posisi pertama.

Pada kondisi **average case**, data yang dicari berada di sekitar tengah kumpulan data. Dari hasil pengujian terlihat bahwa untuk $N = 1000$, jumlah perbandingan sekitar 375, kemudian meningkat menjadi 3746 saat $N = 10000$, lalu 37455 saat $N = 100000$, dan 374541 saat $N = 1000000$. Dari data tersebut terlihat bahwa semakin besar ukuran data, maka jumlah pemeriksaan juga ikut bertambah secara sebanding. Hal ini terjadi karena linear search harus memeriksa elemen satu per satu hingga mencapai posisi target. Walaupun target tidak selalu tepat di tengah, rata-rata jumlah pemeriksaan tetap mendekati sebagian besar data. Karena pertumbuhan jumlah langkah mengikuti pertumbuhan jumlah data, maka kompleksitas waktunya adalah:

$$O(n)$$

Huruf n menunjukkan jumlah data. Jika jumlah data bertambah dua kali lipat, maka jumlah pemeriksaan juga akan bertambah mendekati dua kali lipat. Oleh

karena itu, waktu pencarian pada algoritma linear search akan semakin lama ketika jumlah data semakin banyak.

Pada kondisi **worst case**, data yang dicari tidak ditemukan atau berada di posisi terakhir. Berdasarkan hasil pengujian terlihat bahwa jumlah perbandingan selalu sama dengan jumlah data. Saat $N = 1000$, jumlah perbandingan mencapai 1000. Saat $N = 10000$, jumlah perbandingan menjadi 10000, dan terus meningkat hingga 1000000 perbandingan pada data berukuran 1000000. Hal ini membuktikan bahwa algoritma harus memeriksa seluruh elemen satu per satu sebelum menyimpulkan bahwa data tidak ditemukan. Karena jumlah langkah selalu mengikuti banyaknya data, maka kompleksitas waktunya adalah:

$$O(n)$$

Linear search memiliki karakteristik utama yaitu tidak memerlukan sejumlah data yang sudah terurut. Proses pencarian dilakukan dengan cara memeriksa setiap elemen satu per satu mulai dari indeks pertama hingga data ditemukan. Jika terdapat data 40, 10, 25, 5, 30, algoritma tetap dapat mencari nilai 25 tanpa perlu mengurutkan data terlebih dahulu.

Linear search juga termasuk algoritma yang umumnya diimplementasikan secara iteratif. Implementasi iteratif berarti algoritma menggunakan loop seperti for atau while untuk melakukan pemeriksaan data secara berulang. Pada kode program yang digunakan, linear search memakai loop for untuk memeriksa elemen dari awal hingga akhir. Cara ini lebih sederhana, mudah dipahami, dan lebih hemat memori dibandingkan rekursif. Namun sebenarnya linear search juga dapat dibuat dalam bentuk rekursif (fungsi akan memanggil dirinya sendiri) untuk memeriksa elemen berikutnya. Walaupun hasilnya tetap sama, implementasi rekursif biasanya lebih jarang digunakan karena prosesnya lebih panjang dan menggunakan memori tambahan akibat pemanggilan fungsi berulang.

Mekanisme utama linear search dalam melakukan pencarian adalah membandingkan target dengan setiap elemen data secara berurutan. Algoritma dimulai dari indeks pertama lalu melakukan pemeriksaan satu per satu hingga target ditemukan atau seluruh data selesai diperiksa. Jika elemen yang diperiksa sama dengan target, maka pencarian langsung dihentikan dan indeks data disimpan

sebagai hasil pencarian. Namun, jika elemen tidak sama, algoritma akan melanjutkan pemeriksaan ke elemen berikutnya. Sebagai contoh, jika data berisi [12, 7, 20, 35] dan target yang dicari adalah 20, maka algoritma akan memeriksa 12 terlebih dahulu, kemudian 7, lalu 20. Ketika nilai 20 ditemukan, proses pencarian berhenti. Mekanisme ini menyebabkan jumlah pemeriksaan bisa menjadi sangat banyak ketika ukuran data besar atau data yang dicari berada di bagian akhir.

Berdasarkan hasil pengujian, jumlah comparison yang dilakukan sangat bergantung pada posisi target di dalam data. Pada kondisi best case, jumlah comparison selalu bernilai 1 untuk semua ukuran data, baik 1000, 10000, 100000, maupun 1000000. Hal ini terjadi karena target langsung ditemukan pada indeks pertama sehingga algoritma tidak perlu memeriksa elemen lain. Pada kondisi average case, jumlah comparison meningkat menjadi 375, 3746, 37455, dan 374541. Ini menunjukkan bahwa algoritma linear search harus memeriksa cukup banyak elemen sebelum target ditemukan. Di sisi lain, pada kondisi worst case, jumlah comparison selalu sama dengan jumlah data yaitu 1000, 10000, 100000, dan 1000000. Kondisi ini terjadi karena data tidak ditemukan sehingga seluruh elemen harus diperiksa satu per satu hingga akhir.

Nilai `index_found` menunjukkan posisi data yang berhasil ditemukan di dalam array. Jika data ditemukan, maka program akan menyimpan indeks tempat data tersebut berada. Pada best case, nilai `index_found` selalu 0 karena target berada di posisi pertama. Pada average case, indeks yang ditemukan berada di sekitar tengah data seperti 374, 3745, 37454, dan 374540. Hal ini membuktikan bahwa algoritma memerlukan sejumlah pemeriksaan sebelum mencapai target. Sedangkan pada worst case, nilai yang ditampilkan adalah Not Found, yang berarti `found_index = -1`.

Posisi elemen target sangat mempengaruhi jumlah comparison pada linear search. Semakin dekat posisi target dengan awal array, maka semakin sedikit comparison yang diperlukan. Sebaliknya, semakin jauh posisi target dari awal array, maka semakin banyak comparison yang harus dilakukan. Hal ini terlihat jelas pada hasil pengujian. Saat target berada di indeks pertama, algoritma hanya melakukan satu comparison. Namun, ketika target berada di sekitar tengah data, jumlah comparison meningkat cukup besar. Jika target tidak ada atau berada di bagian

paling akhir, maka algoritma harus memeriksa seluruh data. Karena linear search bekerja secara berurutan dari awal hingga akhir, posisi target menjadi faktor utama yang menentukan cepat atau lambatnya proses pencarian.

Ukuran data (n) juga memberikan pengaruh besar terhadap jumlah operasi pada linear search. Dari hasil pengujian terlihat bahwa ketika ukuran data meningkat dari 1000 menjadi 1000000, jumlah comparison dan waktu eksekusi ikut meningkat secara signifikan. Pada average case misalnya, comparison meningkat dari 375 menjadi 374541, sedangkan waktu eksekusi meningkat dari 2.700 us menjadi 2043.900 us. Pada worst case, peningkatan lebih terlihat lagi karena jumlah comparison selalu sama dengan jumlah data. Hal ini menunjukkan bahwa semakin besar ukuran data, maka semakin banyak operasi yang harus dilakukan oleh linear search. Oleh karena itu, linear search cenderung kurang efisien untuk data berukuran sangat besar karena waktu pencarian akan terus bertambah seiring bertambahnya jumlah data yang diperiksa.

Binary Search

Binary search adalah algoritma pencarian yang bekerja dengan cara membagi data menjadi dua bagian secara terus-menerus hingga data yang dicari ditemukan atau ruang pencarian habis. Berbeda dengan linear search yang memeriksa data satu per satu, binary search langsung memeriksa elemen tengah terlebih dahulu. Jika nilai tengah lebih kecil dari target maka pencarian dilanjutkan ke bagian kanan, sedangkan jika nilai tengah lebih besar maka pencarian dilanjutkan ke bagian kiri. Karena area pencarian selalu dibagi dua pada setiap langkah, binary search jauh lebih cepat dibandingkan linear search, terutama pada data berukuran besar. Namun algoritma ini memiliki syarat bahwa data harus sudah terurut sebelum dilakukan pencarian.

Pada kondisi **best case**, data yang dicari langsung berada pada posisi tengah saat pemeriksaan pertama. Dalam kondisi ideal ini, algoritma hanya membutuhkan satu kali pemeriksaan untuk menemukan target. Secara teori, kompleksitas waktunya adalah:

$$O(1)$$

Kompleksitas ini disebut konstan karena proses pencarian dapat selesai hanya dalam satu langkah tanpa dipengaruhi oleh banyaknya data. Walaupun pada hasil pengujian jumlah comparison tidak selalu bernilai 1, hal tersebut terjadi karena target yang digunakan pada pengujian tidak selalu tepat berada di elemen tengah pertama. Contohnya pada data 1000, comparison mencapai 9, dan pada data 1000000, comparison mencapai 19. Namun secara konsep teori, best case binary search tetap dianggap $O(1)$ karena kondisi terbaik terjadi ketika target langsung ditemukan pada pemeriksaan pertama.

Pada kondisi **average case**, target berada di posisi tertentu sehingga algoritma perlu beberapa kali membagi data sebelum target ditemukan. Dari hasil pengujian terlihat bahwa jumlah comparison tetap kecil walaupun ukuran data meningkat sangat besar. Misalnya pada $N = 1000$, comparison hanya 3, sedangkan pada $N = 1000000$, comparison sekitar 15. Hal ini menunjukkan bahwa penambahan jumlah data tidak membuat jumlah langkah meningkat secara drastis. Penyebabnya adalah karena binary search selalu mengurangi area pencarian menjadi setengah pada setiap langkah. Sebagai contoh sederhana, jika terdapat 16

data maka setelah satu langkah ruang pencarian menjadi 8, lalu 4, kemudian 2, dan akhirnya 1. Karena pengurangan dilakukan dengan pembagian dua secara terus-menerus, maka kompleksitas waktunya adalah:

$$O(\log n)$$

Simbol $\log n$ menunjukkan bahwa jumlah langkah bertambah sangat lambat dibandingkan pertambahan jumlah data. Inilah alasan mengapa binary search sangat efisien untuk data berukuran besar.

Pada kondisi **worst case**, data yang dicari tidak ditemukan atau berada pada posisi yang membutuhkan pemeriksaan paling banyak. Walaupun demikian, binary search tetap jauh lebih cepat dibandingkan linear search karena area pencarian terus diperkecil menjadi setengah. Dari hasil pengujian terlihat bahwa untuk $N = 1000$, comparison maksimum hanya 10, sedangkan untuk $N = 1000000$, comparison maksimum sekitar 20. Jumlah ini sangat kecil dibandingkan linear search yang bisa mencapai jutaan comparison pada ukuran data yang sama. Hal ini membuktikan bahwa walaupun ukuran data meningkat sangat besar, jumlah langkah binary search tetap bertambah perlahan. Oleh karena itu kompleksitas waktunya tetap:

$$O(\log n)$$

Binary search memiliki karakteristik utama yaitu hanya dapat digunakan pada data yang sudah terurut. Hal ini karena mekanisme pencariannya bergantung pada proses pembagian data menjadi dua bagian berdasarkan nilai tengah. Jika data belum terurut, maka algoritma tidak dapat menentukan apakah target berada di sebelah kiri atau kanan dari elemen tengah. Sebagai contoh, jika data terurut seperti [10, 20, 30, 40, 50], lalu target yang dicari adalah 40, algoritma dapat langsung membandingkan target dengan nilai tengah dan menentukan arah pencarian berikutnya. Namun, jika data masih acak seperti [30, 10, 50, 20, 40], proses pembagian tersebut tidak akan bekerja dengan benar karena posisi nilai tidak berurutan. Oleh sebab itu, binary search selalu memerlukan data yang sudah diurutkan terlebih dahulu sebelum pencarian dilakukan.

Binary search umumnya diimplementasikan secara iteratif menggunakan `Looping while`, seperti pada kode program yang digunakan dalam pengujian. Pada implementasi iteratif, algoritma terus memperkecil area pencarian dengan

mengubah batas kiri (left) dan batas kanan (right) hingga data ditemukan atau area pencarian habis. Cara ini lebih sederhana dan lebih hemat memori karena tidak memerlukan pemanggilan fungsi berulang. Namun, binary search juga dapat diimplementasikan secara rekursif. Pada implementasi rekursif, fungsi akan memanggil dirinya sendiri untuk memeriksa bagian kiri atau kanan data berdasarkan hasil comparison sebelumnya. Walaupun hasil pencariannya tetap sama, implementasi rekursif biasanya menggunakan memori tambahan karena setiap pemanggilan fungsi disimpan sementara.

Mekanisme utama binary search dalam melakukan pencarian adalah dengan memeriksa elemen tengah dari data yang sudah terurut. Setelah nilai tengah diperiksa, algoritma akan membandingkan nilai tersebut dengan target yang dicari. Jika nilai tengah sama dengan target, maka pencarian selesai. Jika target lebih kecil dari nilai tengah, maka pencarian dilanjutkan hanya pada bagian kiri data. Sebaliknya, jika target lebih besar dari nilai tengah, maka pencarian dilanjutkan hanya pada bagian kanan data. Proses ini terus dilakukan berulang kali hingga target ditemukan atau area pencarian menjadi kosong. Sebagai contoh, jika data berisi [10, 20, 30, 40, 50, 60, 70] dan target yang dicari adalah 60, maka algoritma pertama kali memeriksa nilai tengah yaitu 40. Karena 60 lebih besar dari 40, pencarian dilanjutkan ke bagian kanan. Kemudian nilai tengah berikutnya diperiksa hingga akhirnya 60 ditemukan. Mekanisme pembagian area pencarian menjadi dua bagian inilah yang membuat binary search jauh lebih cepat dibandingkan linear search, terutama pada data berukuran besar.

Algoritma binary search melakukan pencarian dengan cara membagi area pencarian menjadi dua bagian secara terus-menerus hingga data ditemukan atau area pencarian habis. Karena area pencarian selalu diperkecil, jumlah comparison yang dilakukan jauh lebih sedikit dibandingkan linear search. Pada kondisi best case, jumlah comparison berada pada angka 9, 13, 16, dan 19 untuk ukuran data 1000, 10000, 100000, dan 1000000. Pada average case, jumlah comparison berada pada angka 3, 13, 16, dan 15. Sedangkan pada worst case, jumlah comparison hanya mencapai 10, 14, 17, dan 20. Dari hasil tersebut terlihat bahwa walaupun ukuran data meningkat sangat besar, jumlah comparison tidak bertambah secara drastis. Hal ini terjadi karena setiap langkah binary search langsung mengurangi area

pencarian menjadi setengah, sehingga jumlah pemeriksaan menjadi jauh lebih sedikit dibandingkan algoritma yang memeriksa data satu per satu.

Nilai `index_found` menunjukkan posisi data yang berhasil ditemukan. Pada kondisi best case, nilai `index_found` berada pada indeks tengah data seperti 500, 5000, 50000, dan 500000. Hal ini menunjukkan bahwa target ditemukan setelah beberapa proses pembagian data dilakukan. Pada average case, nilai indeks yang ditemukan berada di sekitar 374, 3745, 37454, dan 374540. Nilai tersebut menunjukkan posisi target di dalam array setelah algoritma melakukan beberapa comparison sebelum data ditemukan. Sedangkan pada worst case, hasil yang ditampilkan adalah Not Found, yang berarti nilai `found_index` = -1. Nilai -1 digunakan sebagai tanda bahwa target tidak ditemukan setelah seluruh proses pembagian area pencarian selesai dilakukan.

Posisi elemen target juga mempengaruhi jumlah comparison pada binary search, walaupun pengaruhnya tidak sebesar pada linear search. Jika target berada dekat dengan nilai tengah yang pertama diperiksa, maka jumlah comparison akan lebih sedikit karena data dapat ditemukan lebih cepat. Namun, jika target berada lebih jauh dari posisi tengah atau bahkan tidak ada di dalam data, maka algoritma perlu melakukan lebih banyak pembagian area pencarian. Walaupun demikian, jumlah langkah yang diperlukan tetap relatif kecil karena setiap comparison langsung mengurangi setengah area pencarian.

Ukuran data (n) memberikan pengaruh terhadap jumlah operasi pada binary search, tetapi peningkatannya terjadi secara perlahan. Dari hasil pengujian terlihat bahwa ketika ukuran data meningkat dari 1000 menjadi 1000000, jumlah comparison hanya berubah dari sekitar 10 menjadi sekitar 20. Perubahan ini sangat kecil dibandingkan linear search yang jumlah comparison-nya bisa meningkat hingga jutaan pemeriksaan. Hal ini terjadi karena binary search tidak memeriksa seluruh data satu per satu, melainkan terus membagi area pencarian menjadi dua bagian. Sebagai contoh sederhana, data berukuran 1000000 tidak perlu diperiksa satu juta kali, tetapi cukup sekitar 20 langkah untuk mempersempit area pencarian hingga target ditemukan atau dipastikan tidak ada.

Berdasarkan hasil pengujian dari Linear Search dan Binary Search yang diperoleh, binary search memiliki waktu eksekusi yang jauh lebih kecil dibandingkan linear search pada kondisi data yang sama. Pada linear search, waktu eksekusi meningkat cukup besar ketika ukuran data bertambah. Sebagai contoh, pada ukuran data 1000, waktu eksekusi worst case masih berada pada kisaran mikrodetik kecil, tetapi ketika ukuran data meningkat menjadi 1000000, waktu eksekusi meningkat hingga ribuan microseconds. Hal ini terjadi karena linear search memeriksa data satu per satu dari awal hingga akhir. Sebaliknya, binary search menunjukkan peningkatan waktu yang jauh lebih kecil walaupun ukuran data meningkat sangat besar. Pada data berukuran 1000000, binary search tetap hanya membutuhkan puluhan comparison dan waktu eksekusi yang relatif rendah. Dari hasil tersebut dapat dilihat bahwa binary search mampu melakukan pencarian lebih cepat karena area pencarian langsung diperkecil menjadi setengah pada setiap langkah.

Untuk jumlah data yang kecil, perbedaan performa antara linear search dan binary search tidak terlalu terlihat jauh. Hal ini karena jumlah data yang diperiksa masih sedikit sehingga linear search masih dapat bekerja cukup cepat. Bahkan dalam beberapa kondisi tertentu, linear search dapat memiliki waktu yang hampir sama ketika target berada di posisi awal data. Namun, ketika jumlah data semakin besar, binary search menjadi jauh lebih efisien dibandingkan linear search. Hal ini terlihat dari jumlah comparison linear search yang dapat mencapai jutaan pemeriksaan, sedangkan binary search tetap hanya membutuhkan belasan hingga puluhan comparison saja. Oleh karena itu, linear search masih cukup layak digunakan pada jumlah data yang kecil atau sederhana, sedangkan binary search lebih cocok digunakan pada jumlah data yang besar karena performanya jauh lebih stabil dan cepat.

Kondisi data juga sangat mempengaruhi performa kedua algoritma. Linear search tidak memerlukan data yang terurut karena algoritma ini hanya memeriksa data satu per satu secara berurutan. Oleh sebab itu, performa linear search pada data acak maupun data terurut tidak memiliki perbedaan yang terlalu besar. Sebaliknya, binary search sangat bergantung pada data yang terurut. Jika data tidak terurut, maka mekanisme pembagian area pencarian tidak dapat bekerja dengan benar

karena algoritma tidak bisa menentukan apakah target berada di bagian kiri atau kanan data tengah. Inilah alasan mengapa sebelum menggunakan binary search biasanya diperlukan proses sorting terlebih dahulu. Walaupun sorting membutuhkan waktu tambahan, binary search tetap lebih menguntungkan untuk jumlah data yang besar karena proses pencariannya jauh lebih cepat setelah data terurut.

Hasil eksperimen juga menunjukkan hubungan yang sesuai dengan teori kompleksitas waktu kedua algoritma. Linear search memiliki kompleksitas:

$$O(n)$$

Hal ini berarti jumlah operasi bertambah sebanding dengan jumlah data. Dari hasil pengujian terlihat bahwa ketika ukuran data meningkat berkali-kali lipat, jumlah comparison dan waktu eksekusi linear search juga meningkat secara besar. Kondisi tersebut membuktikan bahwa linear search memang harus memeriksa banyak elemen ketika ukuran data bertambah. Di sisi lain, binary search memiliki kompleksitas:

$$O(\log n)$$

Kompleksitas ini menunjukkan bahwa jumlah langkah bertambah jauh lebih lambat dibandingkan pertambahan jumlah data. Hasil eksperimen membuktikan hal tersebut karena walaupun ukuran data meningkat dari 1000 menjadi 1000000, selisih dari jumlah comparison binary search sedikit.

MODUL 6 : Graph dan Tree

Soal 1 : Konversi Adjacency List ke Adjacency Matrix

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U,V,W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U,V,W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi **adjacency matrix**. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN
M
U1 V1 W1
U2 V2 W2
. . .
UM VM WM
```

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

```
V1 V2 ... VN
V1 a11 a12 ... a1N
V2 a21 a22 ... a2N
...
VN aN1 aN2 ... aNN
```

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki **self-loop**.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 6 A B 4 A C 2 B D 7 C B 1 C E 6 D C 3	Adjacency Matrix: A B C D E A 0 4 2 0 0 B 0 0 0 7 0 C 0 1 0 0 6 D 0 0 3 0 0 E 0 0 0 0 0

Penjelasan Contoh

Edge $A \rightarrow B$ 4 menghasilkan nilai 4 pada baris A kolom B . Karena graph berarah, nilai pada baris B kolom A tetap 0 selama tidak ada edge dari B menuju A .

Edge $D \rightarrow C$ 3 menghasilkan nilai 3 pada baris D kolom C . Vertex E tidak memiliki edge keluar, sehingga seluruh nilai pada baris E adalah 0.

A. Source Code

Tabel 38. Source Code File *probl.cpp*

```
1  #include <iostream>
2  #include <vector>
3  #include <map>
4  using namespace std;
5
6  int main() {
7      int N;
8      cin >> N;
9
10     vector<char> vertex(N);
11     map<char, int> indexVertex;
12
13     for (int i = 0; i < N; i++) {
14         cin >> vertex[i];
15         indexVertex[vertex[i]] = i;
16     }
17
18     vector<vector<int>> matrix(N, vector<int>(N,
19 0));
20
21     int M;
22     cin >> M;
23
24     for (int i = 0; i < M; i++) {
25         char U, V;
26         int W;
27
28         cin >> U >> V >> W;
29
30         int baris = indexVertex[U];
```

```

30         int kolom = indexVertex[V];
31
32         matrix[baris][kolom] = W;
33     }
34
35     cout << "Adjacency Matrix:" << endl;
36
37     cout << " ";
38     for (int i = 0; i < N; i++) {
39         cout << " " << vertex[i];
40     }
41     cout << endl;
42
43     for (int i = 0; i < N; i++) {
44         cout << vertex[i];
45
46         for (int j = 0; j < N; j++) {
47             cout << " " << matrix[i][j];
48         }
49
50         cout << endl;
51     }
52
53     return 0;
54 }

```

B. Output Program

```

1 #include <iostream>
2 #include <vector>
3 #include <map>
4 using namespace std;
5
6 int main() {
7     int M;
8     cin >> M;
9
10    vector<char> vertices(M);
11    map<char, int> indexVertices;
12
13    for (int i = 0; i < M; i++) {
14        cin >> vertices[i];
15        indexVertices[vertices[i]] = i;
16    }
17
18    vector<vector<int>> matrix(M, vector<int>(M, 0));
19
20    int M;
21    cin >> M;
22
23    for (int i = 0; i < M; i++) {
24        char M, V;
25        int M;
26
27        cin >> M >> V >> M;
28
29        int baris = indexVertices[V];
30        int kolom = indexVertices[V];
31
32        matrix[baris][kolom] = M;
33    }
34 }

```

Gambar 23 Screenshot Output Program Soal 1

C. Pembahasan

Kode program pada Soal 1 digunakan untuk membuat *adjacency matrix* dari sebuah graf. Secara sederhana, *adjacency matrix* adalah bentuk tabel yang digunakan untuk menunjukkan hubungan antar simpul atau vertex dalam graf. Pada program ini, setiap vertex disimpan dalam bentuk huruf, misalnya A, B, C, dan seterusnya. Jika terdapat hubungan dari satu vertex ke vertex lain, maka nilai pada matrix akan diisi dengan bobot hubungan tersebut. Jika tidak ada hubungan, maka nilainya tetap 0.

Pada bagian awal program, baris [1–3] digunakan untuk memanggil library yang dibutuhkan. Library *iostream* digunakan agar program dapat menerima input dan menampilkan output. Library *vector* digunakan untuk menyimpan data vertex dan matrix secara dinamis, sedangkan *map* digunakan untuk menghubungkan nama vertex dengan posisi indeksinya di dalam matrix. Dengan adanya *map*, program dapat mengetahui bahwa misalnya vertex A berada pada indeks ke-0, vertex B berada pada indeks ke-1, dan seterusnya.

Pada baris [6–7], program meminta input berupa jumlah vertex yang akan digunakan dalam graf. Nilai tersebut disimpan dalam variabel N. Setelah itu, pada baris [9–10], dibuat sebuah vector bernama *vertex* untuk menyimpan nama-nama vertex, serta *map* bernama *indexVertex* untuk menyimpan pasangan antara nama vertex dan nomor indeksinya. Bagian ini penting karena *adjacency matrix* bekerja berdasarkan posisi baris dan kolom, bukan langsung berdasarkan huruf vertex.

Pada baris [12–15], program melakukan perulangan untuk membaca semua nama vertex yang dimasukkan oleh pengguna. Setiap vertex disimpan ke dalam *vertex[i]*, lalu dicatat juga ke dalam *indexVertex*. Sebagai contoh, jika pengguna memasukkan vertex A, B, dan C, maka program akan menyimpan A pada indeks 0, B pada indeks 1, dan C pada indeks 2. Dengan cara ini, program dapat lebih mudah mencari posisi baris dan kolom dari setiap vertex ketika hubungan antar vertex dimasukkan.

Pada baris [17], program membuat *adjacency matrix* berukuran N x N. Semua nilai awal pada matrix diisi dengan 0. Nilai 0 menunjukkan bahwa pada awalnya belum ada hubungan antar vertex. Misalnya jika terdapat 3 vertex, maka

matrix yang dibuat berukuran 3 baris dan 3 kolom. Baris mewakili vertex asal, sedangkan kolom mewakili vertex tujuan.

Pada baris [19–20], program menerima input jumlah edge atau sisi yang disimpan dalam variabel M. Edge adalah hubungan antar vertex. Setelah itu, pada baris [22–33], program melakukan perulangan sebanyak jumlah edge. Pada setiap perulangan, pengguna memasukkan vertex asal U, vertex tujuan V, dan bobot hubungan W. Bobot ini dapat diartikan sebagai nilai jarak, biaya, waktu, atau ukuran lain yang menunjukkan kekuatan hubungan antar vertex.

Pada baris [28–29], program mencari posisi indeks dari vertex asal dan vertex tujuan menggunakan `indexVertex`. Variabel baris digunakan untuk menyimpan indeks vertex asal, sedangkan variabel kolom digunakan untuk menyimpan indeks vertex tujuan. Setelah posisi baris dan kolom ditemukan, pada baris [31] program mengisi nilai matrix sesuai bobot yang dimasukkan. Misalnya jika terdapat input A B 5, maka artinya ada hubungan dari vertex A menuju vertex B dengan bobot 5. Jika A berada pada baris 0 dan B berada pada kolom 1, maka `matrix[0][1]` akan diisi dengan nilai 5.

Bagian output dimulai pada baris [35]. Program menampilkan teks "Adjacency Matrix:" sebagai judul hasil matrix. Pada baris [37–41], program mencetak bagian kepala kolom matrix, yaitu nama-nama vertex. Hal ini bertujuan agar pembaca dapat mengetahui kolom tersebut mewakili vertex apa. Setelah itu, pada baris [43–52], program mencetak isi matrix secara lengkap. Setiap baris diawali dengan nama vertex, kemudian diikuti oleh nilai-nilai hubungan dari vertex tersebut menuju vertex lainnya.

Program ini menggambarkan graf berarah, karena nilai hubungan hanya diisi pada posisi `matrix[baris][kolom]`. Artinya, jika ada input hubungan dari A ke B, maka yang diisi hanya posisi baris A kolom B. Posisi sebaliknya, yaitu baris B kolom A, tidak otomatis ikut terisi. Dengan demikian, hubungan A ke B dianggap berbeda dengan hubungan B ke A. Jika ingin membuat graf tidak berarah, maka program perlu menambahkan pengisian sebaliknya, yaitu `matrix[kolom][baris] = W`.

Soal 2: Konversi Adjacency Matrix ke Adjacency List

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang direpresentasikan dalam bentuk **adjacency matrix** berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi **adjacency list**. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN
A11 A12 ... A1N
A21 A22 ... A2N
...
AN1 AN2 ... ANN
```

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .
- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .
- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

Adjacency List:

V1 -> (X1,w1) , (X2,w2) , ...

V2 -> (X1,w1) , (X2,w2) , ...

...

VN -> ...

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

Constraints

- $2 \leq N \leq 10$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge
- Nilai positif berarti bobot edge
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 0 4 2 0 0 0 0 0 7 0 0 1 0 0 6 0 0 3 0 0 0 0 0 0 0	Adjacency List: A -> (B,4), (C,2) B -> (D,7) C -> (B,1), (E,6) D -> (C,3) E -> -

Penjelasan Contoh

Pada baris vertex A, nilai positif muncul pada kolom B dan C. Karena itu, adjacency list untuk A adalah (B, 4) , (C, 2) .

Pada baris vertex E, semua nilai bernilai 0, sehingga vertex E tidak memiliki edge keluar dan output-nya adalah E -> -.

A. Source Code

Tabel 39 Source Code File prob2.cpp

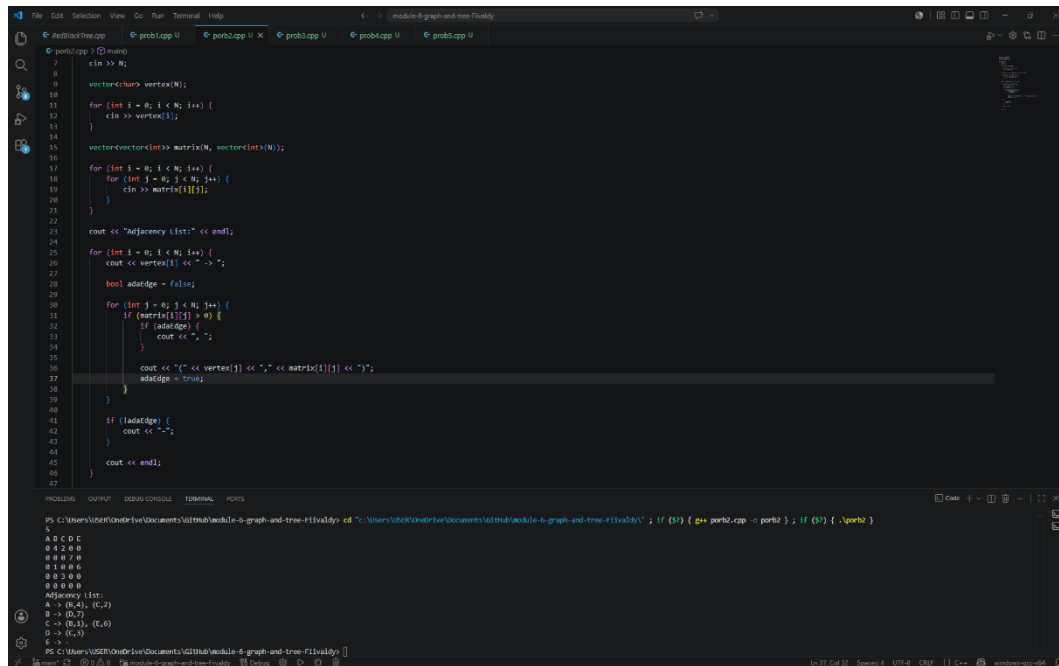
1	#include <iostream>
2	#include <vector>
3	using namespace std;
4	
5	int main() {
6	int N;
7	cin >> N;
8	
9	vector<char> vertex(N);
10	
11	for (int i = 0; i < N; i++) {
12	cin >> vertex[i];
13	}
14	
15	vector<vector<int>> matrix(N, vector<int>(N));
16	
17	for (int i = 0; i < N; i++) {
18	for (int j = 0; j < N; j++) {
19	cin >> matrix[i][j];
20	}
21	}
22	
23	cout << "Adjacency List:" << endl;
24	
25	for (int i = 0; i < N; i++) {
26	cout << vertex[i] << " -> ";
27	
28	bool adaEdge = false;
29	
30	for (int j = 0; j < N; j++) {

```

31         if (matrix[i][j] > 0) {
32             if (adaEdge) {
33                 cout << ", ";
34             }
35
36             cout << "(" << vertex[j] << ", " <<
37 matrix[i][j] << ")";
38             adaEdge = true;
39         }
40
41         if (!adaEdge) {
42             cout << "-";
43         }
44
45         cout << endl;
46     }
47
48     return 0;
49 }

```

B. Output Program



```
PS C:\Users\USER\OneDrive\Documents\Uitdachtmodule-6-graph-and-tree-f1ivaldy> cd "C:\Users\USER\OneDrive\Documents\Uitdachtmodule-6-graph-and-tree-f1ivaldy"; if ($?) { g++ port2.cpp -o port2 }; if ($?) { .\port2 }
```

```
5
A B C D E
0 0 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency list:
A -> (B,4), (C,2)
B -> (D,7)
C -> (D,3), (E,6)
D -> (E,5)
E ->
```

Gambar 24 Screenshot Output Program Soal 2

C. Pembahasan

Program Soal 2 berfungsi untuk mengubah data graf dari bentuk **adjacency matrix** menjadi bentuk **adjacency list**. Secara sederhana, adjacency matrix adalah bentuk tabel yang menunjukkan hubungan antar vertex menggunakan baris dan kolom, sedangkan adjacency list menampilkan hubungan tersebut dalam bentuk daftar. Dengan adjacency list, setiap vertex ditampilkan satu per satu, lalu diikuti daftar vertex lain yang terhubung dengannya. Program ini berguna karena bentuk daftar biasanya lebih mudah dibaca ketika ingin melihat hubungan dari satu vertex ke vertex lain secara langsung.

Pada bagian awal, program memanggil library yang diperlukan melalui [1–2]. Library `iostream` digunakan agar program dapat menerima input dan menampilkan output, sedangkan `vector` digunakan untuk menyimpan kumpulan data seperti daftar vertex dan isi matrix. Setelah itu, pada [5–7], program membaca jumlah vertex yang akan digunakan dan menyimpannya ke dalam variabel `N`. Nilai `N` ini menjadi dasar utama dalam program, karena jumlah vertex menentukan banyaknya nama vertex yang akan dibaca serta ukuran matrix yang harus disiapkan. Setelah jumlah vertex diketahui, program membuat tempat penyimpanan bernama vertex pada [9]. Bagian ini digunakan untuk menyimpan nama-nama vertex dalam bentuk karakter, misalnya `A`, `B`, `C`, dan seterusnya. Kemudian pada [11–13], program membaca nama vertex satu per satu sesuai jumlah yang sudah ditentukan. Misalnya jika `N` bernilai 4, maka program akan membaca 4 nama vertex. Penyimpanan nama vertex ini penting karena nantinya nama tersebut akan dipakai saat program menampilkan hasil adjacency list, sehingga output tidak hanya berupa angka indeks, tetapi tetap menggunakan nama vertex yang mudah dipahami.

Pada [15], program membuat matrix dua dimensi menggunakan `vector<vector<int>>`. Matrix ini berukuran $N \times N$, artinya jumlah baris dan kolomnya sama dengan jumlah vertex. Dalam konsep graf, baris pada matrix dapat dipahami sebagai vertex asal, sedangkan kolom sebagai vertex tujuan. Nilai yang berada pada pertemuan baris dan kolom menunjukkan apakah ada hubungan dari vertex asal ke vertex tujuan. Jika nilainya lebih dari 0, berarti ada hubungan dengan bobot tertentu. Jika nilainya 0, berarti tidak ada hubungan langsung.

Bagian [17–21] digunakan untuk membaca isi adjacency matrix dari input. Program membaca nilai matrix satu per satu menggunakan dua perulangan. Perulangan pertama berjalan berdasarkan baris, sedangkan perulangan kedua berjalan berdasarkan kolom. Dengan cara ini, semua nilai hubungan antar vertex dapat dimasukkan secara lengkap. Misalnya, nilai pada baris A kolom B menunjukkan hubungan dari A menuju B. Jika nilainya 5, maka artinya terdapat hubungan dari A ke B dengan bobot 5. Jika nilainya 0, maka tidak ada hubungan langsung dari A ke B.

Setelah semua data matrix berhasil dibaca, program mulai menampilkan hasil dalam bentuk adjacency list pada [23]. Setiap vertex akan dicetak satu per satu melalui perulangan pada [25–45]. Pada awal setiap baris output, program menampilkan nama vertex asal, lalu diikuti tanda panah $\rightarrow$. Tanda panah ini digunakan agar hasil lebih mudah dibaca, karena menunjukkan bahwa daftar setelah tanda panah adalah vertex-vertex tujuan yang terhubung dari vertex tersebut.

Variabel `adaEdge` pada [28] digunakan sebagai penanda apakah suatu vertex memiliki hubungan ke vertex lain atau tidak. Pada awalnya, nilainya dibuat `false`, karena program belum menemukan hubungan apa pun dari vertex yang sedang diperiksa. Setelah itu, pada [30–38], program memeriksa seluruh kolom pada baris matrix tersebut. Jika ditemukan nilai matrix yang lebih dari 0, berarti ada hubungan dari vertex asal ke vertex tujuan. Program kemudian menampilkan vertex tujuan beserta bobotnya dalam bentuk pasangan, misalnya (B,5), yang berarti vertex asal terhubung ke vertex B dengan bobot 5.

Bagian [32–34] digunakan untuk mengatur tanda koma pada output. Jika sebuah vertex memiliki lebih dari satu hubungan, maka setiap hubungan akan dipisahkan dengan koma agar hasilnya terlihat rapi. Program tidak langsung mencetak koma sejak awal, karena hubungan pertama tidak perlu diawali koma. Setelah satu hubungan berhasil dicetak, nilai `adaEdge` diubah menjadi `true` pada [37]. Perubahan ini menandakan bahwa program sudah menemukan setidaknya satu hubungan dari vertex tersebut.

Jika setelah seluruh kolom diperiksa ternyata tidak ada nilai yang lebih dari 0, maka berarti vertex tersebut tidak memiliki hubungan keluar menuju vertex lain. Kondisi ini diperiksa pada [41–43]. Jika `adaEdge` masih bernilai `false`, program

akan mencetak tanda -. Tanda ini digunakan sebagai penanda bahwa vertex tersebut tidak memiliki edge atau hubungan keluar. Dengan begitu, output tetap jelas dan tidak terlihat kosong.

Soal 3: BFS: Jarak Terpendek Keluar Labirin

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma **Breadth-First Search** (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C
G21
G22
...
G2C
...
GR1 GR2 ... GRC
SR SC
FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0

Format Output

Cetak satu bilangan bulat.

X

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1 .

Constraints

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 7 9	16

Penjelasan Contoh

Posisi awal berada pada $(0, 0)$ dan posisi tujuan berada pada $(7, 9)$. Labirin memiliki beberapa percabangan dan jalan buntu, misalnya cabang menuju area kiri bawah dan cabang menuju sel $(2, 1)$.

Dengan BFS, setiap sel diproses berdasarkan jarak terdekat dari start. Jarak minimum dari $(0, 0)$ ke $(7, 9)$ adalah 16 langkah, sehingga output yang dicetak hanya 16.

A. Source Code

Tabel 40 Source Code File prob3.cpp

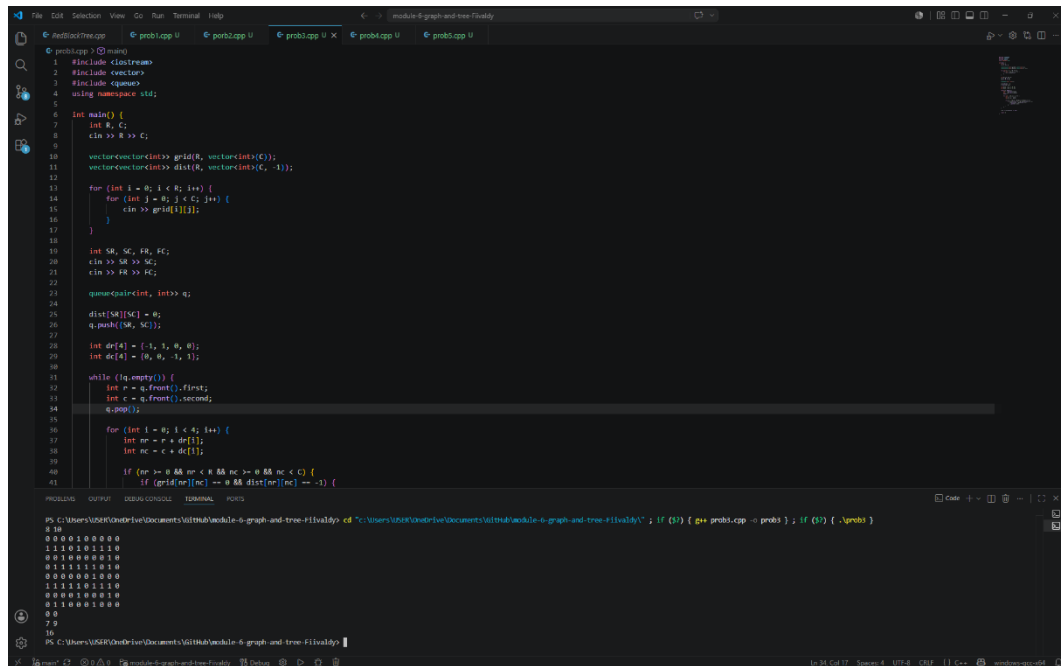
```
1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  using namespace std;
5
6  int main() {
7      int R, C;
8      cin >> R >> C;
9
10     vector<vector<int>> grid(R, vector<int>(C));
11     vector<vector<int>> dist(R, vector<int>(C, -
12 1));
13
14     for (int i = 0; i < R; i++) {
15         for (int j = 0; j < C; j++) {
16             cin >> grid[i][j];
17         }
18     }
19
20     int SR, SC, FR, FC;
21     cin >> SR >> SC;
22     cin >> FR >> FC;
23
24     queue<pair<int, int>> q;
25
26     dist[SR][SC] = 0;
27     q.push({SR, SC});
28
29     int dr[4] = {-1, 1, 0, 0};
30     int dc[4] = {0, 0, -1, 1};
```

```

30
31     while (!q.empty()) {
32         int r = q.front().first;
33         int c = q.front().second;
34         q.pop();
35
36         for (int i = 0; i < 4; i++) {
37             int nr = r + dr[i];
38             int nc = c + dc[i];
39
40             if (nr >= 0 && nr < R && nc >= 0 && nc
< C) {
41                 if (grid[nr][nc] == 0 &&
dist[nr][nc] == -1) {
42                     dist[nr][nc] = dist[r][c] + 1;
43                     q.push({nr, nc});
44                 }
45             }
46         }
47     }
48
49     cout << dist[FR][FC] << endl;
50
51     return 0;
52 }

```

B. Output Program



The screenshot shows a Visual Studio Code editor with a C++ program. The code defines a grid, calculates distances from a source, and finds the shortest path to a target. The output displays the grid with distances and the shortest path.

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 using namespace std;
5
6 int main() {
7     int R, C;
8     cin >> R >> C;
9
10    vector<vector<int>> grid(R, vector<int>(C));
11    vector<vector<int>> dist(R, vector<int>(C, -1));
12
13    for (int i = 0; i < R; i++) {
14        for (int j = 0; j < C; j++) {
15            cin >> grid[i][j];
16        }
17    }
18
19    int SR, SE, FR, FC;
20    cin >> SR >> SE;
21    cin >> FR >> FC;
22
23    queue<pair<int, int>> q;
24
25    dist[SR][SE] = 0;
26    q.push({SR, SE});
27
28    int dr[4] = {-1, 1, 0, 0};
29    int dc[4] = {0, 0, -1, 1};
30
31    while (!q.empty()) {
32        int r = q.front().first;
33        int c = q.front().second;
34        q.pop();
35
36        for (int i = 0; i < 4; i++) {
37            int nr = r + dr[i];
38            int nc = c + dc[i];
39
40            if (nr >= 0 && nr < R && nc >= 0 && nc < C) {
41                if (grid[nr][nc] == 0 && dist[nr][nc] == -1) {
```

Output:

```
PS C:\Users\USER\Documents\GithHub\module-6-graph-and-tree-Flivaldy> cd "C:\Users\USER\Documents\GithHub\module-6-graph-and-tree-Flivaldy"; if ($?) { g++ prob3.cpp -o prob3 }; if ($?) { .\prob3 }
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 0 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 1 0
0 1 1 0 0 1 0 0 0
0 0
7 9
10
```

Gambar 25 Screenshot Output Program Soal 3

C. Pembahasan

Program Soal 3 digunakan untuk mencari **jarak terpendek dari titik awal menuju titik tujuan pada sebuah grid**. Grid dapat dipahami sebagai tabel yang terdiri dari baris dan kolom. Setiap kotak pada grid dapat dianggap sebagai posisi yang bisa dilewati atau tidak bisa dilewati. Dalam program ini, nilai 0 pada grid menunjukkan jalan yang dapat dilewati, sedangkan nilai selain 0 dapat dianggap sebagai penghalang. Program menggunakan cara pencarian yang dikenal sebagai **Breadth-First Search (BFS)**, yaitu metode pencarian yang memeriksa posisi terdekat terlebih dahulu sebelum bergerak ke posisi yang lebih jauh. Cara ini cocok digunakan untuk mencari langkah paling pendek pada grid karena setiap perpindahan dihitung dengan jarak yang sama, yaitu satu langkah.

Pada bagian awal program, library yang digunakan dipanggil pada [1–3]. Library `iostream` digunakan untuk menerima input dan menampilkan output. Library `vector` digunakan untuk menyimpan data dalam bentuk tabel, yaitu grid dan jarak. Sementara itu, library `queue` digunakan untuk membuat antrean posisi yang akan diperiksa. Antrean ini penting dalam BFS karena posisi yang lebih dulu ditemukan akan diperiksa lebih dulu, sehingga pencarian berjalan secara teratur dari jarak yang paling dekat.

Pada [6–7], program membaca ukuran grid berupa jumlah baris R dan jumlah kolom C . Nilai ini menentukan besar tabel yang akan digunakan. Misalnya, jika $R = 3$ dan $C = 4$, maka grid memiliki 3 baris dan 4 kolom. Setelah itu, pada [9], program membuat variabel grid untuk menyimpan isi tabel. Kemudian pada [10], dibuat variabel `dist` dengan ukuran yang sama seperti grid. Variabel `dist` digunakan untuk menyimpan jarak dari titik awal ke setiap posisi dalam grid. Semua nilai awal pada `dist` diisi dengan -1, yang berarti posisi tersebut belum pernah dikunjungi atau belum diketahui jaraknya dari titik awal.

Bagian [12–16] digunakan untuk membaca isi grid dari input. Program membaca nilai satu per satu berdasarkan baris dan kolom. Nilai yang dimasukkan ke dalam `grid[i][j]` menunjukkan kondisi suatu kotak. Jika nilainya 0, maka kotak tersebut dapat dilewati. Jika nilainya bukan 0, maka kotak tersebut tidak akan dilewati oleh program. Dengan cara ini, program dapat membedakan mana jalur yang bisa dilalui dan mana yang menjadi penghalang.

Setelah grid dibaca, pada [18–20] program membaca posisi awal dan posisi tujuan. Posisi awal disimpan dalam SR dan SC, yaitu baris dan kolom awal. Posisi tujuan disimpan dalam FR dan FC, yaitu baris dan kolom akhir. Dalam konteks ini, program akan mencari berapa langkah paling sedikit yang dibutuhkan untuk bergerak dari posisi awal menuju posisi tujuan. Posisi tersebut ditulis menggunakan indeks baris dan kolom, sehingga program dapat langsung mengakses kotak yang dimaksud di dalam grid.

Pada [22], program membuat antrian bernama *q* yang berisi pasangan angka. Pasangan angka tersebut digunakan untuk menyimpan posisi dalam bentuk baris dan kolom. Antrian ini menjadi tempat penyimpanan sementara untuk posisi-posisi yang akan diperiksa. Pada [24], jarak posisi awal diatur menjadi 0, karena titik awal tidak membutuhkan langkah untuk sampai ke dirinya sendiri. Setelah itu, pada [25], posisi awal dimasukkan ke dalam antrian agar menjadi posisi pertama yang diperiksa oleh program.

Pada [27–28], program menyiapkan dua array, yaitu *dr* dan *dc*, yang digunakan untuk menentukan arah perpindahan. Program hanya memperbolehkan gerakan ke empat arah, yaitu atas, bawah, kiri, dan kanan. Nilai *dr* digunakan untuk perubahan baris, sedangkan *dc* digunakan untuk perubahan kolom. Misalnya, untuk bergerak ke atas, baris dikurangi satu dan kolom tetap. Untuk bergerak ke bawah, baris ditambah satu dan kolom tetap. Dengan dua array ini, program tidak perlu menulis empat pemeriksaan arah secara terpisah, karena semuanya bisa dilakukan melalui satu perulangan.

Proses utama pencarian dilakukan pada [30–45]. Selama antrian belum kosong, program akan terus mengambil posisi yang berada di bagian depan antrian. Pada [31–33], program mengambil posisi tersebut, menyimpannya ke dalam variabel *r* dan *c*, lalu menghapusnya dari antrian. Posisi ini kemudian dianggap sebagai posisi yang sedang diperiksa. Setelah itu, program melihat kemungkinan gerakan dari posisi tersebut ke empat arah menggunakan perulangan pada [35–44]. Pada [36–37], program menghitung posisi baru yang mungkin dicapai dari posisi saat ini. Posisi baru tersebut disimpan dalam *nr* dan *nc*. Nilai *nr* menunjukkan baris baru, sedangkan *nc* menunjukkan kolom baru. Setelah posisi baru dihitung, program tidak langsung mengunjunginya. Program terlebih dahulu memeriksa

apakah posisi tersebut masih berada di dalam batas grid pada [39]. Pemeriksaan ini penting agar program tidak mencoba mengakses posisi di luar tabel, misalnya baris negatif atau kolom yang lebih besar dari ukuran grid.

Jika posisi baru masih berada di dalam grid, program kemudian memeriksa dua hal pada [40]. Pertama, apakah kotak tersebut bernilai 0, yang berarti dapat dilewati. Kedua, apakah nilai `dist[nr][nc]` masih -1, yang berarti posisi tersebut belum pernah dikunjungi sebelumnya. Dua syarat ini harus terpenuhi agar posisi baru dapat diproses. Jika posisi sudah pernah dikunjungi, program tidak perlu mengunjunginya lagi, karena BFS sudah memastikan bahwa kunjungan pertama adalah jarak terpendek menuju posisi tersebut.

Jika posisi baru memenuhi syarat, maka pada [41] program mengisi jarak posisi baru dengan nilai jarak posisi saat ini ditambah satu. Artinya, untuk sampai ke posisi baru, program membutuhkan satu langkah tambahan dari posisi sebelumnya. Setelah jaraknya diperbarui, posisi baru dimasukkan ke antrean pada [42] agar nantinya juga diperiksa. Dengan cara ini, program secara bertahap menyebar dari titik awal ke posisi-posisi terdekat, lalu ke posisi yang lebih jauh. Pada bagian akhir, program menampilkan nilai `dist[FR][FC]` pada [48]. Nilai ini merupakan jarak terpendek dari titik awal menuju titik tujuan. Jika posisi tujuan dapat dicapai, maka nilai yang ditampilkan adalah jumlah langkah paling sedikit. Namun, jika posisi tujuan tidak dapat dicapai karena terhalang atau tidak ada jalur yang memungkinkan, nilai pada `dist[FR][FC]` tetap -1. Dengan demikian, output -1 berarti tidak ada jalan dari titik awal menuju titik tujuan.

Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma **Depth-First Search** (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses **backtracking** agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C
G21 G22 ... G2C
...
GR1 GR2 ... GRC
SR SC
FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

T

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

Constraints

- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5	4

Penjelasan Contoh

Posisi awal berada pada $(0, 0)$ dan posisi tujuan berada pada $(4, 5)$. Labirin memiliki lebih dari satu jalur valid serta beberapa cabang yang tidak menghasilkan solusi, misalnya cabang menuju sel $(0, 5)$.

Dengan DFS dan backtracking, seluruh jalur sederhana dari start ke finish dapat dihitung. Pada contoh ini terdapat 4 jalur valid, sehingga output yang dicetak hanya 4.

A. Source Code

Tabel 41 Source Code File prob4.cpp

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int R, C;
6  int SR, SC, FR, FC;
7  int totalPath = 0;
8
9  vector<vector<int>> grid;
10 vector<vector<bool>> visited;
11
12 int dr[4] = {-1, 1, 0, 0};
13 int dc[4] = {0, 0, -1, 1};
14
15 void dfs(int r, int c) {
16     if (r == FR && c == FC) {
17         totalPath++;
18         return;
19     }
20
21     visited[r][c] = true;
22
23     for (int i = 0; i < 4; i++) {
24         int nr = r + dr[i];
25         int nc = c + dc[i];
26
27         if (nr >= 0 && nr < R && nc >= 0 && nc <
28             C) {
29             if (grid[nr][nc] == 0 &&
30                 !visited[nr][nc]) {
```

```

29         dfs(nr, nc);
30     }
31 }
32 }
33
34     visited[r][c] = false;
35 }
36
37 int main() {
38     cin >> R >> C;
39
40     grid.assign(R, vector<int>(C));
41     visited.assign(R, vector<bool>(C, false));
42
43     for (int i = 0; i < R; i++) {
44         for (int j = 0; j < C; j++) {
45             cin >> grid[i][j];
46         }
47     }
48
49     cin >> SR >> SC;
50     cin >> FR >> FC;
51
52     dfs(SR, SC);
53
54     cout << totalPath << endl;
55
56     return 0;
57 }

```


B. Output Program

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 int R, C;
6 int SR, SC, FR, FC;
7 int totalPath = 0;
8
9 vector<vector<int>> grid;
10 vector<vector<bool>> visited;
11
12 int dr[4] = {1, 1, 0, 0};
13 int dc[4] = {0, 0, -1, 1};
14
15 void dfs(int r, int c) {
16     if (r == FR && c == FC) {
17         totalPath++;
18         return;
19     }
20
21     visited[r][c] = true;
22
23     for (int i = 0; i < 4; i++) {
24         int nr = r + dr[i];
25         int nc = c + dc[i];
26
27         if (nr >= 0 && nr < R && nc >= 0 && nc < C) {
28             if (grid[nr][nc] == 0 && !visited[nr][nc]) {
29                 dfs(nr, nc);
30             }
31         }
32     }
33
34     visited[r][c] = false;
35 }
36
37 int main() {
38     cin >> R >> C;
39
40     grid.assign(R, vector<int>(C));
41     visited.assign(R, vector<bool>(C, false));
42
43     dfs(0, 0);
44
45     cout << totalPath << endl;
46 }
```

PS C:\Users\USER\OneDrive\Documents\module-6-graph-and-tree-flivaldy> cd "C:\Users\USER\OneDrive\Documents\module-6-graph-and-tree-flivaldy"; if (\$?) { g++ prob4.cpp -o prob4 }; if (\$?) { .\prob4 }

5 5
0 0 0 1 0 0
0 1 0 0 0 2
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4

Gambar 26 Screenshot Output Program Soal 4

C. Pembahasan

Program Soal 4 digunakan untuk menghitung jumlah semua kemungkinan jalur dari titik awal menuju titik tujuan pada sebuah grid. Grid dapat dipahami sebagai tabel yang terdiri dari baris dan kolom. Setiap kotak di dalam grid memiliki nilai tertentu, di mana nilai 0 menunjukkan bahwa kotak tersebut dapat dilewati, sedangkan nilai selain 0 dianggap sebagai penghalang. Berbeda dengan program BFS sebelumnya yang mencari jarak terpendek, program ini tidak mencari jalur paling pendek, tetapi menghitung berapa banyak jalur berbeda yang dapat ditempuh dari titik awal ke titik akhir tanpa melewati kotak yang sama dalam satu jalur.

Pada bagian awal, program memanggil library `iostream` dan `vector` pada [1–2]. Setelah itu, pada [5–7], program mendeklarasikan beberapa variabel utama secara global. Variabel `R` dan `C` digunakan untuk menyimpan jumlah baris dan kolom grid. Variabel `SR` dan `SC` menyimpan posisi awal, sedangkan `FR` dan `FC` menyimpan posisi tujuan. Variabel `totalPath` digunakan untuk menghitung jumlah jalur yang berhasil mencapai titik tujuan.

Pada [9–10], program membuat dua tabel utama, yaitu `grid` dan `visited`. Tabel `grid` digunakan untuk menyimpan kondisi setiap kotak, apakah bisa dilewati atau tidak. Sementara itu, `visited` digunakan untuk menandai apakah suatu kotak sudah pernah dilewati dalam jalur yang sedang dicoba. Bagian ini penting karena program tidak boleh terus-menerus kembali ke kotak yang sama dalam satu jalur. Jika tidak ada penanda seperti `visited`, program dapat berputar tanpa henti, misalnya dari satu kotak ke kotak sebelahnya lalu kembali lagi ke kotak sebelumnya.

Pada [12–13], program menyiapkan dua array, yaitu `dr` dan `dc`, untuk mengatur arah gerakan. Program hanya mengizinkan pergerakan ke empat arah, yaitu atas, bawah, kiri, dan kanan. Array `dr` menunjukkan perubahan baris, sedangkan `dc` menunjukkan perubahan kolom. Misalnya, gerakan ke atas berarti baris dikurangi satu, sedangkan kolom tetap. Dengan adanya dua array ini, program dapat memeriksa semua arah menggunakan satu loop, sehingga alurnya lebih ringkas dan teratur.

Fungsi utama untuk mencari jalur terdapat pada fungsi `dfs` di [15–35]. Fungsi ini menggunakan metode **Depth-First Search (DFS)**, yaitu cara pencarian yang mencoba berjalan sejauh mungkin melalui satu jalur terlebih dahulu sebelum

kembali dan mencoba jalur lain. Secara sederhana, program akan memilih satu arah, terus berjalan selama masih bisa, lalu jika sudah tidak bisa lanjut, program akan mundur ke posisi sebelumnya untuk mencoba kemungkinan arah lainnya. Cara ini cocok digunakan ketika program harus menghitung semua kemungkinan jalur, bukan hanya mencari satu jalur saja.

Terdapat pemeriksaan pada [16–19]. Jika posisi saat ini sama dengan posisi tujuan, maka artinya program sudah berhasil menemukan satu jalur dari titik awal ke titik akhir. Ketika kondisi ini terjadi, nilai `totalPath` ditambah satu. Setelah itu, fungsi langsung berhenti dengan `return`, karena jalur tersebut sudah selesai dihitung. Program kemudian akan kembali ke langkah sebelumnya untuk mencari kemungkinan jalur lain.

Jika posisi saat ini belum mencapai tujuan, maka pada [21] posisi tersebut ditandai sebagai sudah dikunjungi dengan `visited[r][c] = true`. Penandaan ini berarti kotak tersebut sedang digunakan dalam jalur yang sedang dicoba. Tujuannya agar program tidak melewati kotak yang sama dua kali dalam jalur yang sama. Hal ini penting untuk menjaga agar jalur yang dihitung benar-benar jalur yang valid dan tidak berulang-ulang di tempat yang sama.

Setelah posisi ditandai, program memeriksa empat kemungkinan arah melalui perulangan pada [23–31]. Pada setiap perulangan, program menghitung posisi baru yang mungkin dituju menggunakan `nr` dan `nc` pada [24–25]. Variabel `nr` menunjukkan baris baru, sedangkan `nc` menunjukkan kolom baru. Setelah posisi baru dihitung, program memeriksa apakah posisi tersebut masih berada di dalam batas grid pada [27]. Pemeriksaan ini diperlukan agar program tidak mencoba membaca posisi di luar tabel, seperti baris negatif atau kolom yang melebihi ukuran grid.

Jika posisi baru masih berada dalam batas grid, program melanjutkan pemeriksaan pada [28]. Pada bagian ini, program memastikan bahwa kotak tersebut bernilai 0 dan belum pernah dikunjungi dalam jalur yang sedang berjalan. Nilai 0 menunjukkan bahwa kotak dapat dilewati, sedangkan `!visited[nr][nc]` memastikan bahwa kotak tersebut belum dipakai sebelumnya pada jalur yang sama. Jika kedua syarat terpenuhi, maka fungsi `dfs` dipanggil kembali pada [29] untuk melanjutkan pencarian dari posisi baru tersebut.

Pada [34], yaitu `visited[r][c] = false`. Perintah ini digunakan untuk membatalkan tanda kunjungan setelah semua kemungkinan dari posisi tersebut selesai diperiksa. Proses ini disebut sebagai langkah kembali atau **backtracking**. Secara sederhana, ketika program sudah selesai mencoba semua jalan dari suatu kotak, kotak tersebut dibuka kembali agar dapat digunakan pada jalur lain yang berbeda. Tanpa langkah ini, program hanya akan menghitung sebagian jalur saja, karena banyak kotak akan tetap dianggap sudah dikunjungi meskipun sebenarnya jalur sebelumnya sudah selesai diperiksa.

Bagian main dimulai pada [37]. Pada [38], program membaca ukuran grid berupa jumlah baris dan kolom. Setelah itu, pada [40–41], program menyiapkan ukuran grid dan `visited` sesuai dengan jumlah baris dan kolom yang dimasukkan. Semua nilai pada `visited` diatur menjadi `false`, yang berarti pada awalnya belum ada kotak yang dikunjungi. Kemudian pada [43–47], program membaca isi grid satu per satu. Data ini menjadi dasar bagi program untuk mengetahui kotak mana yang dapat dilewati dan kotak mana yang menjadi penghalang.

Pada [49–50], program membaca posisi awal dan posisi tujuan. Setelah semua data yang dibutuhkan tersedia, program memulai pencarian dengan memanggil `dfs(SR, SC)` pada [52]. Pemanggilan ini berarti program mulai mencari semua kemungkinan jalur dari posisi awal. Fungsi `dfs` akan berjalan secara berulang melalui pemanggilan dirinya sendiri sampai semua jalur yang memungkinkan selesai diperiksa.

Setelah proses pencarian selesai, program mencetak nilai `totalPath` pada [54]. Nilai ini menunjukkan jumlah seluruh jalur yang berhasil ditemukan dari titik awal menuju titik tujuan. Jika hasilnya 0, berarti tidak ada jalur yang memungkinkan, misalnya karena titik tujuan terhalang atau seluruh jalan menuju tujuan tertutup. Jika hasilnya lebih dari 0, maka angka tersebut menunjukkan banyaknya pilihan jalur yang dapat ditempuh.

Soal 5: Tree Traversal

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah **RedBlack Tree** secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.
- ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

```
N
A1 A2 ... AN
Q
T1
T2
...
TQ
```

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTree.
- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

```
[Preorder]   : daftar_angka
[Inorder]    : daftar_angka
[Postorder]  : daftar_angka
```

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Constraints

- $0 \leq N \leq 1000$
- $-10000000000 \leq A_i \leq 10000000000$
- $1 \leq Q \leq 20$
- T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Input	Output
7	[Preorder] : 50 30 20 40 70 60 80
50 30 70 20 40 60 80	[Inorder] : 20 30 40 50 60 70 80
4	[Postorder] : 20 40 30 60 80 70 50
PREORDER	[Preorder] : 50 30 20 40 70 60 80
INORDER	[Inorder] : 20 30 40 50 60 70 80
POSTORDER	[Postorder] : 20 40 30 60 80 70 50
ALL	

Penjelasan Contoh

Bilangan dimasukkan ke RBTree secara berurutan. Nilai 50 menjadi root. Nilai yang lebih kecil dari 50 masuk ke subtree kiri, sedangkan nilai yang lebih besar masuk ke subtree kanan.

Hasil INORDER pada RBTree selalu menghasilkan urutan nilai dari kecil ke besar. Pada contoh ini, hasil inorder adalah 20 30 40 50 60 70 80, sehingga struktur RBTree yang dibentuk sudah konsisten dengan aturan RBTree.

A. Source Code

Tabel 42 Source Code File RedBlackTree.h

1	<code>#ifndef RED_BLACK_TREE_H</code>
2	<code>#define RED_BLACK_TREE_H</code>
3	
4	<code>#include <cstdint></code>
5	
6	<code>class RedBlackTree {</code>
7	<code>public:</code>
8	<code>enum class Color { RED, BLACK };</code>
9	
10	<code>struct Node {</code>
11	<code>int key;</code>
12	<code>Color color;</code>
13	<code>Node* left;</code>
14	<code>Node* right;</code>
15	<code>Node* parent;</code>
16	<code>bool isNil;</code>
17	
18	<code>Node(int value, Color nodeColor, bool sentinel);</code>
19	<code>};</code>
20	
21	<code>private:</code>
22	<code>Node* rootNode;</code>
23	<code>Node* nilNode;</code>
24	<code>std::size_t nodeCount;</code>
25	
26	<code>private:</code>
27	<code>void rotateLeft(Node* node);</code>
28	<code>void rotateRight(Node* node);</code>
29	<code>void fixInsert(Node* node);</code>

30	void destroy(Node* node);
31	
32	public:
33	RedBlackTree();
34	~RedBlackTree();
35	
36	RedBlackTree(const RedBlackTree& other) = delete;
37	RedBlackTree& operator=(const RedBlackTree& other) = delete;
38	
39	void insert(int key);
40	bool contains(int key) const;
41	
42	bool lowerBound(int key, int& result) const;
43	bool upperBound(int key, int& result) const;
44	
45	const Node* root() const;
46	const Node* nil() const;
47	
48	bool empty() const;
49	std::size_t size() const;
50	
51	void clear();
52	};
53	
54	#endif

Tabel 43 Source Code File RedBlackTree.cpp

1	#include "RedBlackTree.h"
2	

	RedBlackTree::Node::Node(int value, Color
3	nodeColor, bool sentinel)
4	: key(value),
5	color(nodeColor),
6	left(nullptr),
7	right(nullptr),
8	parent(nullptr),
9	isNil(sentinel) {}
10	
11	RedBlackTree::RedBlackTree()
12	: rootNode(nullptr),
13	nilNode(nullptr),
14	nodeCount(0) {
15	nilNode = new Node(0, Color::BLACK, true);
16	nilNode->left = nilNode;
17	nilNode->right = nilNode;
18	nilNode->parent = nilNode;
19	
20	rootNode = nilNode;
21	}
22	
23	RedBlackTree::~~RedBlackTree() {
24	clear();
25	delete nilNode;
26	}
27	
28	void RedBlackTree::rotateLeft(Node* node) {
29	Node* child = node->right;
30	
31	node->right = child->left;
32	if (!child->left->isNil) {
33	child->left->parent = node;

34	}
35	
36	child->parent = node->parent;
37	
38	if (node->parent->isNil) {
39	rootNode = child;
40	} else if (node == node->parent->left) {
41	node->parent->left = child;
42	} else {
43	node->parent->right = child;
44	}
45	
46	child->left = node;
47	node->parent = child;
48	}
49	
50	void RedBlackTree::rotateRight(Node* node) {
51	Node* child = node->left;
52	
53	node->left = child->right;
54	if (!child->right->isNil) {
55	child->right->parent = node;
56	}
57	
58	child->parent = node->parent;
59	
60	if (node->parent->isNil) {
61	rootNode = child;
62	} else if (node == node->parent->right) {
63	node->parent->right = child;
64	} else {
65	node->parent->left = child;

66	}
67	
68	child->right = node;
69	node->parent = child;
70	}
71	
72	void RedBlackTree::fixInsert(Node* node) {
73	while (node->parent->color == Color::RED) {
74	Node* parent = node->parent;
75	Node* grandparent = parent->parent;
76	
77	if (parent == grandparent->left) {
78	Node* uncle = grandparent->right;
79	
80	if (uncle->color == Color::RED) {
81	parent->color = Color::BLACK;
82	uncle->color = Color::BLACK;
83	grandparent->color = Color::RED;
84	node = grandparent;
85	} else {
86	if (node == parent->right) {
87	node = parent;
88	rotateLeft(node);
89	}
90	
91	node->parent->color = Color::BLACK;
	node->parent->parent->color =
92	Color::RED;
93	rotateRight(node->parent->parent);
94	}
95	} else {
96	Node* uncle = grandparent->left;

97	
98	if (uncle->color == Color::RED) {
99	parent->color = Color::BLACK;
100	uncle->color = Color::BLACK;
101	grandparent->color = Color::RED;
102	node = grandparent;
103	} else {
104	if (node == parent->left) {
105	node = parent;
106	rotateRight(node);
107	}
108	
109	node->parent->color = Color::BLACK;
	node->parent->parent->color =
110	Color::RED;
111	rotateLeft(node->parent->parent);
112	}
113	}
114	}
115	
116	rootNode->color = Color::BLACK;
117	}
118	
119	void RedBlackTree::insert(int key) {
	Node* newNode = new Node(key, Color::RED,
120	false);
121	newNode->left = nilNode;
122	newNode->right = nilNode;
123	newNode->parent = nilNode;
124	
125	Node* parent = nilNode;
126	Node* current = rootNode;

```

127
128     while (!current->isNil) {
129         parent = current;
130
131         if (key < current->key) {
132             current = current->left;
133         } else {
134             current = current->right;
135         }
136     }
137
138     newNode->parent = parent;
139
140     if (parent->isNil) {
141         rootNode = newNode;
142     } else if (key < parent->key) {
143         parent->left = newNode;
144     } else {
145         parent->right = newNode;
146     }
147
148     nodeCount++;
149     fixInsert(newNode);
150 }
151
152 bool RedBlackTree::contains(int key) const {
153     Node* current = rootNode;
154
155     while (!current->isNil) {
156         if (key == current->key) {
157             return true;
158         }

```

```

159
160         if (key < current->key) {
161             current = current->left;
162         } else {
163             current = current->right;
164         }
165     }
166
167     return false;
168 }
169
170 bool RedBlackTree::lowerBound(int key, int&
171 result) const {
172     Node* current = rootNode;
173     Node* answer = nilNode;
174
175     while (!current->isNil) {
176         if (current->key >= key) {
177             answer = current;
178             current = current->left;
179         } else {
180             current = current->right;
181         }
182     }
183
184     if (answer->isNil) {
185         return false;
186     }
187
188     result = answer->key;
189     return true;
190 }

```

```

190     bool    RedBlackTree::upperBound(int    key,    int&
191     result) const {
192         Node* current = rootNode;
193         Node* answer = nilNode;
194
195         while (!current->isNil) {
196             if (current->key > key) {
197                 answer = current;
198                 current = current->left;
199             } else {
200                 current = current->right;
201             }
202         }
203
204         if (answer->isNil) {
205             return false;
206         }
207
208         result = answer->key;
209         return true;
210     }
211
212     const    RedBlackTree::Node*    RedBlackTree::root()
213     const {
214         return rootNode;
215     }
216
217     const    RedBlackTree::Node*    RedBlackTree::nil()
218     const {
219         return nilNode;
220     }

```

219	
220	bool RedBlackTree::empty() const {
221	return nodeCount == 0;
222	}
223	
224	std::size_t RedBlackTree::size() const {
225	return nodeCount;
226	}
227	
228	void RedBlackTree::destroy(Node* node) {
229	if (node->isNil) {
230	return;
231	}
232	
233	destroy(node->left);
234	destroy(node->right);
235	delete node;
236	}
237	
238	void RedBlackTree::clear() {
239	destroy(rootNode);
240	rootNode = nilNode;
241	nodeCount = 0;
242	}

Tabel 44 Source Code File prob5.cpp

1	#include <iostream>
2	#include <vector>
3	#include <string>
4	#include "RedBlackTree.h"
5	using namespace std;
6	

	void preorder(const RedBlackTree::Node* node,
	const RedBlackTree::Node* nil, vector<int>&
7	result) {
8	if (node == nil node->isNil) {
9	return;
10	}
11	
12	result.push_back(node->key);
13	preorder(node->left, nil, result);
14	preorder(node->right, nil, result);
15	}
16	
	void inorder(const RedBlackTree::Node* node,
	const RedBlackTree::Node* nil, vector<int>&
17	result) {
18	if (node == nil node->isNil) {
19	return;
20	}
21	
22	inorder(node->left, nil, result);
23	result.push_back(node->key);
24	inorder(node->right, nil, result);
25	}
26	
	void postorder(const RedBlackTree::Node* node,
	const RedBlackTree::Node* nil, vector<int>&
27	result) {
28	if (node == nil node->isNil) {
29	return;
30	}
31	
32	postorder(node->left, nil, result);

```

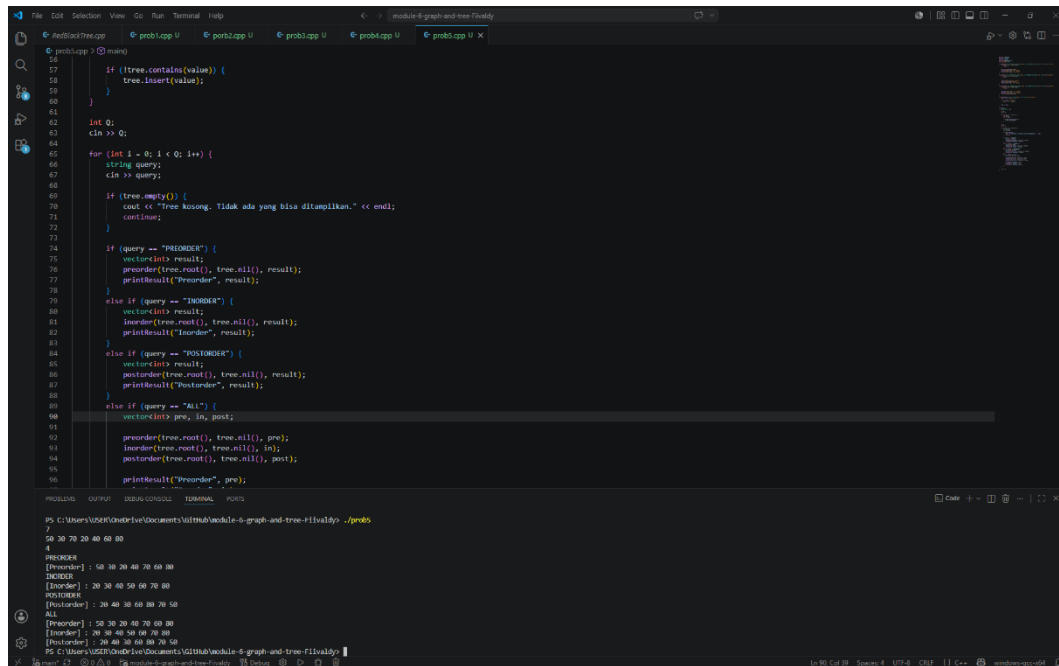
33     postorder(node->right, nil, result);
34     result.push_back(node->key);
35 }
36
37 void printResult(const string& label, const
vector<int>& result) {
38     cout << "[" << label << "]" :";
39
40     for (int value : result) {
41         cout << " " << value;
42     }
43
44     cout << endl;
45 }
46
47 int main() {
48     RedBlackTree tree;
49
50     int N;
51     cin >> N;
52
53     for (int i = 0; i < N; i++) {
54         int value;
55         cin >> value;
56
57         if (!tree.contains(value)) {
58             tree.insert(value);
59         }
60     }
61
62     int Q;
63     cin >> Q;

```

64	
65	for (int i = 0; i < Q; i++) {
66	string query;
67	cin >> query;
68	
69	if (tree.empty()) {
	cout << "Tree kosong. Tidak ada yang
70	bisa ditampilkan." << endl;
71	continue;
72	}
73	
74	if (query == "PREORDER") {
75	vector<int> result;
	preorder(tree.root(), tree.nil(),
76	result);
77	printResult("Preorder", result);
78	}
79	else if (query == "INORDER") {
80	vector<int> result;
	inorder(tree.root(), tree.nil(),
81	result);
82	printResult("Inorder", result);
83	}
84	else if (query == "POSTORDER") {
85	vector<int> result;
	postorder(tree.root(), tree.nil(),
86	result);
87	printResult("Postorder", result);
88	}
89	else if (query == "ALL") {
90	vector<int> pre, in, post;
91	

	preorder(tree.root(), tree.nil(),
92	pre);
93	inorder(tree.root(), tree.nil(), in);
	postorder(tree.root(), tree.nil(),
94	post);
95	
96	printResult("Preorder", pre);
97	printResult("Inorder", in);
98	printResult("Postorder", post);
99	}
100	}
101	
102	return 0;
103	}

B. Output Program



```
56
57     if (tree.contains(value)) {
58         tree.insert(value);
59     }
60
61
62     int Q;
63     cin >> Q;
64
65     for (int i = 0; i < Q; i++) {
66         string query;
67         cin >> query;
68
69         if (tree.empty()) {
70             cout << "Tree kosong. Tidak ada yang bisa ditampilkan." << endl;
71             continue;
72         }
73
74         if (query == "PREORDER") {
75             vector<int> result;
76             preorder(tree.root(), tree.nil(), result);
77             printResult("Preorder", result);
78         }
79         else if (query == "INORDER") {
80             vector<int> result;
81             inorder(tree.root(), tree.nil(), result);
82             printResult("Inorder", result);
83         }
84         else if (query == "POSTORDER") {
85             vector<int> result;
86             postorder(tree.root(), tree.nil(), result);
87             printResult("Postorder", result);
88         }
89         else if (query == "ALL") {
90             vector<int> pre, in, post;
91
92             preorder(tree.root(), tree.nil(), pre);
93             inorder(tree.root(), tree.nil(), in);
94             postorder(tree.root(), tree.nil(), post);
95
96             printResult("Preorder", pre);
97
98         }
99     }
100 }
```

PS C:\Users\User\OneDrive\Documents\gitHub\acchie-6-graph-and-tree-flivaldy> ./prob5

50 30 70 20 40 60 80

4

PREORDER

[Preorder] : 50 30 20 40 70 60 80

INORDER

[Inorder] : 20 30 40 50 60 70 80

POSTORDER

[Postorder] : 20 40 30 60 80 70 50

ALL

[Preorder] : 50 30 20 40 70 60 80

[Inorder] : 20 30 40 50 60 70 80

[Postorder] : 20 60 30 60 80 70 50

Gambar 27 Screenshot Output Program Soal 5

C. Pembahasan

File `RedBlackTree.h` merupakan file header yang berfungsi sebagai rancangan awal dari struktur data **Red-Black Tree**. File ini tidak berisi proses kerja secara lengkap, tetapi hanya berisi daftar komponen, fungsi, dan aturan dasar yang nantinya akan digunakan pada file implementasi, biasanya bernama `RedBlackTree.cpp`. Dengan kata lain, file ini dapat dipahami sebagai “kerangka” atau “daftar isi” dari class `RedBlackTree`, sedangkan isi langkah-langkah detail dari setiap fungsi akan ditulis di file `.cpp`.

Pada bagian [1–2], terdapat perintah `#ifndef`, `#define`, dan pada bagian akhir [49] terdapat `#endif`. Bagian ini digunakan sebagai pelindung agar isi file header tidak dibaca berulang kali oleh compiler. Dalam program C++, file header bisa saja dipanggil oleh beberapa file lain. Jika tidak ada pelindung seperti ini, isi class dapat terbaca lebih dari satu kali dan menyebabkan error. Oleh karena itu, bagian ini penting untuk memastikan bahwa definisi `RedBlackTree` hanya dimasukkan satu kali saat proses kompilasi.

Pada [4], program menyertakan library `<cstdint>`. Library ini digunakan karena program memakai tipe data `std::size_t` pada bagian jumlah node. `std::size_t` biasanya digunakan untuk menyimpan ukuran atau jumlah data karena nilainya tidak bernilai negatif. Dalam konteks Red-Black Tree, tipe ini cocok digunakan untuk menyimpan jumlah node yang ada di dalam tree.

Pada [6], yaitu deklarasi class `RedBlackTree`. Class ini dapat dipahami sebagai wadah yang menyimpan seluruh data dan fungsi yang berhubungan dengan Red-Black Tree. Red-Black Tree sendiri adalah salah satu bentuk binary search tree yang memiliki aturan tambahan berupa warna merah dan hitam pada setiap node. Tujuan dari aturan warna tersebut adalah menjaga agar tree tetap seimbang, sehingga proses pencarian, penambahan, dan beberapa operasi lain dapat berjalan lebih efisien.

Pada [8], terdapat enum class `Color { RED, BLACK }`. Bagian ini digunakan untuk membuat dua pilihan warna pada node, yaitu merah dan hitam. Dalam Red-Black Tree, warna bukan hanya sekadar penanda tampilan, tetapi menjadi bagian dari aturan keseimbangan tree. Warna membantu program menjaga

agar posisi node tidak terlalu berat ke salah satu sisi. Dengan adanya enum class, warna node menjadi lebih jelas karena hanya boleh bernilai RED atau BLACK.

Pada [10–17], program mendefinisikan struktur Node. Node adalah satu bagian atau satu elemen dari tree. Setiap node menyimpan sebuah nilai dan memiliki hubungan dengan node lain. Pada [11], key digunakan untuk menyimpan nilai utama dari node. Nilai inilah yang digunakan saat proses pencarian atau penyisipan data. Pada [12], color digunakan untuk menyimpan warna node, apakah merah atau hitam. Pada [13–15], terdapat pointer left, right, dan parent. Pointer left menunjuk ke anak kiri, right menunjuk ke anak kanan, sedangkan parent menunjuk ke induk dari node tersebut. Dengan tiga hubungan ini, setiap node dapat terhubung membentuk struktur tree.

Pada [16], terdapat variabel isNil. Variabel ini digunakan untuk menandai apakah sebuah node merupakan node khusus bernama nil. Dalam Red-Black Tree, nil biasanya digunakan sebagai penanda ujung atau daun kosong. Jadi, daripada menggunakan nilai kosong biasa, program menyiapkan node khusus untuk mewakili bagian yang tidak memiliki anak. Hal ini membuat pengaturan tree menjadi lebih rapi karena setiap ujung tree tetap memiliki penanda yang jelas.

Pada [18], terdapat constructor untuk Node, yaitu Node(int value, Color nodeColor, bool sentinel);. Constructor ini berfungsi untuk membuat node baru dengan nilai, warna, dan penanda tertentu. Parameter value digunakan untuk mengisi nilai node, nodeColor digunakan untuk menentukan warna node, dan sentinel digunakan untuk menentukan apakah node tersebut merupakan node biasa atau node nil.

Pada [21–23], terdapat tiga data utama yang dimiliki oleh class RedBlackTree, yaitu rootNode, nilNode, dan nodeCount. rootNode adalah penunjuk ke akar tree, yaitu node paling atas dalam struktur tree. Semua pencarian dan penyisipan biasanya dimulai dari root. nilNode adalah node khusus yang digunakan sebagai penanda kosong pada ujung tree. Sementara itu, nodeCount digunakan untuk menyimpan jumlah node yang ada di dalam tree. Ketiga bagian ini dibuat dalam area private, sehingga tidak dapat diakses langsung dari luar class. Hal ini bertujuan agar data utama tree tidak diubah sembarangan dari luar program.

Pada [26–29], terdapat beberapa fungsi private, yaitu `rotateLeft`, `rotateRight`, `fixInsert`, dan `destroy`. Fungsi-fungsi ini hanya digunakan di dalam class karena berhubungan dengan pengaturan internal tree. Fungsi `rotateLeft` dan `rotateRight` digunakan untuk memutar susunan node ke kiri atau ke kanan. Rotasi ini penting pada Red-Black Tree karena ketika node baru dimasukkan, posisi tree bisa menjadi tidak seimbang. Dengan rotasi, susunan tree dapat diperbaiki tanpa merusak aturan urutan nilai. Fungsi `fixInsert` digunakan setelah proses penambahan data untuk memperbaiki aturan warna dan posisi node agar tetap sesuai dengan aturan Red-Black Tree. Sementara itu, `destroy` digunakan untuk menghapus node-node dari memori, biasanya saat tree dibersihkan atau objek dihancurkan.

Pada [32], terdapat constructor `RedBlackTree()`. Fungsi ini akan dijalankan saat objek Red-Black Tree dibuat. Tugas utamanya biasanya adalah menyiapkan tree kosong, membuat node nil, mengatur root awal, dan mengatur jumlah node menjadi nol. Pada [33], terdapat destructor `~RedBlackTree()`. Destructur dijalankan saat objek sudah tidak digunakan lagi. Fungsinya adalah membersihkan memori yang sebelumnya digunakan oleh node-node di dalam tree. Ini penting karena node dibuat menggunakan pointer, sehingga memori harus dibersihkan agar tidak terjadi pemborosan memori.

Pada [35–36], terdapat dua perintah yang menghapus kemampuan penyalinan objek, yaitu copy constructor dan operator assignment. Maksudnya, objek `RedBlackTree` tidak boleh disalin secara langsung menggunakan cara biasa. Hal ini dilakukan karena tree menggunakan banyak pointer yang saling terhubung. Jika tree disalin sembarangan, bisa terjadi masalah seperti dua objek menunjuk ke node yang sama, sehingga saat salah satu objek menghapus node, objek lain bisa ikut rusak. Oleh karena itu, penyalinan dibuat tidak diperbolehkan agar struktur tree tetap aman.

Pada [38], terdapat fungsi `insert(int key)`. Fungsi ini digunakan untuk menambahkan nilai baru ke dalam Red-Black Tree. Nilai yang dimasukkan akan ditempatkan sesuai aturan binary search tree, yaitu nilai yang lebih kecil berada di sebelah kiri dan nilai yang lebih besar berada di sebelah kanan. Setelah nilai dimasukkan, biasanya program akan memanggil proses perbaikan agar aturan Red-Black Tree tetap terjaga.

Pada [39], terdapat fungsi `contains(int key) const`. Fungsi ini digunakan untuk memeriksa apakah suatu nilai terdapat di dalam tree atau tidak. Jika nilai ditemukan, fungsi akan mengembalikan `true`. Jika tidak ditemukan, fungsi akan mengembalikan `false`. Kata `const` menunjukkan bahwa fungsi ini hanya memeriksa data dan tidak mengubah isi tree.

Pada [41–42], terdapat fungsi `lowerBound` dan `upperBound`. Fungsi `lowerBound` digunakan untuk mencari nilai terkecil di dalam tree yang nilainya lebih besar atau sama dengan nilai yang dicari. Sementara itu, `upperBound` digunakan untuk mencari nilai terkecil yang nilainya benar-benar lebih besar dari nilai yang dicari. Hasil pencarian disimpan ke dalam parameter `result`. Kedua fungsi ini berguna ketika program tidak hanya ingin mengetahui apakah sebuah nilai ada, tetapi juga ingin mencari nilai terdekat berdasarkan urutan.

Pada [44–45], terdapat fungsi `root()` dan `nil()`. Fungsi `root()` digunakan untuk mengakses node akar dari tree, sedangkan `nil()` digunakan untuk mengakses node khusus `nil`. Keduanya mengembalikan pointer yang bersifat `const`, sehingga pengguna dapat melihat data tersebut tetapi tidak dapat mengubahnya secara langsung. Hal ini membuat data internal tree tetap aman.

Pada [47–48], terdapat fungsi `empty()` dan `size()`. Fungsi `empty()` digunakan untuk memeriksa apakah tree masih kosong atau sudah memiliki node. Jika tree kosong, fungsi akan mengembalikan `true`. Fungsi `size()` digunakan untuk mengetahui jumlah node yang ada di dalam tree. Kedua fungsi ini membantu pengguna mengetahui kondisi tree tanpa harus memeriksa node satu per satu.

Pada [50], terdapat fungsi `clear()`. Fungsi ini digunakan untuk menghapus seluruh isi tree dan mengembalikannya ke kondisi kosong. Fungsi ini biasanya akan bekerja dengan memanggil fungsi `destroy` untuk menghapus node-node yang ada, lalu mengatur ulang `root` dan jumlah node. Dengan adanya fungsi ini, pengguna dapat mengosongkan tree tanpa harus membuat objek baru.

File **RedBlackTree.cpp** merupakan file implementasi dari rancangan yang sebelumnya sudah dibuat pada **RedBlackTree.h**. Jika file .h berisi daftar fungsi dan bentuk struktur datanya, maka file .cpp ini berisi isi nyata dari fungsi-fungsi tersebut.

Pada bagian awal [1], program memanggil file RedBlackTree.h. Hal ini dilakukan karena semua nama class, struktur Node, warna node, dan daftar fungsi sudah dideklarasikan di dalam file header tersebut. File .cpp ini tidak membuat rancangan baru, tetapi melengkapi isi dari rancangan yang sudah ada. Dengan cara ini, program menjadi lebih rapi karena bagian deklarasi dan bagian implementasi dipisahkan.

Bagian [3–9] berisi constructor untuk Node. Constructor ini digunakan ketika sebuah node baru dibuat. Setiap node memiliki nilai utama yang disimpan pada key, warna yang disimpan pada color, serta hubungan ke node lain melalui left, right, dan parent. Pada awal dibuat, hubungan kiri, kanan, dan induknya masih diisi nullptr karena belum disambungkan ke tree. Selain itu, variabel isNil digunakan untuk membedakan apakah node tersebut adalah node biasa atau node khusus penanda kosong. Node khusus ini penting dalam Red-Black Tree karena ujung-ujung tree tidak dibiarkan benar-benar kosong, tetapi ditandai menggunakan node khusus yang disebut nilNode.

Bagian [11–21] merupakan constructor untuk class RedBlackTree. Fungsi ini berjalan otomatis ketika objek Red-Black Tree dibuat. Pada awalnya, rootNode, nilNode, dan nodeCount disiapkan terlebih dahulu. Nilai nodeCount dibuat 0 karena tree masih kosong. Setelah itu, program membuat nilNode dengan warna hitam. Dalam Red-Black Tree, node kosong atau penanda ujung biasanya dianggap berwarna hitam agar aturan tree tetap terjaga. Kemudian nilNode diarahkan ke dirinya sendiri pada bagian kiri, kanan, dan parent. Hal ini membuat program lebih aman ketika memeriksa node kosong, karena nilNode tetap memiliki alamat yang valid dan tidak menyebabkan kesalahan akses memori. Setelah nilNode siap, rootNode diarahkan ke nilNode, yang berarti tree pada awalnya belum memiliki data.

Bagian [23–26] adalah destructor dari RedBlackTree. Destructor berjalan ketika objek Red-Black Tree sudah tidak digunakan lagi. Di dalamnya, program

memanggil `clear()` untuk menghapus semua node biasa yang ada di dalam tree. Setelah itu, `nilNode` juga dihapus dari memori. Bagian ini penting karena struktur tree menggunakan `new` untuk membuat node, sehingga memori yang digunakan harus dibersihkan secara manual agar tidak terjadi pemborosan memori.

Fungsi `rotateLeft` pada [28–49] digunakan untuk melakukan rotasi ke kiri. Rotasi dapat dipahami sebagai proses mengubah susunan node tanpa mengubah urutan nilai di dalam tree. Pada Red-Black Tree, rotasi diperlukan ketika penambahan node baru membuat susunan tree menjadi kurang seimbang atau melanggar aturan warna. Dalam rotasi kiri, anak kanan dari sebuah node akan naik menggantikan posisi node tersebut, sedangkan node lama akan turun ke sisi kiri dari anak tersebut. Walaupun posisi bentuk tree berubah, urutan nilai tetap benar: nilai yang lebih kecil tetap berada di kiri, dan nilai yang lebih besar tetap berada di kanan. Pada [29], program mengambil anak kanan dari node yang sedang diputar dan menyimpannya dalam variabel `child`. Anak kanan ini nantinya akan naik menggantikan posisi node. Pada [31–34], bagian kiri dari `child` dipindahkan menjadi anak kanan dari node. Jika bagian kiri `child` bukan `nilNode`, maka `parent`-nya diperbarui agar menunjuk ke node. Setelah itu, pada [36–44], program menghubungkan `child` dengan `parent` lama dari node. Jika node sebelumnya adalah root, maka root diganti menjadi `child`. Jika node adalah anak kiri atau anak kanan dari `parent`-nya, maka hubungan tersebut juga disesuaikan. Terakhir, pada [46–47], node dijadikan anak kiri dari `child`, dan `parent` dari node diubah menjadi `child`. Dengan langkah ini, rotasi kiri selesai dilakukan.

Fungsi `rotateRight` pada [51–72] memiliki tujuan yang mirip dengan `rotateLeft`, tetapi arah perubahannya berlawanan. Pada rotasi kanan, anak kiri dari sebuah node akan naik menggantikan posisi node tersebut, sedangkan node lama turun menjadi anak kanan dari anak tersebut. Fungsi ini digunakan ketika struktur tree perlu diperbaiki ke arah kanan. Pada [52], program mengambil anak kiri sebagai `child`. Kemudian pada [54–57], anak kanan dari `child` dipindahkan menjadi anak kiri dari node. Setelah itu, hubungan `parent` diperbaiki pada [59–67], lalu pada [69–70], node dijadikan anak kanan dari `child`. Jadi, rotasi kanan membantu menjaga susunan tree tetap seimbang tanpa merusak aturan urutan nilai.

Fungsi `fixInsert` pada [74–119] merupakan salah satu bagian paling penting dalam Red-Black Tree. Fungsi ini digunakan setelah sebuah node baru dimasukkan ke dalam tree. Dalam Red-Black Tree, node baru biasanya diberi warna merah. Namun, penambahan node merah bisa menyebabkan aturan tree dilanggar, terutama jika parent dari node tersebut juga berwarna merah. Oleh karena itu, fungsi `fixInsert` bertugas memperbaiki warna dan posisi node agar tree kembali memenuhi aturan Red-Black Tree.

Pada [75], program menjalankan perulangan selama parent dari node yang sedang diperiksa berwarna merah. Kondisi ini menunjukkan adanya pelanggaran aturan, karena dalam Red-Black Tree node merah tidak boleh memiliki parent yang juga merah. Pada [76–77], program mengambil parent dan grandparent dari node. Grandparent adalah induk dari parent. Informasi ini dibutuhkan karena perbaikan biasanya melibatkan tiga posisi utama, yaitu node baru, parent, dan grandparent. Pada [79–101], program menangani kondisi ketika parent berada di sisi kiri grandparent. Dalam kondisi ini, program juga mengambil uncle, yaitu saudara dari parent, yang berada di sisi kanan grandparent. Jika uncle berwarna merah, seperti pada [82], maka masalah dapat diselesaikan dengan mengubah warna. Parent dan uncle diubah menjadi hitam, sedangkan grandparent diubah menjadi merah. Setelah itu, node yang sedang diperiksa dinaikkan ke grandparent agar program dapat memeriksa apakah perubahan warna tersebut menyebabkan masalah baru di bagian atas tree.

Jika uncle berwarna hitam, maka perbaikan tidak cukup hanya dengan mengganti warna. Program perlu melakukan rotasi. Pada [88–91], jika node berada di sisi kanan parent, maka program melakukan rotasi kiri terlebih dahulu agar bentuknya menjadi lebih mudah diperbaiki. Setelah itu, pada [94–96], parent diubah menjadi hitam, grandparent diubah menjadi merah, lalu dilakukan rotasi kanan pada grandparent. Langkah ini membuat susunan tree kembali lebih seimbang dan aturan warna dapat dipenuhi.

Bagian [102–115] menangani kondisi sebaliknya, yaitu ketika parent berada di sisi kanan grandparent. Cara kerjanya sama, tetapi arah kiri dan kanan dibalik. Jika uncle berwarna merah, program cukup mengganti warna parent, uncle, dan grandparent. Jika uncle berwarna hitam, program melakukan rotasi kanan atau

rotasi kiri sesuai posisi node. Dengan adanya dua bagian ini, fungsi `fixInsert` dapat memperbaiki tree baik ketika masalah terjadi di sisi kiri maupun sisi kanan.

Pada [118], root selalu diubah menjadi hitam. Ini adalah aturan penting dalam Red-Black Tree, yaitu akar tree harus berwarna hitam. Walaupun proses perbaikan sebelumnya bisa mengubah warna beberapa node, bagian akhir ini memastikan bahwa root tetap sesuai aturan. Dengan demikian, setelah `fixInsert` selesai, tree kembali berada dalam keadaan yang valid.

Fungsi `insert` pada [121–154] digunakan untuk memasukkan nilai baru ke dalam Red-Black Tree. Pada [122], program membuat node baru dengan nilai key, warna merah, dan penanda bahwa node tersebut bukan `nilNode`. Anak kiri, anak kanan, dan parent dari node baru diatur ke `nilNode` pada [123–125]. Warna merah diberikan karena dalam Red-Black Tree, node baru umumnya dimulai sebagai node merah agar tidak langsung menambah jumlah node hitam pada jalur tree.

Pada [127–129], program menyiapkan dua pointer, yaitu `parent` dan `current`. `current` digunakan untuk menelusuri tree dari root, sedangkan `parent` menyimpan node terakhir sebelum posisi kosong ditemukan. Perulangan pada [131–139] digunakan untuk mencari tempat yang tepat bagi node baru. Jika nilai yang dimasukkan lebih kecil dari nilai node saat ini, program bergerak ke kiri. Jika nilainya lebih besar atau sama, program bergerak ke kanan. Cara ini mengikuti aturan binary search tree, yaitu nilai kecil berada di kiri dan nilai besar berada di kanan.

Setelah posisi kosong ditemukan, pada [141] `parent` dari node baru diatur sesuai hasil pencarian. Jika `parent` masih `nilNode`, berarti tree sebelumnya kosong, sehingga node baru menjadi root seperti pada [143–144]. Jika tree tidak kosong, program menentukan apakah node baru menjadi anak kiri atau anak kanan dari `parent` berdasarkan perbandingan nilainya pada [145–149]. Setelah node berhasil dipasang, jumlah node ditambah pada [152], lalu `fixInsert(newNode)` dipanggil pada [153] untuk memperbaiki aturan Red-Black Tree. Jadi, proses `insert` tidak hanya menaruh data baru, tetapi juga menjaga agar tree tetap seimbang.

Fungsi `contains` pada [156–174] digunakan untuk memeriksa apakah suatu nilai terdapat di dalam tree. Program memulai pencarian dari `rootNode`. Selama node yang diperiksa bukan `nilNode`, program membandingkan nilai yang dicari

dengan nilai node saat ini. Jika sama, fungsi langsung mengembalikan true, yang berarti nilai ditemukan. Jika nilai yang dicari lebih kecil, pencarian bergerak ke kiri. Jika lebih besar, pencarian bergerak ke kanan. Jika pencarian akhirnya mencapai nilNode, artinya nilai tidak ditemukan, sehingga fungsi mengembalikan false. Fungsi ini tidak mengubah isi tree, hanya memeriksa keberadaan data.

Fungsi lowerBound pada [176–197] digunakan untuk mencari nilai terkecil di dalam tree yang nilainya lebih besar atau sama dengan nilai yang dicari. Pada awalnya, answer diisi dengan nilNode, yang berarti belum ada jawaban. Selama pencarian berlangsung, jika nilai node saat ini lebih besar atau sama dengan key, maka node tersebut menjadi calon jawaban. Namun, program masih bergerak ke kiri untuk mencari kemungkinan nilai yang lebih kecil tetapi tetap memenuhi syarat. Jika nilai node saat ini lebih kecil dari key, program bergerak ke kanan karena nilai yang dibutuhkan pasti lebih besar. Setelah pencarian selesai, jika answer masih nilNode, berarti tidak ada nilai yang memenuhi syarat. Jika ada, hasilnya disimpan ke dalam result, lalu fungsi mengembalikan true.

Fungsi upperBound pada [199–220] hampir sama dengan lowerBound, tetapi syaratnya sedikit berbeda. Fungsi ini mencari nilai terkecil yang benar-benar lebih besar dari key, bukan sama atau lebih besar. Perbedaannya terlihat pada [204], yaitu kondisi $\text{current->key} > \text{key}$. Jika nilai node lebih besar dari nilai yang dicari, node tersebut menjadi calon jawaban dan pencarian dilanjutkan ke kiri. Jika tidak, pencarian bergerak ke kanan. Fungsi ini berguna ketika program membutuhkan nilai berikutnya setelah suatu nilai tertentu.

Fungsi root pada [222–224] digunakan untuk mengembalikan alamat node root. Fungsi ini memberi akses kepada bagian luar program untuk melihat akar tree. Namun, karena hasilnya bertipe `const Node*`, pengguna hanya dapat melihat data tersebut dan tidak dapat mengubahnya secara langsung. Fungsi nil pada [226–228] memiliki tujuan serupa, yaitu mengembalikan node khusus nilNode. Kedua fungsi ini membantu program lain membaca struktur tree tanpa merusak isi internalnya. Fungsi empty pada [230–232] digunakan untuk memeriksa apakah tree kosong. Jika nodeCount bernilai 0, maka fungsi mengembalikan true. Artinya, belum ada data biasa yang tersimpan di dalam tree. Fungsi size pada [234–236] digunakan untuk mengembalikan jumlah node yang ada di dalam tree. Nilai ini berasal dari

nodeCount, yang bertambah setiap kali data baru berhasil dimasukkan melalui fungsi insert.

Fungsi destroy pada [238–246] digunakan untuk menghapus node dari memori. Fungsi ini bekerja secara bertahap dari suatu node, lalu menghapus anak kiri dan anak kanannya terlebih dahulu. Jika node yang diperiksa adalah nilNode, fungsi langsung berhenti karena nilNode tidak boleh dihapus oleh fungsi ini. Setelah anak kiri dan kanan selesai dihapus, node saat ini baru dihapus dengan delete. Cara ini penting karena jika parent dihapus terlebih dahulu, program bisa kehilangan akses ke anak-anaknya.

Fungsi clear pada [248–252] digunakan untuk menghapus seluruh isi tree. Fungsi ini memanggil destroy(rootNode) untuk menghapus semua node dari root sampai ke bawah. Setelah semua node biasa dihapus, rootNode dikembalikan ke nilNode, dan nodeCount diatur kembali menjadi 0. Dengan demikian, tree kembali ke keadaan kosong seperti saat pertama kali dibuat, tetapi nilNode tetap tersedia untuk digunakan kembali.

File **prob5.cpp** digunakan untuk membuat sebuah **Red-Black Tree**, memasukkan beberapa nilai ke dalam tree, lalu menampilkan isi tree menggunakan beberapa cara penelusuran, yaitu **preorder**, **inorder**, dan **postorder**. Program ini bergantung pada file **RedBlackTree.h**, sehingga proses utama seperti membuat tree, memasukkan data, mengecek data, dan mengambil root sudah disediakan oleh class **RedBlackTree**. File ini lebih berfokus pada cara membaca input, mencegah data yang sama masuk dua kali, serta menampilkan isi tree berdasarkan perintah dari pengguna.

Pada bagian awal [1–4], program memanggil beberapa library dan file pendukung. Library **iostream** digunakan untuk menerima input dan menampilkan output, **vector** digunakan untuk menyimpan hasil penelusuran tree, dan **string** digunakan untuk membaca perintah seperti **PREORDER**, **INORDER**, **POSTORDER**, atau **ALL**. Sementara itu, **RedBlackTree.h** dipanggil agar program dapat menggunakan class **RedBlackTree** yang sudah dibuat sebelumnya. Dengan adanya file header tersebut, program ini tidak perlu menulis ulang cara kerja **Red-Black Tree** dari awal.

Fungsi **preorder** pada [7–15] digunakan untuk menelusuri tree dengan urutan **root**, **kiri**, **kanan**. Maksudnya, program akan membaca nilai node yang sedang dikunjungi terlebih dahulu, lalu melanjutkan ke anak kiri, kemudian ke anak kanan. Pada [8–10], program memeriksa apakah node yang sedang dikunjungi adalah nil atau node kosong. Jika iya, fungsi langsung berhenti agar program tidak mencoba membaca bagian yang sebenarnya tidak berisi data. Jika node tersebut valid, nilainya dimasukkan ke dalam **result** pada [12], lalu program melanjutkan penelusuran ke kiri dan kanan pada [13–14]. Cara ini membuat nilai paling atas dari tree akan muncul lebih dulu dalam hasil.

Fungsi **inorder** pada [17–25] digunakan untuk menelusuri tree dengan urutan **kiri**, **root**, **kanan**. Pada **Red-Black Tree** yang juga mengikuti aturan **Binary Search Tree**, penelusuran **inorder** biasanya menghasilkan data dalam urutan menaik. Program terlebih dahulu menelusuri bagian kiri pada [22], kemudian menyimpan nilai node saat ini pada [23], lalu menelusuri bagian kanan pada [24]. Dengan pola seperti ini, nilai yang lebih kecil akan dikunjungi lebih dahulu, kemudian nilai

tengah, lalu nilai yang lebih besar. Karena itulah, hasil inorder sangat berguna untuk melihat apakah data di dalam tree tersusun dengan benar.

Fungsi postorder pada [27–35] digunakan untuk menelusuri tree dengan urutan **kiri, kanan, root**. Artinya, program akan mengunjungi anak kiri terlebih dahulu, kemudian anak kanan, dan nilai node utama dimasukkan paling akhir. Pada [32–34], urutan tersebut terlihat jelas karena fungsi memanggil bagian kiri, lalu bagian kanan, baru kemudian menyimpan nilai node ke dalam result. Cara penelusuran ini sering digunakan ketika ingin memproses bagian bawah tree terlebih dahulu sebelum node induknya.

Fungsi printResult pada [37–45] digunakan untuk menampilkan hasil penelusuran tree secara rapi. Fungsi ini menerima dua data, yaitu label sebagai nama jenis penelusuran dan result sebagai daftar nilai hasil penelusuran. Pada [38], program menampilkan label seperti [Preorder] :. Setelah itu, pada [40–42], program mencetak seluruh nilai yang ada di dalam result satu per satu. Dengan fungsi ini, program tidak perlu menulis ulang cara mencetak hasil untuk preorder, inorder, dan postorder.

Pada fungsi main di [47]. Pada [48], program membuat objek tree dari class RedBlackTree. Objek inilah yang akan menyimpan seluruh nilai yang dimasukkan pengguna. Setelah itu, pada [50–51], program membaca jumlah data yang akan dimasukkan, yaitu N. Nilai N menentukan berapa banyak angka yang akan dibaca dan dicoba dimasukkan ke dalam tree.

Pada [53–61], program membaca nilai sebanyak N kali. Setiap nilai disimpan sementara dalam variabel value. Sebelum nilai tersebut dimasukkan ke dalam tree, program memeriksa terlebih dahulu apakah nilai tersebut sudah ada menggunakan tree.contains(value) pada [57]. Jika nilai belum ada, barulah program memasukkannya dengan tree.insert(value) pada [58]. Pemeriksaan ini penting karena program ingin menghindari data ganda. Dengan begitu, jika pengguna memasukkan angka yang sama lebih dari satu kali, angka tersebut hanya akan disimpan sekali di dalam tree.

Setelah proses input data selesai, program membaca jumlah perintah pada [64–65]. Jumlah perintah ini disimpan dalam variabel Q. Kemudian pada [67–99], program melakukan perulangan sebanyak Q kali untuk membaca dan menjalankan

setiap perintah. Setiap perintah disimpan dalam variabel query. Perintah inilah yang menentukan jenis penelusuran apa yang akan ditampilkan oleh program.

Sebelum menjalankan perintah, program memeriksa apakah tree kosong pada [71–74]. Jika tree kosong, maka program menampilkan pesan bahwa tidak ada data yang bisa ditampilkan, lalu melanjutkan ke perintah berikutnya. Pemeriksaan ini penting agar program tidak mencoba menelusuri tree yang belum memiliki data. Walaupun Red-Black Tree memiliki nilNode sebagai penanda kosong, dari sisi pengguna tetap perlu diberi pesan yang jelas bahwa tree belum berisi nilai.

Jika input yang dimasukkan adalah PREORDER, maka bagian [77–80] akan dijalankan. Program membuat vector kosong bernama result, kemudian memanggil fungsi preorder dengan root tree sebagai titik awal. Setelah proses penelusuran selesai, hasilnya dicetak menggunakan printResult. Dengan perintah ini, pengguna dapat melihat isi tree dimulai dari node paling atas, kemudian turun ke bagian kiri dan kanan.

Jika input yang dimasukkan adalah INORDER, maka bagian [82–85] akan dijalankan. Program membuat vector result, memanggil fungsi inorder, lalu mencetak hasilnya. Hasil inorder pada tree seperti ini biasanya berbentuk urutan angka dari kecil ke besar. Oleh karena itu, perintah INORDER dapat membantu pengguna melihat data yang tersimpan secara teratur.

Jika input yang dimasukkan adalah POSTORDER, maka bagian [87–90] akan dijalankan. Program menjalankan fungsi postorder, kemudian mencetak hasilnya. Penelusuran ini menunjukkan urutan pembacaan dari bagian bawah tree terlebih dahulu, lalu berakhir pada node paling atas. Dengan begitu, pengguna dapat membandingkan perbedaan pola penelusuran antara preorder, inorder, dan postorder.

Jika input yang dimasukkan adalah ALL, maka bagian [92–99] akan dijalankan. Program membuat tiga vector sekaligus, yaitu pre, in, dan post, untuk menyimpan hasil dari tiga jenis penelusuran. Setelah itu, program menjalankan preorder, inorder, dan postorder secara berurutan, lalu mencetak semuanya. Perintah ALL berguna ketika pengguna ingin melihat seluruh bentuk hasil penelusuran tanpa harus mengetik tiga perintah secara terpisah.

TAUTAN GITHUB

<https://github.com/Fiivaldy/PRAKTIKUM-ALGORITMA-DAN-STRUKTUR-DATA.git>

